



**SAPIENZA**  
UNIVERSITÀ DI ROMA

---

**TESINA RELATIVA**

**AL CORSO:**

**LINGUAGGI PER IL WEB**

**SISTEMA INFORMATICO PER LA GESTIONE  
DI UNA PALESTRA “PALESTRW”**

**Studenti:**

Riccardo D’Annibale [REDACTED]  
[REDACTED]

---

**CORSO DI LAUREA TRIENNALE IN INGEGNERIA DELL’INFORMAZIONE**

**A.A. 2025/2026**

## Sommario

|                                                                                                                                                                                                                                                                                                                                                                              |    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Capitolo 1 – Introduzione e descrizione dei dati .....                                                                                                                                                                                                                                                                                                                       | 9  |
| Introduzione .....                                                                                                                                                                                                                                                                                                                                                           | 9  |
| L'applicazione web "PALESTRW" è una piattaforma digitale progettata per semplificare la gestione di una palestra. Essa consente l'interazione tra clienti e amministratori, automatizzando processi come la registrazione utenti, la gestione degli abbonamenti, la prenotazione dei corsi, la comunicazione interna, la gestione delle FAQ e la messaggistica privata. .... |    |
| Questa sezione descrive in dettaglio: .....                                                                                                                                                                                                                                                                                                                                  | 9  |
| Note.....                                                                                                                                                                                                                                                                                                                                                                    | 9  |
| In questa sezione si chiariscono alcuni concetti chiave menzionati in introduzione: .....                                                                                                                                                                                                                                                                                    |    |
| Abbonamento: Contratto tra utente e palestra con durata definita (mensile, trimestrale, annuale), vincoli di utilizzo (numero di prenotazioni settimanali), date di inizio e scadenza, stati (attivo, scaduto). ....                                                                                                                                                         |    |
| Promozione:.....                                                                                                                                                                                                                                                                                                                                                             | 9  |
| Offerta temporanea gestita dall'amministratore, associata a un codice univoco, con validità definita da data di inizio e fine e condizioni d'uso specifiche.....                                                                                                                                                                                                             |    |
| Valutazione/Statistica: .....                                                                                                                                                                                                                                                                                                                                                | 9  |
| Sistema di feedback su base numerica (scala da 1 a 5); le valutazioni vengono aggregate per fornire statistiche utili al miglioramento dei corsi. ....                                                                                                                                                                                                                       |    |
| Messaggistica privata: .....                                                                                                                                                                                                                                                                                                                                                 | 9  |
| Conversazioni tra staff e utente con abbonamento attivo. ....                                                                                                                                                                                                                                                                                                                | 10 |
| Partecipazione alla lezione: Un utente abbonato è considerato partecipante a una lezione se ha effettuato la prenotazione, non l'ha disdetta e la data della lezione è trascorsa. Solo in questo caso gli viene consentito di esprimere una valutazione. ....                                                                                                                |    |
| Reputazione: Punteggio dinamico (0-100) basato sulla qualità dei contributi nel forum e nelle valutazioni, calcolato tramite algoritmo che considera consensi ricevuti e attività recente. ....                                                                                                                                                                              |    |
| Validazione feedback: Gli utenti possono valutare solo commenti di altri membri (non propri) entro 30 giorni dalla pubblicazione. ....                                                                                                                                                                                                                                       |    |
| <b>Tipologie di Utenti</b> .....                                                                                                                                                                                                                                                                                                                                             | 10 |
| <b>Menu Utente (in base al ruolo)</b> .....                                                                                                                                                                                                                                                                                                                                  | 10 |
| Funzionalità e Casi d'Uso .....                                                                                                                                                                                                                                                                                                                                              | 12 |
| Registrazione .....                                                                                                                                                                                                                                                                                                                                                          | 12 |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |    |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Quando un visitatore arriva su PALESTRW, può avviare subito la procedura di registrazione cliccando sul pulsante dedicato(Area riservata). Viene quindi presentato un form in cui inserire username, email ,password, indirizzo e telefono. Il sistema controlla lato client il formato dell'email, quindi invia i dati al server, dove viene verificato che l'indirizzo email non sia già registrato. In caso di duplicato, l'utente riceve un messaggio di errore con suggerimento di recupero credenziali. Se tutto è corretto, la password viene hashata (bcrypt) e si crea un nuovo record nella tabella "users" con timestamp di registrazione. Dopo la registrazione, l'utente può accedere alla propria area personale, ma non potrà ancora prenotare corsi e altre funzionalità fino all'attivazione di un abbonamento. .... | 12 |
| Accesso all'area riservata .....                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | 12 |
| Un utente già registrato può effettuare il login inserendo email/username e password. Il backend recupera il record corrispondente da "users" e confronta la password tramite bcrypt: in caso di esito positivo, viene creata una sessione sicura e l'utente viene reindirizzato alla home page, con accesso alle sezioni riservate per utenti.....                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | 12 |
| Il sistema distingue due categorie di utenti, ciascuna con permessi e funzionalità differenti: .....                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 12 |
| ✓ Utente senza abbonamento: accesso ristretto, limitato alle operazioni essenziali (es. visualizza il profilo) .....                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 12 |
| ✓ Utente con abbonamento attivo: accesso completo a funzionalità avanzate, come prenotazione corsi, storico prenotazioni, messaggistica privata con lo staff. ....                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | 12 |
| Prenotazione corsi.....                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | 12 |
| Solo gli utenti con abbonamento attivo possono prenotare una lezione. Dalla pagina dei corsi, il sistema legge corsi_base.xml per mostrare titolo, descrizione, orari e posti disponibili. Al clic su "Prenota", verifica lo stato dell'abbonamento e la disponibilità di posti: se validi, aggiunge un tag <prenotante> in lezioni.xml e conferma la prenotazione; altrimenti mostra un messaggio di errore ("Login+Abbonamento" o "Posti esauriti"). ...                                                                                                                                                                                                                                                                                                                                                                            | 13 |
| Tentativo di prenotazione senza abbonamento .....                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | 13 |
| Se un utente registrato ma senza abbonamento attivo tenta di prenotare, l'operazione viene bloccata dal backend, migliorando l'esperienza utente e guidando verso azioni consentite.....                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | 13 |
| Visualizzazione corsi e promozioni.....                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | 13 |
| Tutti gli utenti (anche non registrati) possono esplorare l'offerta formativa. Ogni corso presenta una scheda con dettagli e posti disponibili. Nell'area promozioni vengono visualizzate le offerte attive, con titolo, descrizione, condizioni, codice sconto e limitazioni. ....                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | 13 |
| Visualizzazione stato abbonamento.....                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 13 |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Nell'area riservata, l'utente trova il modulo "abbonamento", nella sezione profilo, che legge user_abbonamenti.xml e mostra tipo di piano, date di inizio/fine e stato ("Attivo", "Scaduto" o "in scadenza").                                                                                                                                                                                                                                                                                                                                              | 13 |
| Consultazione FAQ                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 13 |
| La sezione FAQ è basata su faq.xml: un elenco di domande cliccabili che mostrano la risposta in modo animato; è presente un campo di ricerca. Gli amministratori possono aggiungere, modificare o eliminare domande.                                                                                                                                                                                                                                                                                                                                       | 13 |
| Valutazione corsi                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 13 |
| Dopo la partecipazione, Gli utenti abbonati possono lasciare:                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | 13 |
| • Voto numerico (1-5 stelle)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | 13 |
| Il sistema verifica la partecipazione in corsi.xml e salva il voto in feedback.xml.                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 13 |
| Messaggistica privata                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | 13 |
| Utenti e staff scambiano messaggi in private_chat.xml. Ogni messaggio crea o aggiorna un nodo cronologico; lo staff può rispondere dall'area dedicata. Se dopo due giorni non arriva risposta, il sistema invia automaticamente un messaggio di sollecito all'utente.                                                                                                                                                                                                                                                                                      | 14 |
| Gestione corsi e lezioni (Admin)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | 14 |
| L'amministratore accede a un pannello avanzato per creare, modificare o rimuovere corsi; definire sessioni, orari, limiti partecipanti.                                                                                                                                                                                                                                                                                                                                                                                                                    | 14 |
| Gestione promozioni (Admin)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 14 |
| Modulo per creare o eliminare promozioni, definendo titolo, descrizione, codice sconto, date di validità e limiti; le promozioni sono visibili nella sezione pubblica.                                                                                                                                                                                                                                                                                                                                                                                     | 14 |
| Gestione utenti (Admin)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | 14 |
| Elenco completo di utenti abbonati attivi, con dettagli personali, tipo di abbonamento.                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | 14 |
| Gestione prenotazioni (Admin)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | 14 |
| Dashboard delle prenotazioni con liste partecipanti; possibilità di inviare notifiche in caso di cancellazioni o overbooking, e di analizzare i dati per previsioni future.                                                                                                                                                                                                                                                                                                                                                                                | 14 |
| Comunicazioni dallo staff (Admin) L'amministratore può inviare comunicazioni visibili a tutti gli utenti, utili per segnalare modifiche importanti, aggiornamenti del sistema o per condividere avvisi urgenti.                                                                                                                                                                                                                                                                                                                                            | 14 |
| Sistema di Reputazione                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | 14 |
| Il sistema di reputazione assegna a ciascun utente un punteggio che riflette il valore dei suoi contributi nel forum. Questo punteggio viene calcolato tenendo conto di quattro elementi principali: la qualità delle valutazioni ricevute sui commenti, il livello di partecipazione alle discussioni, l'utilità dei contenuti riconosciuta dalla community e l'attività recente dell'utente. In particolare, il sistema di reputazione utilizza un algoritmo che considera l'ultima attività dell'utente per calcolare il decadimento della reputazione, |    |

|                                                                                                                                                                                                                                                                                                                     |    |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| identificare utenti inattivi e aggiornare dinamicamente i punteggi. La reputazione è visibile nel profilo utente e incide sulla visibilità dei propri interventi: contributi provenienti da utenti con alta reputazione tendono infatti ad avere maggiore rilievo all'interno del forum. ....                       | 15 |
| Forum Pubblico .....                                                                                                                                                                                                                                                                                                | 15 |
| Gli utenti abbonati hanno accesso a funzionalità aggiuntive che arricchiscono l'esperienza nel forum. Possono avviare discussioni tematiche, proporre idee per nuovi corsi e lasciare commenti sulle attività della palestra, contribuendo attivamente allo sviluppo della community. ....                          | 15 |
| L'utente abbonato può aggiungere: .....                                                                                                                                                                                                                                                                             | 15 |
| • Commento testuale.....                                                                                                                                                                                                                                                                                            | 15 |
| • Dare un consenso(pollice di approvazione) .....                                                                                                                                                                                                                                                                   | 15 |
| E può valutare i commenti: .....                                                                                                                                                                                                                                                                                    | 15 |
| • Utilità del commento (1-5).....                                                                                                                                                                                                                                                                                   | 15 |
| • Accordo con la valutazione (1-5).....                                                                                                                                                                                                                                                                             | 15 |
| L'utente abbonato può accedere alla cronologia dei contributi, inclusi i commenti e le partecipazioni alle discussioni nel forum.....                                                                                                                                                                               | 15 |
| L'amministratore ha invece un ruolo di gestione e supervisione. Può modificare o eliminare contenuti, bloccare alcuni thread di conversazione e pinnarne altri, creare FAQ basate sulle discussioni più rilevanti e monitorare la reputazione degli utenti per garantire un ambiente costruttivo e di qualità. .... | 15 |
| Gestione del Profilo Utente .....                                                                                                                                                                                                                                                                                   | 15 |
| Ogni utente ha la possibilità di gestire il proprio profilo in modo semplice nell' area dedicata. È possibile aggiornare i propri dati anagrafici, come indirizzo, numero di telefono, email e password e anche consultare il proprio stato di reputazione.....                                                     | 16 |
| Statistiche Avanzate (per Amministratori).....                                                                                                                                                                                                                                                                      | 16 |
| Gli amministratori hanno a disposizione strumenti di analisi approfondita per monitorare l'andamento del forum e l'interazione degli utenti. Tra queste statistiche rientrano: .....                                                                                                                                | 16 |
| • Le medie delle valutazioni ricevute per ciascun corso, utili a valutarne l'efficacia..                                                                                                                                                                                                                            | 16 |
| • I trend relativi alla reputazione degli utenti, per identificare comportamenti virtuosi o problematici. ....                                                                                                                                                                                                      | 16 |
| • Mappe termiche che mostrano le aree del forum con maggiore partecipazione.....                                                                                                                                                                                                                                    | 16 |
| • Rapporti sull'utilizzo delle promozioni, per valutarne l'impatto. ....                                                                                                                                                                                                                                            | 16 |
| • Un indice complessivo di soddisfazione, che offre una panoramica generale sul gradimento della community. ....                                                                                                                                                                                                    | 16 |

|                                                                                                                                                                                                |    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Gestione privilegi — SuperAdmin.....                                                                                                                                                           | 16 |
| L'utente SuperAdmin (root) ha la possibilità di promuovere altri utenti al ruolo di amministratore o di revocarne i privilegi. ....                                                            | 16 |
| Oltre a queste funzioni, dispone di tutti i permessi di un normale admin e può visualizzare i profili di tutti gli utenti, mantenendo quindi il pieno controllo dell'intera applicazione. .... | 16 |
| <b>Sistema Promozioni</b> .....                                                                                                                                                                | 17 |
| <b>Gestione Prenotazioni</b> .....                                                                                                                                                             | 17 |
| Strutture Dati .....                                                                                                                                                                           | 17 |
| <b>Tabella “users” (MySQL)</b> .....                                                                                                                                                           | 17 |
| Tutte le entità diverse da “users” sono gestite tramite XML esterni.....                                                                                                                       | 18 |
| <b>Esempi XML e DTD</b> .....                                                                                                                                                                  | 18 |
| <b>CORSI</b> .....                                                                                                                                                                             | 18 |
| <b>PROMO</b> .....                                                                                                                                                                             | 18 |
| <b>ABBONAMENTI</b> .....                                                                                                                                                                       | 20 |
| <b>FAQ</b> .....                                                                                                                                                                               | 20 |
| <b>FEEDBACK</b> .....                                                                                                                                                                          | 21 |
| <b>COMUNICAZIONI</b> .....                                                                                                                                                                     | 21 |
| <b>PRIVATE_CHAT</b> .....                                                                                                                                                                      | 22 |
| <b>USER_ABBONAMENTI</b> .....                                                                                                                                                                  | 23 |
| <b>FORUM</b> .....                                                                                                                                                                             | 24 |
| <b>REPUTAZIONE</b> .....                                                                                                                                                                       | 27 |
| <b>STATISTICHE</b> .....                                                                                                                                                                       | 28 |
| <b>ADMIN_AUDIT</b> .....                                                                                                                                                                       | 28 |
| <b>LEZIONI</b> .....                                                                                                                                                                           | 29 |
| MySQL – Creazione tabella “users”: .....                                                                                                                                                       | 30 |
| Dati di esempio (hash bcrypt): .....                                                                                                                                                           | 31 |
| Analisi di alcuni elementi XML.....                                                                                                                                                            | 32 |
| <b>corsi_base.xml</b> .....                                                                                                                                                                    | 32 |
| <b>promo.xml</b> .....                                                                                                                                                                         | 32 |
| <b>abbonamenti.xml</b> .....                                                                                                                                                                   | 33 |
| <b>faq.xml</b> .....                                                                                                                                                                           | 33 |
| <b>feedback.xml</b> .....                                                                                                                                                                      | 33 |

|                                          |    |
|------------------------------------------|----|
| <b>comunicazioni.xml</b> .....           | 34 |
| <b>private_chat.xml</b> .....            | 34 |
| <b>user_abbonamenti.xml</b> .....        | 35 |
| <b>forum.xml</b> .....                   | 35 |
| <b>reputazione.xml</b> .....             | 36 |
| <b>statistiche.xml</b> .....             | 36 |
| <b>admin_audit.xml</b> .....             | 37 |
| <b>lezioni.xml</b> .....                 | 37 |
| Capitolo 2 – Manuale Utente .....        | 38 |
| 1. REGISTRAZIONE ACCOUNT .....           | 38 |
| 2. LOGIN .....                           | 38 |
| 3. RECUPERA PASSWORD .....               | 39 |
| 4. HOME PAGE PER UN VISITATORE .....     | 39 |
| 5. HOME PAGE PER UN UTENTE LOGGATO ..... | 39 |
| 6. HOME PAGE PER ADMIN .....             | 40 |
| 7. AGGIUNGI COMUNICAZIONE .....          | 41 |
| 8. PROMO PER UTENTI .....                | 42 |
| 9. PROMO PER ADMIN .....                 | 42 |
| 10. CORSI PER VISITATORE .....           | 42 |
| 11. CORSI PER UTENTE ABBONATO .....      | 43 |
| 12. CORSI PER ADMIN .....                | 44 |
| 13. AGGIUNGI CORSO .....                 | 44 |
| 14. AGGIUNGI LEZIONE .....               | 45 |
| 15. MODIFICA LEZIONE .....               | 45 |
| 16. LISTA DEI PRENOTATI .....            | 46 |
| 17. CHI SIAMO UTENTI .....               | 46 |
| 18. FAQ ADMIN .....                      | 47 |
| 19. FAQ UTENTE .....                     | 47 |
| 20. PROFILO UTENTE ABBONATO .....        | 48 |
| 21. CAMBIA PASSWORD UTENTE .....         | 48 |
| 22. PROFILO ADMIN .....                  | 49 |
| 23. AGGIUNGI ABBONATO .....              | 49 |

|                                   |                                                                                                                                 |     |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------|-----|
| 24.                               | PRIVATE CHAT .....                                                                                                              | 50  |
| 25.                               | STATISTICHE AVANZATE PER ADMIN .....                                                                                            | 51  |
| 26.                               | FORUM UTENTE .....                                                                                                              | 51  |
| 27.                               | FORUM ADMIN .....                                                                                                               | 52  |
| 28.                               | CRONOLOGIA CONTRIBUTI FORUM .....                                                                                               | 52  |
| 29.                               | GESTIONE PER “SUPERADMIN” .....                                                                                                 | 53  |
| 30.                               | PROFILO PUBBLICO PER SUPERADMIN .....                                                                                           | 53  |
| Capitolo 3 – Programmazione ..... |                                                                                                                                 | 54  |
| 1.                                | SCRIPT:login.php .....                                                                                                          | 54  |
| 2.                                | SCRIPT:register.php.....                                                                                                        | 56  |
| 3.                                | SCRIPT>Password_dimenticata.php.....                                                                                            | 58  |
| 4.                                | SCRIPT:Db.php;Dati_generali.php .....                                                                                           | 60  |
| 5.                                | SCRIPT:home_page.php .....                                                                                                      | 61  |
| 6.                                | SCRIPT:Promo.php.....                                                                                                           | 69  |
| 7.                                | SCRIPT:Corsi.php .....                                                                                                          | 73  |
| 8.                                | SCRIPT:Chi_siamo.php.....                                                                                                       | 78  |
| 11.                               | SCRIPT: faq.php .....                                                                                                           | 79  |
| 10.                               | SCRIPT: Profilo.php;Mark_messages_read.php .....                                                                                | 81  |
| 12.                               | SCRIPT:save_feedback.php;modifica_lezione.php;cancella_lezione.php<br>;aggiungi_lezione.php.....                                | 87  |
| 13.                               | SCRIPT:<br>invio_mess_utente.php;invio_mess_admin.php;auto_responder.php;get_chat_messages.p<br>hp;check_new_messages.php ..... | 92  |
| 14.                               | SCRIPT:Admin_chat.php .....                                                                                                     | 97  |
| 15.                               | SCRIPT:toggle_admin.php,valuta_commento.php,mi_piace.php .....                                                                  | 99  |
| 16.                               | SCRIPT: superadmin.php .....                                                                                                    | 102 |
| 11.                               | SCRIPT: profilo_pubblico.php .....                                                                                              | 104 |
| Capitolo 4 – Elenco utenti .....  |                                                                                                                                 | 106 |

# Capitolo 1 – Introduzione e descrizione dei dati

## Introduzione

L'applicazione web "PALESTRW" è una piattaforma digitale progettata per semplificare la gestione di una palestra. Essa consente l'interazione tra clienti e amministratori, automatizzando processi come la registrazione utenti, la gestione degli abbonamenti, la prenotazione dei corsi, la comunicazione interna, la gestione delle FAQ e la messaggistica privata.

Questa sezione descrive in dettaglio:

- *Cosa fa l'applicazione e a che serve*
- *Tipologie di utenti supportate*
- *Funzionalità principali offerte per ciascun ruolo*
- *Voci di menu disponibili a seconda del profilo*
- *Architettura dati: una sola tabella "users" in database, tutti gli altri dati (abbonamenti, corsi, prenotazioni, feedback, promozioni, FAQ, messaggi..) gestiti tramite file XML esterni*

## Note

In questa sezione si chiariscono alcuni concetti chiave menzionati in introduzione:

### Abbonamento:

Contratto tra utente e palestra con durata definita (mensile, trimestrale, annuale), vincoli di utilizzo (numero di prenotazioni settimanali), date di inizio e scadenza, stati (attivo, scaduto).

### Promozione:

Offerta temporanea gestita dall'amministratore, associata a un codice univoco, con validità definita da data di inizio e fine e condizioni d'uso specifiche.

### Valutazione/Statistica:

Sistema di feedback su base numerica (scala da 1 a 5); le valutazioni vengono aggregate per fornire statistiche utili al miglioramento dei corsi.

### Messaggistica privata:

Conversazioni tra staff e utente con abbonamento attivo.

#### Partecipazione alla lezione:

Un utente abbonato è considerato partecipante a una lezione se ha effettuato la prenotazione, non l'ha disdetta e la data della lezione è trascorsa. Solo in questo caso gli viene consentito di esprimere una valutazione.

#### Reputazione:

Punteggio dinamico (0-100) basato sulla qualità dei contributi nel forum e nelle valutazioni, calcolato tramite algoritmo che considera consensi ricevuti e attività recente.

#### Validazione feedback:

Gli utenti possono valutare solo commenti di altri membri (non propri) entro 30 giorni dalla pubblicazione.

### **Tipologie di Utenti**

| Ruolo                  | Funzionalità principali                                                                                                |
|------------------------|------------------------------------------------------------------------------------------------------------------------|
| Visitatore             | Accesso homepage, comunicazioni generali, corsi disponibili, promozioni attive, FAQ                                    |
| Utente registrato      | Tutto il visitatore + area personale, modifica password, stato abbonamento (non attivo), può richiedere un abbonamento |
| Utente con abbonamento | Tutto l'abbonato + gestione comunicazioni, utenti abbonati, corsi, statistiche, promozioni, FAQ, forum, reputazioni    |
| Amministratore         | Tutto l'abbonato + gestione comunicazioni, utenti abbonati, corsi, statistiche, promozioni, FAQ, forum, reputazioni    |

### **Menu Utente (in base al ruolo)**

| Voce Menu        | Visitatore | Registrato | Con Abbonamento | Admin/Gestore | “superAdmin” |
|------------------|------------|------------|-----------------|---------------|--------------|
| Home             | Sì         | Sì         | Sì              | Sì            | Sì           |
| Visualizza corsi | Sì         | Sì         | Sì              | Sì            | Sì           |

|                           |    |    |    |    |    |
|---------------------------|----|----|----|----|----|
| Prenota corso             | No | No | Sì | No | No |
| Stato abbonamento         | No | Sì | Sì | Sì | Sì |
| Storico prenotazioni      | No | No | Sì | Sì | Sì |
| Gestione corsi            | No | No | No | Sì | Sì |
| Promozioni                | Sì | Sì | Sì | Sì | Sì |
| FAQ                       | Sì | Sì | Sì | Sì | Sì |
| Messaggistica privata     | No | No | Sì | Sì | Sì |
| Valutazioni corsi         | No | No | Sì | Sì | Sì |
| Comunicazioni staff       | Sì | Sì | Sì | Sì | Sì |
| Forum pubblico            | No | No | Sì | Sì | Sì |
| Reputazione               | No | No | Sì | Sì | Sì |
| Modifica profilo          | No | Sì | Sì | Sì | Sì |
| Valutazioni commenti      | No | No | Sì | Sì | Sì |
| Gestione forum            | No | No | No | Sì | Sì |
| Statistiche avanzate      | No | No | No | Sì | Sì |
| Visualizza lista abbonati | No | No | No | Sì | Sì |
| Gestione privilegi        | No | No | No | No | Sì |

## Funzionalità e Casi d'Uso

### Registrazione

Quando un visitatore arriva su PALESTRW, può avviare subito la procedura di registrazione cliccando sul pulsante dedicato (Area riservata). Viene quindi presentato un form in cui inserire username, email, password, indirizzo e telefono. Il sistema controlla lato client il formato dell'email, quindi invia i dati al server, dove viene verificato che l'indirizzo email non sia già registrato. In caso di duplicato, l'utente riceve un messaggio di errore con suggerimento di recupero credenziali. Se tutto è corretto, la password viene hashata (bcrypt) e si crea un nuovo record nella tabella "users" con timestamp di registrazione. Dopo la registrazione, l'utente può accedere alla propria area personale, ma non potrà ancora prenotare corsi e altre funzionalità fino all'attivazione di un abbonamento.

### Accesso all'area riservata

Un utente già registrato può effettuare il login inserendo email/username e password. Il backend recupera il record corrispondente da "users" e confronta la password tramite bcrypt: in caso di esito positivo, viene creata una sessione sicura e l'utente viene reindirizzato alla home page, con accesso alle sezioni riservate per utenti.

Il sistema distingue due categorie di utenti, ciascuna con permessi e funzionalità differenti:

- ✓ Utente senza abbonamento: accesso ristretto, limitato alle operazioni essenziali (es. visualizza il profilo)
- ✓ Utente con abbonamento attivo: accesso completo a funzionalità avanzate, come prenotazione corsi, storico prenotazioni, messaggistica privata con lo staff.

### Prenotazione corsi

Solo gli utenti con abbonamento attivo possono prenotare una lezione. Dalla pagina dei corsi, il sistema legge corsi\_base.xml per mostrare titolo, descrizione, orari e posti disponibili. Al clic su “Prenota”, verifica lo stato dell’abbonamento e la disponibilità di posti: se validi, aggiunge un tag <prenotante> in lezioni.xml e conferma la prenotazione; altrimenti mostra un messaggio di errore (“Login+Abbonamento” o “Posti esauriti”).

#### Tentativo di prenotazione senza abbonamento

Se un utente registrato ma senza abbonamento attivo tenta di prenotare, l’operazione viene bloccata dal backend, migliorando l’esperienza utente e guidando verso azioni consentite.

#### Visualizzazione corsi e promozioni

Tutti gli utenti (anche non registrati) possono esplorare l’offerta formativa. Ogni corso presenta una scheda con dettagli e posti disponibili. Nell’area promozioni vengono visualizzate le offerte attive, con titolo, descrizione, condizioni, codice sconto e limitazioni.

#### Visualizzazione stato abbonamento

Nell’area riservata, l’utente trova il modulo “abbonamento”, nella sezione profilo, che legge user\_abbonamenti.xml e mostra tipo di piano, date di inizio/fine e stato (“Attivo”, “Scaduto” o “in scadenza”).

#### Consultazione FAQ

La sezione FAQ è basata su faq.xml: un elenco di domande cliccabili che mostrano la risposta in modo animato; è presente un campo di ricerca. Gli amministratori possono aggiungere, modificare o eliminare domande.

#### Valutazione corsi

Dopo la partecipazione, Gli utenti abbonati possono lasciare:

- Voto numerico (1-5 stelle)

Il sistema verifica la partecipazione in corsi.xml e salva il voto in feedback.xml.

#### Messaggistica privata

Utenti e staff scambiano messaggi in private\_chat.xml. Ogni messaggio crea o aggiorna un nodo cronologico; lo staff può rispondere dall'area dedicata. Se dopo due giorni non arriva risposta, il sistema invia automaticamente un messaggio di sollecito all'utente.

#### Gestione corsi e lezioni (Admin)

L'amministratore accede a un pannello avanzato per creare, modificare o rimuovere corsi; definire sessioni, orari, limiti partecipanti.

#### Gestione promozioni (Admin)

Modulo per creare o eliminare promozioni, definendo titolo, descrizione, codice sconto, date di validità e limiti; le promozioni sono visibili nella sezione pubblica.

#### Gestione utenti (Admin)

Elenco completo di utenti abbonati attivi, con dettagli personali, tipo di abbonamento.

#### Gestione prenotazioni (Admin)

Dashboard delle prenotazioni con liste partecipanti; possibilità di inviare notifiche in caso di cancellazioni o overbooking, e di analizzare i dati per previsioni future.

#### Comunicazioni dallo staff (Admin)

L'amministratore può inviare comunicazioni visibili a tutti gli utenti, utili per segnalare modifiche importanti, aggiornamenti del sistema o per condividere avvisi urgenti.

#### Sistema di Reputazione

Il sistema di reputazione assegna a ciascun utente un punteggio che riflette il valore dei suoi contributi nel forum. Questo punteggio viene calcolato tenendo conto di quattro elementi principali: la qualità delle valutazioni ricevute sui commenti, il livello di partecipazione alle discussioni, l'utilità dei contenuti riconosciuta dalla community e l'attività recente dell'utente. In particolare, il sistema di reputazione utilizza un algoritmo che considera l'ultima attività dell'utente per calcolare il decadimento della reputazione, identificare utenti inattivi e aggiornare dinamicamente i punteggi. La reputazione è visibile nel profilo utente e incide sulla visibilità dei propri interventi: contributi provenienti da utenti con alta reputazione tendono infatti ad avere maggiore rilievo all'interno del forum.

### Forum Pubblico

Gli utenti abbonati hanno accesso a funzionalità aggiuntive che arricchiscono l'esperienza nel forum. Possono avviare discussioni tematiche, proporre idee per nuovi corsi e lasciare commenti sulle attività della palestra, contribuendo attivamente allo sviluppo della community.

L'utente abbonato può aggiungere:

- Commento testuale
- Dare un consenso(pollice di approvazione)

E può valutare i commenti:

- Utilità del commento (1-5)
- Accordo con la valutazione (1-5)

L'utente abbonato può accedere alla cronologia dei contributi, inclusi i commenti e le partecipazioni alle discussioni nel forum.

L'amministratore ha invece un ruolo di gestione e supervisione. Può modificare o eliminare contenuti, bloccare alcuni thread di conversazione e pinnarne altri, creare FAQ basate sulle discussioni più rilevanti e monitorare la reputazione degli utenti per garantire un ambiente costruttivo e di qualità.

### Gestione del Profilo Utente

Ogni utente ha la possibilità di gestire il proprio profilo in modo semplice nell'area dedicata. È possibile aggiornare i propri dati anagrafici, come indirizzo, numero di telefono, email e password e anche consultare il proprio stato di reputazione.

#### Statistiche Avanzate (per Amministratori)

Gli amministratori hanno a disposizione strumenti di analisi approfondita per monitorare l'andamento del forum e l'interazione degli utenti. Tra queste statistiche rientrano:

- Le medie delle valutazioni ricevute per ciascun corso, utili a valutarne l'efficacia.
- I trend relativi alla reputazione degli utenti, per identificare comportamenti virtuosi o problematici.
- Mappe termiche che mostrano le aree del forum con maggiore partecipazione.
- Rapporti sull'utilizzo delle promozioni, per valutarne l'impatto.
- Un indice complessivo di soddisfazione, che offre una panoramica generale sul gradimento della community.

#### Gestione privilegi — SuperAdmin

L'utente SuperAdmin (root) ha la possibilità di promuovere altri utenti al ruolo di amministratore o di revocarne i privilegi.

Oltre a queste funzioni, dispone di tutti i permessi di un normale admin e può visualizzare i profili di tutti gli utenti, mantenendo quindi il pieno controllo dell'intera applicazione.

## Sistema Promozioni

Ogni promozione contiene:

- Codice sconto univoco
- Titolo e descrizione dettagliata
- Data di inizio e fine validità
- Stato (attiva/disattiva)
- Percentuale di sconto
- Tipo di applicazione dello sconto
- Associazione con il tipo di abbonamento

Le promozioni possono essere applicate dagli utenti per sconti all'attivazione dell'abbonamento

## Gestione Prenotazioni

- Prenotazione consentita solo agli utenti con abbonamento attivo
- Massimo una prenotazione per corso per utente
- Verifica disponibilità posti totali per ogni lezione
- Blocco automatico delle prenotazioni doppie o non autorizzate

## Strutture Dati

### Tabella "users" (MySQL)

| Campo           | Tipo          | Descrizione                    |
|-----------------|---------------|--------------------------------|
| id_user         | INT PK        | Identificatore univoco         |
| username        | VARCHAR(50)   | Nome utente                    |
| password        | VARCHAR(255)  | Password hashata               |
| email           | VARCHAR(100)  | Email univoca                  |
| is_admin        | TINYINT(1)    | 0 = utente, 1 = admin          |
| password_temp   | TINYINT(1)    | 0 = permanente, 1 = temporaneo |
| Force_pw_change | TINYINT(1)    | Flag per password dimenticata  |
| indirizzo       | VARCHAR(255)  | Indirizzo residenza            |
| telefono        | VARCHAR(20)   | Numero di telefono             |
| reputazione     | INT DEFAULT 0 | Punteggio reputazione (0-100)  |

Tutte le entità diverse da “users” sono gestite tramite XML esterni.

## Esempi XML e DTD

### CORSI

Esempio di corsi\_base.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE corsi_base SYSTEM "../dtd/corsi_base.dtd">
<corsi_base>
  <corso_base id="1">
    <nome>Yoga Mattutino</nome>
    <descrizione>Un corso rilassante per iniziare la giornata con energia
positiva.</descrizione>
    <difficolta>1</difficolta>
  </corso_base>
  ...
</corsi_base>
```

DTD per corsi\_base.xml:

```
<!ELEMENT corsi_base (corso_base*)>
<!ELEMENT corso_base (nome, descrizione, difficolta)>
<!ATTLIST corso_base
  id CDATA #REQUIRED>
<!ELEMENT nome (#PCDATA)>
<!ELEMENT descrizione (#PCDATA)>
<!ELEMENT difficolta (#PCDATA)>
```

### PROMO

Esempio di promo.xml:

```
<?xml version="1.0"?>
<!DOCTYPE promozioni SYSTEM "../dtd/promo.dtd">
<promozioni>
  <promo id="1">
    <titolo>Benvenuto in Palestra!</titolo>
    <descrizione>Primo mese a met&#xE0; prezzo per i nuovi iscritti.</descrizione>
```

```

<data_inizio>2025-06-01</data_inizio>
<data_fine>2025-12-31</data_fine>
<codice_sconto>WELCOME50</codice_sconto>
<attiva>1</attiva>
<percentuale>50.00</percentuale>
<applica>
  <tipo>Mensile</tipo>
  <tipo>Trimestrale</tipo>
</applica>
</promo>
...
</promozioni>

```

DTD per promo.xml:

```

<!ELEMENT promozioni (promo+)>
<!ELEMENT promo (
  titolo,
  descrizione,
  data_inizio,
  data_fine,
  codice_sconto,
  attiva,
  percentuale?,
  applica
)>
<!ATTLIST promo
  id CDATA #REQUIRED>
<!ELEMENT titolo (#PCDATA)>
<!ELEMENT descrizione (#PCDATA)>
<!ELEMENT data_inizio (#PCDATA)>
<!ELEMENT data_fine (#PCDATA)>
<!ELEMENT codice_sconto (#PCDATA)>
<!ELEMENT attiva (#PCDATA)>
<!ELEMENT percentuale (#PCDATA)>

```

<!ELEMENT applica (tipo+)>  
<!ELEMENT tipo (#PCDATA)>

## ABBONAMENTI

Esempio di abbonamenti.xml:

```
<?xml version="1.0"?>
<!DOCTYPE abbonamenti SYSTEM "../dtd/abbonamenti.dtd">
<abbonamenti>
  <abbonamento id="1">
    <tipo>Mensile</tipo>
    <durata_mesi>1</durata_mesi>
    <prezzo>30.00</prezzo>
  </abbonamento>
  ...
</abbonamenti>
```

DTD per abbonamenti.xml:

```
<!ELEMENT abbonamenti (abbonamento+)>
<!ELEMENT abbonamento (tipo, durata_mesi, prezzo)>
<!ATTLIST abbonamento id ID #REQUIRED>
<!ELEMENT tipo (#PCDATA)>
<!ELEMENT durata_mesi (#PCDATA)>
<!ELEMENT prezzo (#PCDATA)>
```

## FAQ

Esempio di faq.xml:

```
<?xml version="1.0"?>
<!DOCTYPE faqs SYSTEM "../dtd/faq.dtd">
<faqs>
  <faq>
    <id>1</id>
    <question>Quali orari sono disponibili?</question>
    <answer>Dal luned&#xE9; al sabato, 9:00-23:00</answer>
  </faq>
  ...
</faqs>
```

#### DTD per faq.xml:

```
<!ELEMENT faqs (faq+)>
<!ELEMENT faq (id, question, answer)>
<!ELEMENT id (#PCDATA)>
<!ELEMENT question (#PCDATA)>
<!ELEMENT answer (#PCDATA)>
```

## **FEEDBACK**

#### Esempio di feedback.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE feedback SYSTEM "../dtd/feedback.dtd">
<feedback>
  <valutazione_corso>
    <id_lezione>1</id_lezione>
    <voto>4</voto>
    <id_user_abbonamento>a1</id_user_abbonamento>
  </valutazione_corso>
  ...
</feedback>
```

#### DTD per feedback.xml:

```
<!ELEMENT feedback (valutazione_corso+)>
<!ELEMENT valutazione_corso (id_lezione, voto, id_user_abbonamento)>
<!ELEMENT id_lezione (#PCDATA)>
<!ELEMENT voto (#PCDATA)>
<!ELEMENT id_user_abbonamento (#PCDATA)>
```

## **COMUNICAZIONI**

#### Esempio di comunicazioni.xml:

```
<?xml version="1.0"?>
<!DOCTYPE comunicazioni SYSTEM "../dtd/comunicazioni.dtd">
<comunicazioni>
  <messaggio autore="3" data_invio="2025-07-07 18:00:00">
    <titolo>Invito speciale</titolo>
    <contenuto>Ti invitiamo al workshop di stretching di venerd&#xE9; prossimo</contenuto>
  </messaggio>
```

```

<messaggio autore="1" data_invio="2025-07-21 17:04:02">
  <titolo>a</titolo>
  <contenuto>a</contenuto>
</messaggio>

```

...

```

</comunicazioni>

```

DTD per comunicazioni.xml:

```

<!ELEMENT comunicazioni (messaggio*)>
<!ELEMENT messaggio (titolo, contenuto)>
<!ATTLIST messaggio
  autore CDATA #REQUIRED
  data_invio CDATA #REQUIRED>
<!ELEMENT titolo (#PCDATA)>
<!ELEMENT contenuto (#PCDATA)>

```

## PRIVATE\_CHAT

Esempio di private\_chat.xml:

```

<?xml version="1.0"?>
<!DOCTYPE chat SYSTEM "../dtd/private_chat.dtd">
<chat>
  <messaggio id="m_689b2322ca4708.38499815" mittente="mario" destinatario="admin"
data="2025-08-12 13:18:58" risposta="1" sollecito_inviato="0">
    <testo><![CDATA[ok grazie]]></testo>
  </messaggio>
  <messaggio id="m_689b232aa481f4.75717275" mittente="mario" destinatario="admin"
data="2025-08-12 13:19:06" risposta="1" sollecito_inviato="0">
    <testo><![CDATA[ok grazie]]></testo>
  </messaggio>

```

...

```

</chat>

```

DTD per private\_chat.xml:

```

<!ELEMENT chat (messaggio*)>
<!ELEMENT messaggio (testo)>
<!ATTLIST messaggio
  id CDATA #REQUIRED

```

mittente CDATA #REQUIRED  
destinatario CDATA #REQUIRED  
data CDATA #REQUIRED  
risposta CDATA #IMPLIED  
sollecito\_inviato CDATA #REQUIRED  
admin\_id CDATA #IMPLIED>  
<!ELEMENT testo (#PCDATA)>

## **USER\_ABBONAMENTI**

*Esempio di user\_abbonamenti.xml:*

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE user_abbonamenti SYSTEM "../dtd/user_abbonamenti.dtd">
<user_abbonamenti>
  <abbonamento_utente id="a1">
    <id_user>1</id_user>
    <id_abbonamento>3</id_abbonamento>
    <data_inizio>2025-01-01</data_inizio>
    <data_scadenza>2026-01-01</data_scadenza>
    <stato>attivo</stato>
    <prezzo>299.00</prezzo>
    <valuta>EUR</valuta>
    <prezzo_base>300.00</prezzo_base>
    <max_prenotazioni_settimanali>10</max_prenotazioni_settimanali>
  </abbonamento_utente>
  ...
</user_abbonamenti>
```

*DTD per user\_abbonamenti.xml:*

```
<!ELEMENT user_abbonamenti (abbonamento_utente*)>
<!ELEMENT abbonamento_utente (id_user, id_abbonamento, data_inizio, data_scadenza,
stato, prezzo, valuta?, prezzo_base?, max_prenotazioni_settimanali?, id_promo?,
test_convalida?)>
<!ATTLIST abbonamento_utente
  id ID #REQUIRED>
<!ELEMENT id_user (#PCDATA)>
<!ELEMENT id_abbonamento (#PCDATA)>
```

```

<!ELEMENT data_inizio (#PCDATA)>
<!ELEMENT data_scadenza (#PCDATA)>
<!ELEMENT stato (#PCDATA)>
<!ELEMENT prezzo (#PCDATA)>
<!ELEMENT valuta (#PCDATA)>
<!ELEMENT prezzo_base (#PCDATA)>
<!ELEMENT max_prenotazioni_settimanali (#PCDATA)>
<!ELEMENT id_promo (#PCDATA)>
<!ELEMENT test_convalida (prezzo_atteso?, conforme?, differenza?, note?)>
<!ELEMENT prezzo_atteso (#PCDATA)>
<!ELEMENT conforme (#PCDATA)>
<!ELEMENT differenza (#PCDATA)>
<!ELEMENT note (#PCDATA)>

```

## FORUM

Esempio di forum.xml:

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE forum SYSTEM "../dtd/forum.dtd">
<forum>
  <thread id="1">
    <titolo>Nuovi corsi richiesti</titolo>
    <autore>mario</autore>
    <data_creazione>2025-07-01 10:00:00</data_creazione>
    <contenuto>Quali corsi vorreste vedere?</contenuto>
    <commenti>
      <commento id="101">
        <autore>luisa</autore>
        <contenuto>Pilates avanzato!</contenuto>
        <data>2025-07-01 11:00:00</data>
        <consensi>
          <consenso>mario</consenso>
          <consenso>luisa</consenso>
        </consensi>
        <valutazioni_commento>

```

```

    <valutazione valutatore="luisa">
      <utilita>1</utilita>
      <accordo>1</accordo>
    </valutazione>
    <valutazione valutatore="mario">
      <utilita>1</utilita>
      <accordo>1</accordo>
    </valutazione>
  </valutazioni_commento>
</commento>
<commento id="102">
  <autore>alice</autore>
  <contenuto>Magari un corso di HIIT serale!</contenuto>
  <data>2025-07-01 12:15:00</data>
  <consensi>
    <consenso>mario</consenso>
    <consenso>luisa</consenso>
    <consenso>alice</consenso>
  </consensi>
  <valutazioni_commento>
    <valutazione valutatore="luisa">
      <utilita>2</utilita>
      <accordo>2</accordo>
    </valutazione>
  </valutazioni_commento>
</commento>
<commento id="103">
  <autore>luisa</autore>
  <contenuto>a</contenuto>
  <data>2025-07-23 19:39:43</data>
  <consensi/>
  <valutazioni_commento>
    <valutazione valutatore="luisa">

```

```

    <utilita>3</utilita>
    <accordo>3</accordo>
  </valutazione>
</valutazioni_commento>
</commento>
<commento id="104">
  <autore>sara</autore>
  <contenuto>(nessun contenuto)</contenuto>
  <data>2025-07-30 13:43:40</data>
  <consensi/>
</commento>
</commenti>
</thread>

```

...

```
</forum>
```

DTD per forum.xml:

```

<!ELEMENT forum (thread+)>
<!ELEMENT thread (titolo, autore, data_creazione, contenuto, commenti)>
<!ATTLIST thread
  id CDATA #REQUIRED
  pinned (true|false) #IMPLIED
  locked (true|false) #IMPLIED
  last_edit_by CDATA #IMPLIED
  last_edit_date CDATA #IMPLIED>
<!ELEMENT titolo (#PCDATA)>
<!ELEMENT autore (#PCDATA)>
<!ELEMENT data_creazione (#PCDATA)>
<!ELEMENT contenuto (#PCDATA)>
<!ELEMENT commenti (commento*)>
<!ELEMENT commento (autore, contenuto, data, consensi, valutazioni_commento?,
valutazione_corso?)>
<!ATTLIST commento
  id CDATA #IMPLIED>
<!ELEMENT data (#PCDATA)>

```

```

<!ELEMENT consensi (#PCDATA | consenso)*>
<!ELEMENT consenso (#PCDATA)>
<!ELEMENT valutazioni_commento (valutazione*)>
<!ELEMENT valutazione (utilita, accordo)>
<!ATTLIST valutazione
  valutatore CDATA #REQUIRED>
<!ELEMENT utilita (#PCDATA)>
<!ELEMENT accordo (#PCDATA)>
<!ELEMENT valutazione_corso (#PCDATA)>

```

## REPUTAZIONE

Esempio di reputazione.xml:

```

<?xml version="1.0"?>
<!DOCTYPE reputazioni SYSTEM "./dtd/reputazione.dtd">
<reputazioni>
  <utente id="1">
    <username>mario</username>
    <punteggio>90</punteggio>
    <ultima_attivita>2025-07-07</ultima_attivita>
  </utente>
  <utente id="2">
    <username>luisa</username>
    <punteggio>91</punteggio>
    <ultima_attivita>2025-07-30</ultima_attivita>
  </utente>
  ...
</reputazioni>

```

DTD per reputazione.xml:

```

<!ELEMENT reputazioni (utente*)>
<!ELEMENT utente (username, punteggio, ultima_attivita)>
<!ATTLIST utente id CDATA #REQUIRED>
<!ELEMENT username (#PCDATA)>
<!ELEMENT punteggio (#PCDATA)>
<!ELEMENT ultima_attivita (#PCDATA)>

```

## STATISTICHE

### Esempio di statistiche.xml:

```
<!DOCTYPE statistiche SYSTEM "../dtd/statistiche.dtd">
<statistiche>
  <corso id="9">
    <valutazione_media>4.4</valutazione_media>
    <partecipazioni>75</partecipazioni>
  </corso>
  <corso id="10">
    <valutazione_media>4.0</valutazione_media>
    <partecipazioni>90</partecipazioni>
  </corso>
  <forum>
    <attivita_media_giornaliera>45</attivita_media_giornaliera>
    <thread_attivi>12</thread_attivi>
  </forum>
  <soddisfazione_generale>88</soddisfazione_generale>
  ...
</statistiche>
```

### DTD per statistiche.xml:

```
<!ELEMENT statistiche (corso+, forum, soddisfazione_generale)>
<!ELEMENT corso (valutazione_media, partecipazioni)>
<!ATTLIST corso id CDATA #REQUIRED>
<!ELEMENT valutazione_media (#PCDATA)>
<!ELEMENT partecipazioni (#PCDATA)>
<!ELEMENT forum (attivita_media_giornaliera, thread_attivi)>
<!ELEMENT attivita_media_giornaliera (#PCDATA)>
<!ELEMENT thread_attivi (#PCDATA)>
<!ELEMENT soddisfazione_generale (#PCDATA)>
```

## ADMIN\_AUDIT

### Esempio di admin\_audit.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE admin_actions SYSTEM "../dtd/admin_audit.dtd">
```

```

<admin_actions>
  <action timestamp="2025-08-12 10:36:11" admin="mario" type="delete"
target_type="thread" target_id="6">
    <detail name="thread_id"/>
    <detail name="content">qa</detail>
    <detail name="author">luisa</detail>
    <detail name="admin">mario</detail>
  </action>
</admin_actions>

```

DTD per admin\_audit.xml:

```

<!ELEMENT admin_actions (action+)>
<!ELEMENT action (detail+)>
<!ATTLIST action
  timestamp CDATA #REQUIRED
  admin CDATA #REQUIRED
  type (edit | delete | pin | unpin) #REQUIRED
  target_type (comment | thread) #REQUIRED
  target_id CDATA #REQUIRED>
<!ELEMENT detail (#PCDATA)>
<!ATTLIST detail name CDATA #REQUIRED>

```

## LEZIONI

Esempio di lezioni.xml:

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE lezioni SYSTEM "../dtd/lezioni.dtd">
<lezioni>
  <!-- id: 1 - lezione passata -->
  <lezione id="1">
    <id_corso_base>1</id_corso_base>
    <datetime_lezione>2025-08-05 09:00:00</datetime_lezione>
    <posti_totali>20</posti_totali>
    <prenotanti>
      <prenotante id="1" id_abb="a1" user="mario" stato="confermato"
data_prenotazione="2025-07-30 10:00:00"/>
    </prenotanti>
  </lezione>

```

```
<prenotante id="2" id_abb="a2" user="anna" stato="confermato"
data_prenotazione="2025-07-31 14:30:00"/>
```

```
</prenotanti>
```

```
</lezione>
```

```
...
```

```
</lezioni>
```

DTD per lezioni.xml:

```
<!ELEMENT lezioni (lezione*)>
```

```
<!ELEMENT lezione (id_corso_base, datetime_lezione, posti_totali, prenotanti)>
```

```
<!ATTLIST lezione
```

```
  id ID #REQUIRED>
```

```
<!ELEMENT id_corso_base (#PCDATA)>
```

```
<!ELEMENT datetime_lezione (#PCDATA)>
```

```
<!ELEMENT posti_totali (#PCDATA)>
```

```
<!ELEMENT prenotanti (prenotante*)>
```

```
<!ELEMENT prenotante EMPTY>
```

```
<!ATTLIST prenotante
```

```
  id ID #REQUIRED
```

```
  id_abb CDATA #REQUIRED
```

```
  user CDATA #REQUIRED
```

```
  stato (confermato|cancellato) "confermato"
```

```
  data_prenotazione CDATA #IMPLIED
```

```
  data_cancellazione CDATA #IMPLIED>
```

MySQL – Creazione tabella “users”:

```
CREATE TABLE users (
```

```
  id_user INT AUTO_INCREMENT PRIMARY KEY,
```

```
  username VARCHAR(50) NOT NULL UNIQUE,
```

```
  password VARCHAR(255) NOT NULL,
```

```
  email VARCHAR(100) NOT NULL,
```

```
  is_admin TINYINT(1) DEFAULT 0,
```

```
  password_temp TINYINT(1) DEFAULT 0,
```

```
  force_pw_change TINYINT(1) DEFAULT 0,
```

```
indirizzo VARCHAR(255),  
telefono VARCHAR(50),  
reputazione INT DEFAULT 0  
);
```

### Dati di esempio (hash bcrypt):

*-- utenti con password hashate (bcrypt, 60 caratteri)*

```
INSERT INTO users (id_user, username, password, email, is_admin, password_temp,  
force_pw_change, indirizzo, telefono, reputazione) VALUES  
  
(1, 'mario', '$2y$10$RkDMVeMN0.oe65IH0d8NuOZy0Mk678yyvdvnDeosvEqZ6tuxpjNMO',  
'mario@example.com', 1, 0, 0, 'Via Roma 1, 00100 Roma RM', '+39 06 12345678', 0),  
  
(2, 'luisa', '$2y$10$w.1iy.hD2lV1Znjoql2uCOJ4CC2Wsj6v9b4sRGgq0k0mVaqTvLgZm',  
'luisa@example.com', 0, 0, 0, 'Corso Italia 10, 20100 Milano MI', '+39 02 23456789', 0),  
  
(3, 'gianni', '$2y$10$P9FNJkB.RxKB.Nqs31BySunPTJ6xVvlk1lbgWj5jzXe9N4TaRtWoW',  
'gianni@example.com', 0, 0, 0, 'Piazza Garibaldi 5, 80100 Napoli NA', '+39 081 3456789', 0),  
  
(4, 'sara', '$2y$10$y4DPpUbT3oX2RkikRxReGu8gouvri3zLDzfvxyD6Eqpm2P5fS7FU2',  
'sara@example.com', 0, 0, 0, 'Via Verdi 12, 16100 Genova GE', '+39 010 4567890', 0),  
  
(5, 'alice', '$2y$10$QMnmslL0K8ZL6JykOA5B7epXlkdOJoLM7sdZbdUweyidsf2gbrXWa',  
'alice@example.com', 0, 0, 0, 'Via Dante 7, 50100 Firenze FI', '+39 055 5678901', 0),  
  
(6, 'bob', '$2y$10$raUKGlcXduj83YNtvFqBhuJdBeEemS50PgydCXK3OkQA54cHGvKum',  
'bob@example.com', 0, 0, 0, 'Lungomare 3, 90100 Palermo PA', '+39 091 6789012', 0),  
  
(7, 'carla', '$2y$10$Vyl/NR8JlObUczxfNUgHOK.QPiPg9vi0Fn0/vrlckluCmrk54QKW',  
'carla@example.com', 0, 0, 0, 'Via XX Settembre 20, 70100 Bari BA', '+39 080 7890123', 0),  
  
(8, 'dario', '$2y$10$8pfrqKgjiK.06DCpUWxCnOvtxd.46Rd/0nvl8eQv2o5yTcgOCM0nG',  
'dario@example.com', 0, 0, 0, 'Piazza Duomo 4, 09100 Cagliari CA', '+39 070 8901234', 0),  
  
(9, 'admin', '$2y$10$.LurbelqN3TTFxinKplvwuna1IT5.3pkRemFsMGJ783Xd//1u.jNu',  
'admin@gym.it', 1, 0, 0, 'Via Libertà 99, 95100 Catania CT', '+39 095 9012345', 0);
```

## Analisi di alcuni elementi XML

### **corsi\_base.xml**

a) Attributo *id* in `<corso_base>`

I. Posizione:

*Pagina 17 Linea 8*

II. Riferimento specifica:

- *"Architettura dati" (Gestione corsi tramite XML - Pagina 17 Linea 8)*
- *"Prenotazione corsi" (Associazione lezione-corsi - Pagina 11 Linea 1-2)*

III. Giustificazione:

- *Fondamentale per associare prenotazioni ai corsi corrispondenti e prevenire duplicati.*

b) Elemento `<descrizione>`

I. Posizione:

*Pagina 17 Linea 10*

II. Riferimento specifica:

- *"Visualizzazione corsi" (Dettagli offerta formativa - Pagina 12 Linea 13)*

III. Giustificazione:

- *Essenziale per fornire informazioni dettagliate agli utenti durante la selezione dei corsi.*

### **promo.xml**

a) Elemento `<codice_sconto>`

I. Posizione:

*Pagina 18 Linea 3*

II. Riferimento specifica:

- *"Promozione" (Codice univoco - Pagina 8 Linea 23)*
- *"Sistema Promozioni" (Pagina 15 Linea 20)*

III. Giustificazione:

- *Cruciale per implementare sconti univoci e tracciarne l'utilizzo.*

b) Elemento `<data_fine>`

I. Posizione:

*Pagina 18 Linea 2*

II. Riferimento specifica:

- *"Promozione" (Validità temporale - Pagina 8 Linea 24)*

III. Giustificazione:

- *Necessario per automatizzare la disattivazione delle promozioni scadute.*

## **abbonamenti.xml**

### *a) Elemento <tipo>*

#### I. Posizione:

*Pagina 19 Linea 14*

#### II. Riferimento specifica:

- *"Abbonamento" (Tipologie piano - Pagina 8 Linea 19)*

#### III. Giustificazione:

- *Fondamentale per differenziare piani mensili/trimestrali/annuali nell'interfaccia utente.*

### *b) Elemento <prezzo>*

#### I. Posizione:

*Pagina 19 Linea 16*

#### II. Riferimento specifica:

- *"Visualizzazione stato abbonamento" (Pagina 12 Linea 17)*

#### III. Giustificazione:

- *Essenziale per il calcolo dei costi e l'applicazione degli sconti promozionali.*

## **faq.xml**

### *a) Elemento <id>*

#### I. Posizione:

*Pagina 19 Linea 33*

#### II. Riferimento specifica:

- *"Consultazione FAQ" (Gestione domande - Pagina 12 Linea 23)*

#### III. Giustificazione:

- *Indispensabile per identificare univocamente le FAQ durante le operazioni di modifica.*

### *b) Elemento <answer>*

#### I. Posizione:

*Pagina 20 Linea 2*

#### II. Riferimento specifica:

- *"Consultazione FAQ" (Visualizzazione risposte - Pagina 12 Linea 22)*

#### III. Giustificazione:

- *Cruciale per fornire soluzioni immediate alle domande più frequenti degli utenti.*

## **feedback.xml**

### *a) Elemento <voto>*

#### I. Posizione:

*Pagina 20 Linea 19*

II. Riferimento specifica:

- *"Valutazione corsi" (Scala 1-5 - Pagina 12 Linea 26)*

III. Giustificazione:

- *Fondamentale per quantificare la soddisfazione degli utenti e generare statistiche.*

*b) Elemento <id\_user\_abbonamento>*

I. Posizione:

*Pagina 19 Linea 20*

II. Riferimento specifica:

- *"Partecipazione alla lezione" (Verifica abbonamento attivo - Pagina 9 Linea 3)*

III. Giustificazione:

- *Essenziale per validare l'effettiva partecipazione alle lezioni prima della valutazione.*

## **comunicazioni.xml**

*a) Attributo data\_invio*

I. Posizione:

*Pagina 21 Linea 2*

II. Riferimento specifica:

- *"Comunicazioni dallo staff" (Tracciamento temporale - Pagina 13 Linea 20)*

III. Giustificazione:

- *Necessario per ordinare cronologicamente i messaggi e determinare la rilevanza.*

*b) Elemento <titolo>*

I. Posizione:

*Pagina 21 Linea 3*

II. Riferimento specifica:

- *"Comunicazioni dallo staff" (Priorità messaggi - Pagina 13 Linea 22)*

III. Giustificazione:

- *Cruciale per identificare l'oggetto della comunicazione e migliorare l'usabilità.*

## **private\_chat.xml**

*a) Attributo sollecito\_inviato*

I. Posizione:

*Pagina 21 Linea 26*

II. Riferimento specifica:

- *"Messaggistica privata" (Notifiche automatiche - Pagina 13 Linea 3)*

III. Giustificazione:

- *Fondamentale per gestire il sistema di solleciti dopo 48 ore senza risposta.*

*b) Attributo risposta*

I. Posizione:

*Pagina 21 Linea 26*

II. Riferimento specifica:

- *"Messaggistica privata" (Tracciamento conversazioni - Pagina 13 Linea 1)*

III. Giustificazione:

- *Essenziale per distinguere messaggi iniziali da risposte e calcolare tempi di attesa.*

## **user\_abbonamenti.xml**

*a) Elemento <stato>*

I. Posizione:

*Pagina 22 Linea 23*

II. Riferimento specifica:

- *"Abbonamento" (Stati attivo/scaduto - Pagina 8 Linea 21)*

III. Giustificazione:

- *Indispensabile per abilitare/disabilitare funzionalità basate sullo stato corrente.*

*b) Elemento <max\_prenotazioni\_settimanali>*

I. Posizione:

*Pagina 22 Linea 27*

II. Riferimento specifica:

- *"Abbonamento" (Vincoli utilizzo - Pagina 8 Linea 21)*

III. Giustificazione:

- *Cruciale per applicare i limiti contrattuali nelle prenotazioni.*

## **forum.xml**

*a) Elemento <consensi>*

I. Posizione:

*Pagina 24 Linea 2*

II. Riferimento specifica:

- *"Sistema di Reputazione" (Calcolo punteggio - Pagina 14 Linea 2)*

III. Giustificazione:

- *Fondamentale per quantificare l'apprezzamento dei contributi nella community.*

*b) Attributo id in <thread>*

I. Posizione:

*Pagina 23 Linea 26*

II. Riferimento specifica:

- *"Forum pubblico" (Identificazione discussioni - Pagina 14 Linea 14)*

III. Giustificazione:

- *Essenziale per riferimenti incrociati e generazione FAQ da discussioni.*

## **reputazione.xml**

a) Elemento <punteggio>

I. Posizione:

*Pagina 26 Linea 24*

II. Riferimento specifica:

- *"Sistema di Reputazione" (Range 0-100 - Pagina 14 Linea 1)*

III. Giustificazione:

- *Necessario per implementare privilegi differenziati nel forum basati sulla reputazione.*

b) Elemento <ultima\_attivita>

I. Posizione:

*Pagina 26 Linea 25*

II. Riferimento specifica:

- *"Sistema di Reputazione" (Decadimento punteggio - Pagina 14 Linea 7)*

III. Giustificazione:

- *Cruciale per calcolare l'obsolescenza della reputazione degli utenti inattivi.*

## **statistiche.xml**

a) Elemento <valutazione\_media>

I. Posizione:

*Pagina 27 Linea 13*

II. Riferimento specifica:

- *"Statistiche Avanzate" (Monitoraggio corsi - Pagina 15 Linea 6)*

III. Giustificazione:

- *Fondamentale per valutare l'efficacia dei corsi e guidare miglioramenti.*

b) Elemento <soddisfazione\_generale>

I. Posizione:

*Pagina 27 Linea 24*

II. Riferimento specifica:

- *"Statistiche Avanzate" (Indice complessivo - Pagina 15 Linea 16)*

III. Giustificazione:

- *Essenziale per misurare il successo complessivo della palestra.*

## **admin\_audit.xml**

### *a) Attributo type*

#### I. Posizione:

*Pagina 28 Linea 9*

#### II. Riferimento specifica:

- *"Forum pubblico" (Tipologia operazioni - Pagina 14 Linea 25)*

#### III. Giustificazione:

- *Indispensabile per catalogare le azioni amministrative (modifica/elimina/fissa/blocca).*

### *b) Attributo admin*

#### I. Posizione:

*Pagina 28 Linea 9*

#### II. Riferimento specifica:

- *"Amministratore" (Tracciamento responsabili - Pagina 14 Linea 24)*

#### III. Giustificazione:

- *Cruciale per identificare gli autori delle modifiche e garantire accountability.*

## **lezioni.xml**

### *a) Elemento <posti\_totali>*

#### I. Posizione:

*Pagina 29 Linea 4*

#### II. Riferimento specifica:

- *"Prenotazione corsi" (Controllo disponibilità - Pagina 12 Linea 3)*

#### III. Giustificazione:

- *Fondamentale per implementare i vincoli di capienza delle sale e storico prenotazioni per utente.*

### *b) Attributo stato in <prenotante>*

#### I. Posizione:

*Pagina 29 Linea 6*

#### II. Riferimento specifica:

- *"Gestione Prenotazioni" (Stati confermato/cancellato - Pagina 13 Linea 17)*

#### III. Giustificazione:

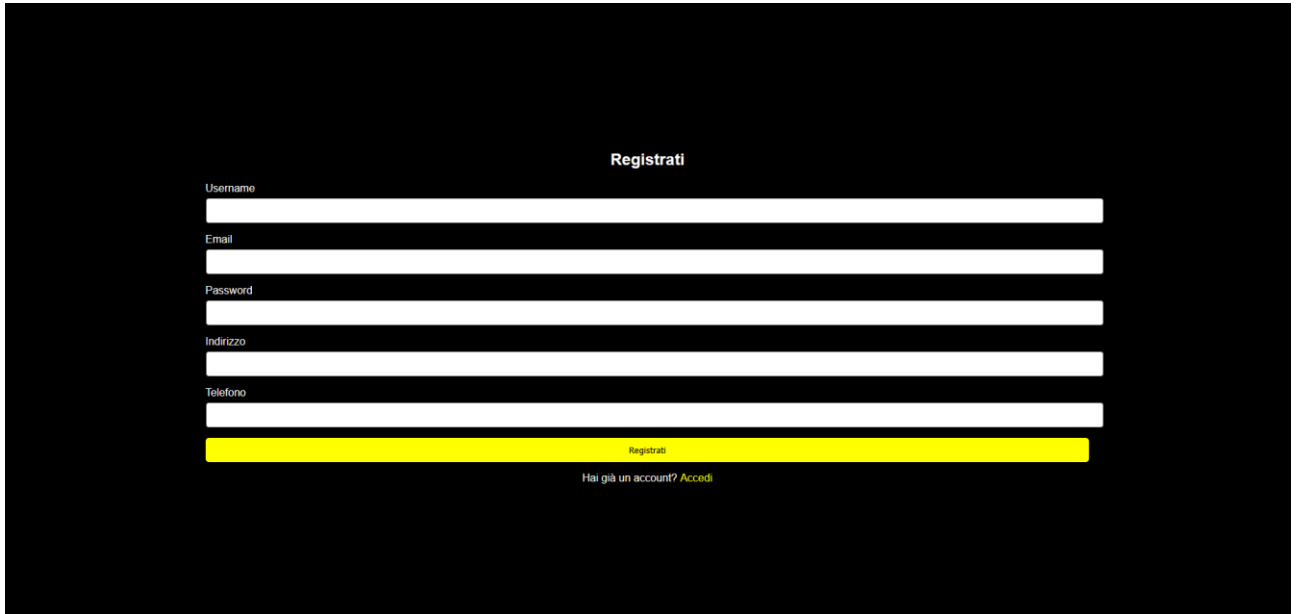
- *Essenziale per gestire le variazioni delle prenotazioni e l'overbooking.*

## Capitolo 2 – Manuale Utente

### 1. REGISTRAZIONE ACCOUNT

*Descrizione: Un visitatore può accedere all'area riservata per creare il proprio account.*

*Screenshot:*

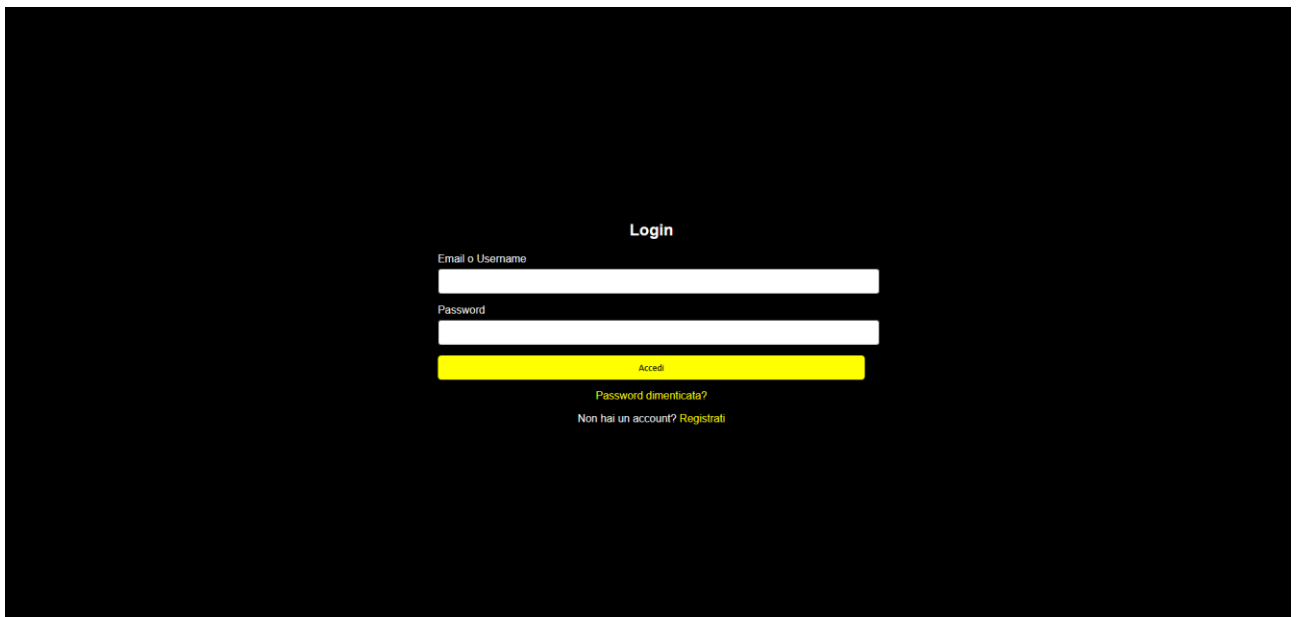


The screenshot shows a registration form titled "Registrati" on a dark background. The form contains six input fields: "Username", "Email", "Password", "Indirizzo", and "Telefono". Below these fields is a large yellow button labeled "Registrati". At the bottom, there is a link that says "Hai già un account? [Accedi](#)".

### 2. LOGIN

*Descrizione: Un utente registrato può effettuare il login con le proprie credenziali e accedere alla propria area riservata.*

*Screenshot:*

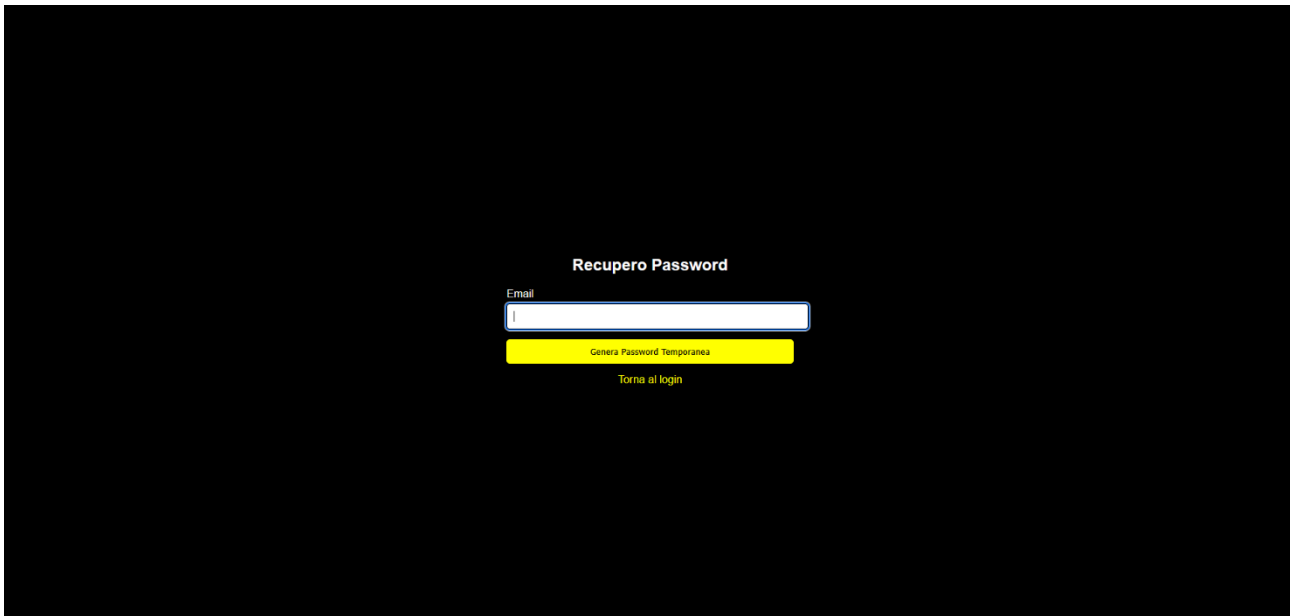


The screenshot shows a login form titled "Login" on a dark background. The form contains two input fields: "Email o Username" and "Password". Below these fields is a large yellow button labeled "Accedi". At the bottom, there are two links: "Password dimenticata?" and "Non hai un account? [Registrati](#)".

### 3. RECUPERA PASSWORD

*Descrizione: L'utente registrato può recuperare la propria password tramite un meccanismo sicuro. Viene generata una password temporanea che permette il login, dopodiché l'utente è obbligato a impostare una nuova password nel proprio profilo. Tutte le password sono memorizzate nel database in forma cifrata (hashata).*

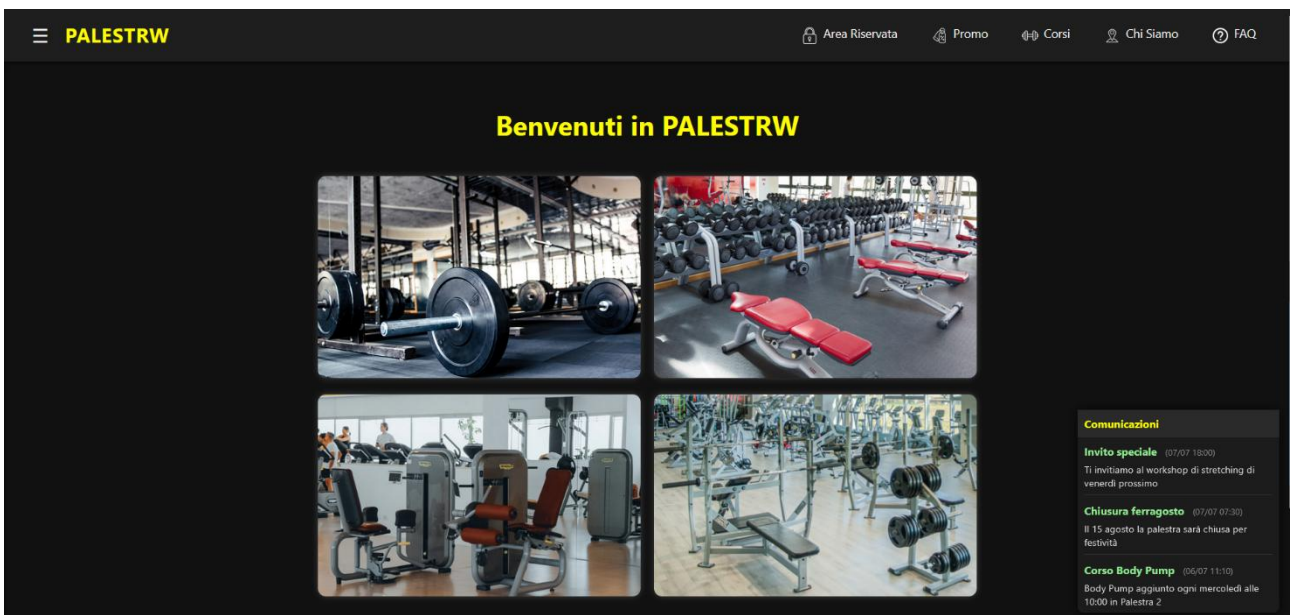
*Screenshot:*



### 4. HOME PAGE PER UN VISITATORE

*Descrizione: questa è la home page visualizzata da un visitatore.*

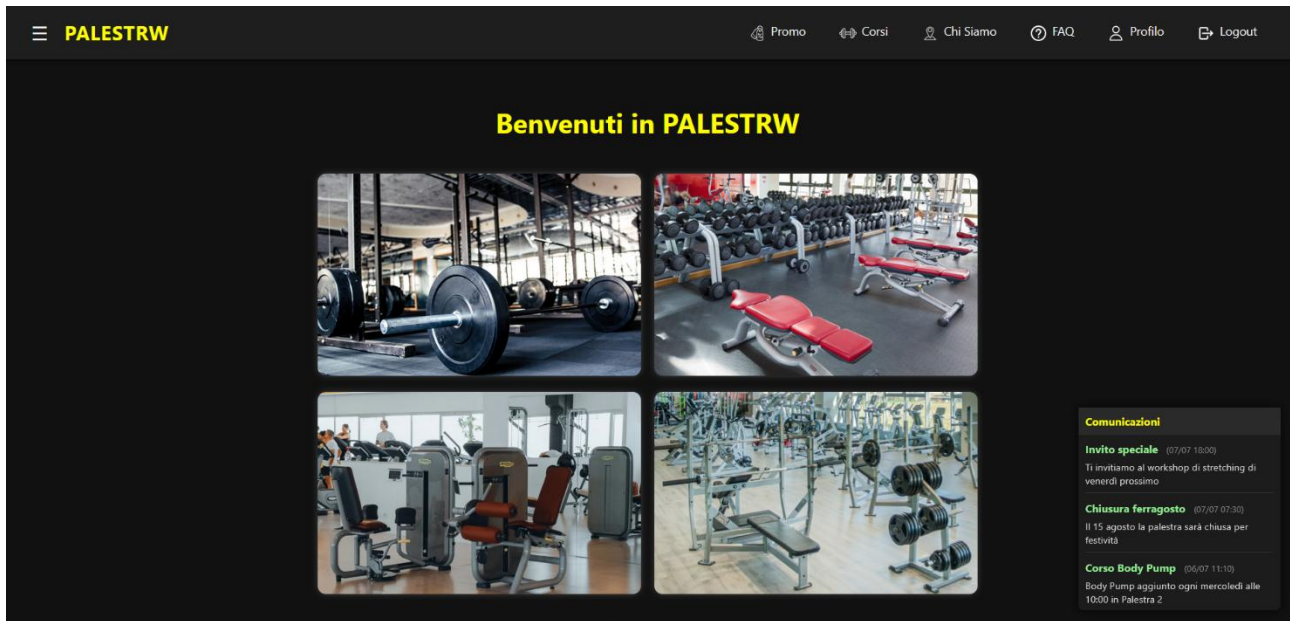
*Screenshot:*



### 5. HOME PAGE PER UN UTENTE LOGGATO

*Descrizione: questa è la home page visualizzata da un utente loggato.*

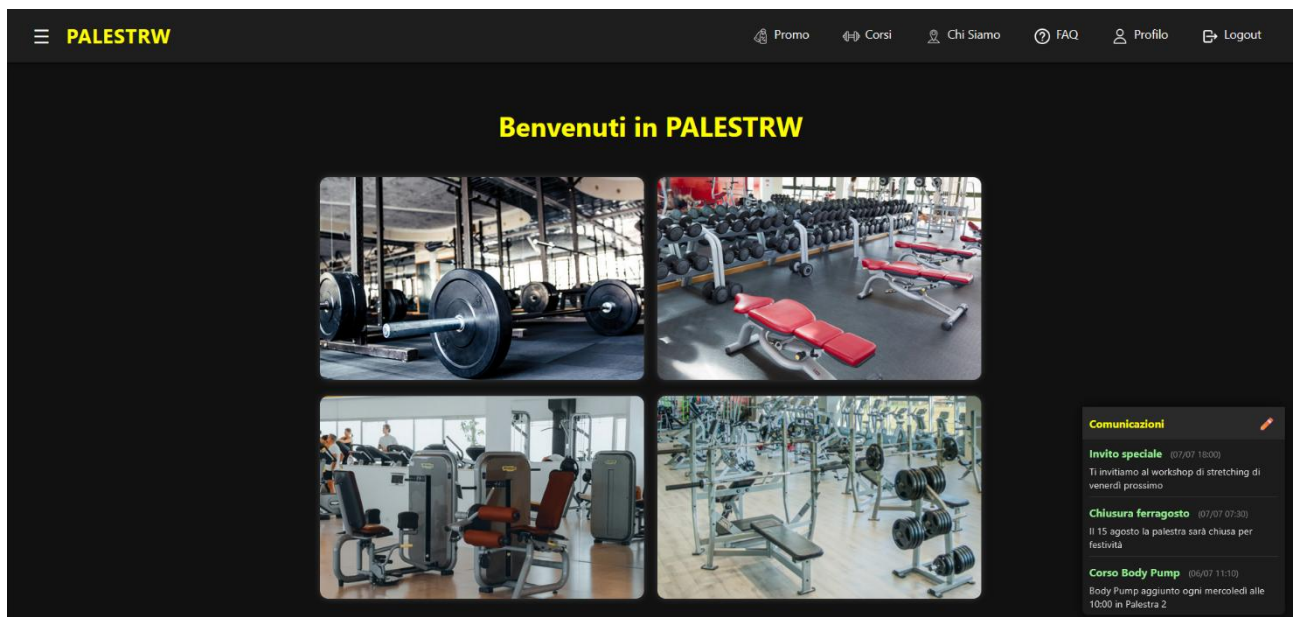
Screenshot:



## 6. HOME PAGE PER ADMIN

Descrizione: questa è la home page visualizzata da un admin.

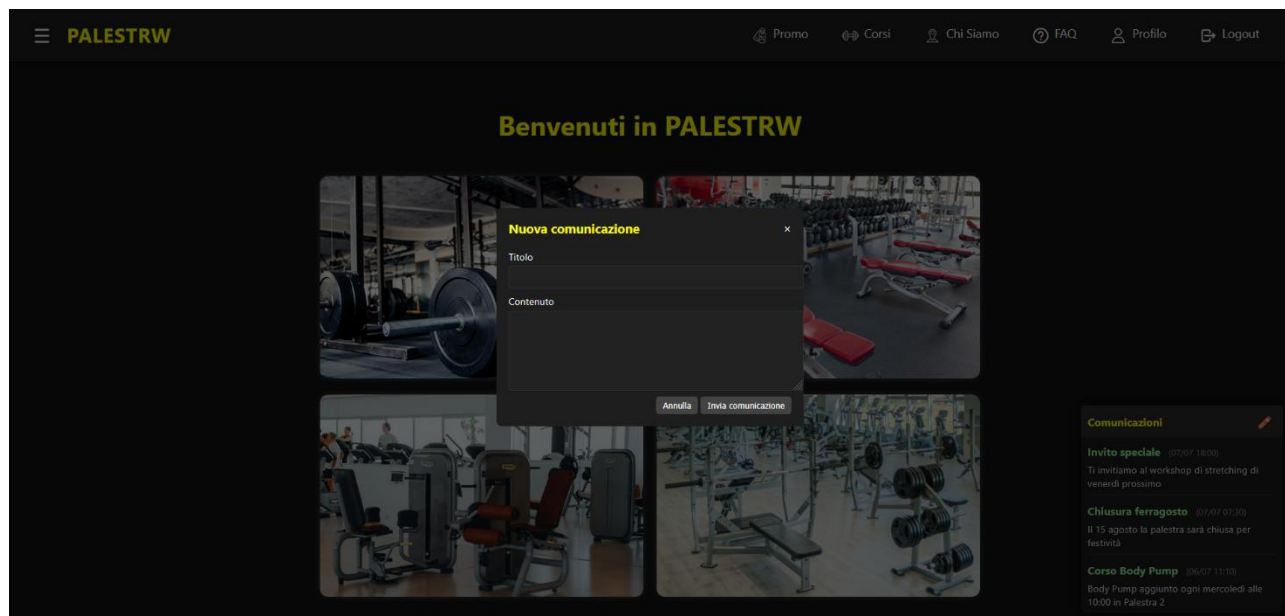
Screenshot:



## 7. AGGIUNGI COMUNICAZIONE

*Descrizione: l'admin può aggiungere una nuova comunicazione visibile a tutta la palestra tramite l'apposito pulsante.*

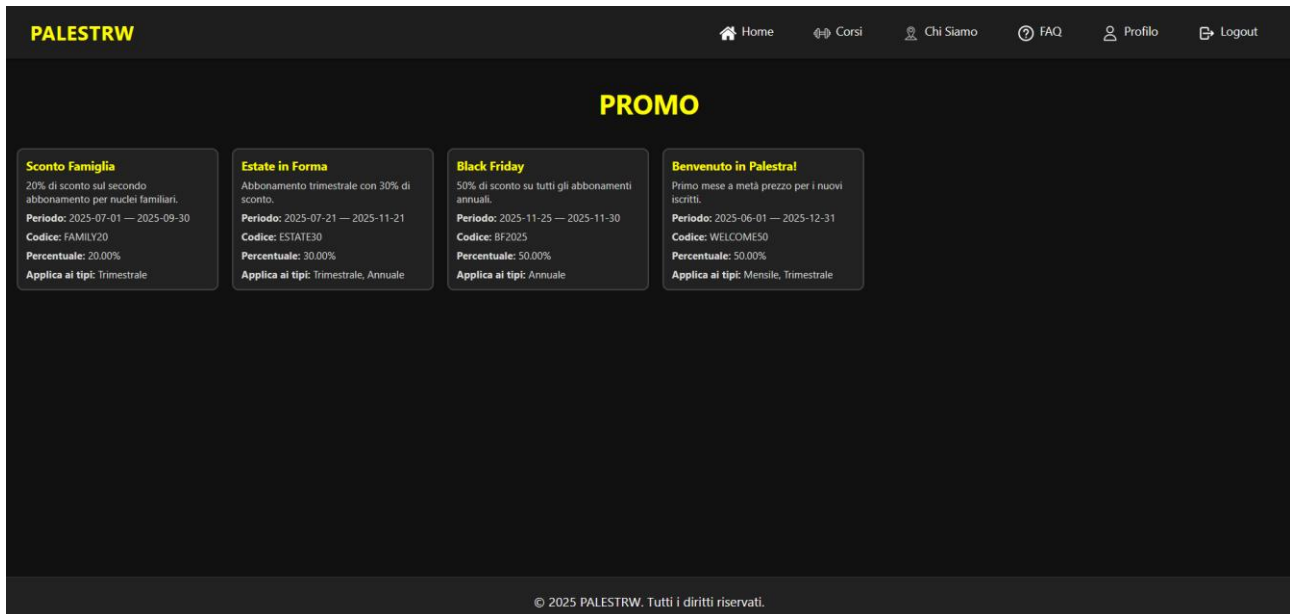
*Screenshot:*



## 8. PROMO PER UTENTI

*Descrizione: questa è la pagina delle promo visualizzata dagli utenti e visitatori.*

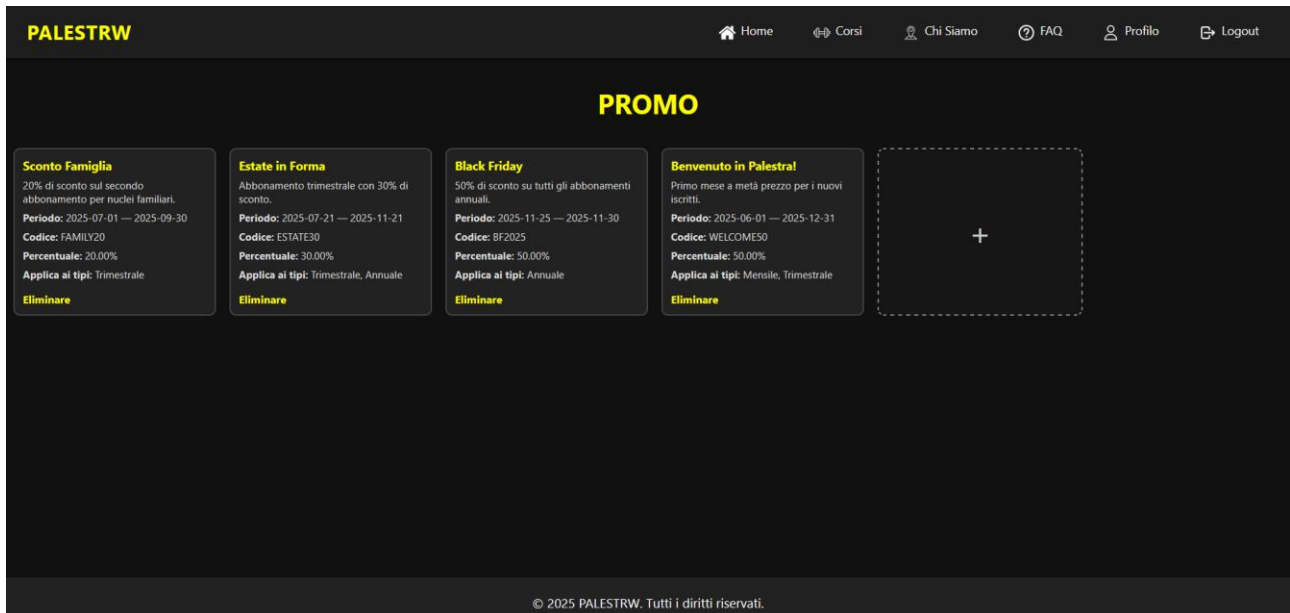
*Screenshot:*



## 9. PROMO PER ADMIN

*Descrizione: questa è la pagina delle promo visualizzata dall'admin.*

*Screenshot:*



## 10. CORSI PER VISITATORE

*Descrizione: questa è la pagina dei corsi per un visitatore o un utente senza abbonamento.*

Screenshot:

The screenshot shows the PALESTRW website with a dark theme. The header includes the logo and navigation links: Area Riservata, Home, Promo, Chi Siamo, and FAQ. The main section is titled 'Le nostre lezioni' and contains a table with three rows of classes.

| Corso                 | Descrizione                                                                 | Data & Ora       | Prenota             | Prenotati |
|-----------------------|-----------------------------------------------------------------------------|------------------|---------------------|-----------|
| Yoga Mattutino        | Un corso rilassante per iniziare la giornata con energia positiva.          | 21/10/2025 05:11 | Login + Abbonamento | 1/21      |
| HIIT Sera             | Allenamento ad alta intensità per bruciare calorie e migliorare resistenza. | 18/09/2025 21:21 | Login + Abbonamento | 0/23      |
| Stretching & Mobilità | Sessione per migliorare flessibilità e recupero muscolare.                  | 19/09/2025 09:00 | Login + Abbonamento | 0/34      |

At the bottom, there is a copyright notice: © 2025 PALESTRW. Tutti i diritti riservati.

## 11. CORSI PER UTENTE ABBONATO

Descrizione: questa è la pagina dei corsi visualizzata da un utente con abbonamento.

Screenshot:

The screenshot shows the PALESTRW website for a subscribed user. The header includes the logo and navigation links: Home, Promo, Chi Siamo, FAQ, Profilo, and Logout. The main section is titled 'Le nostre lezioni' and contains a table with three rows of classes. Below this table, there is a section titled 'Le tue prenotazioni future' with a table showing a future booking. At the bottom, there is a section titled 'Lezioni già svolte: lascia una valutazione' with a table showing past classes and their ratings.

| Corso                 | Descrizione                                                                 | Data & Ora       | Prenota       | Prenotati |
|-----------------------|-----------------------------------------------------------------------------|------------------|---------------|-----------|
| Yoga Mattutino        | Un corso rilassante per iniziare la giornata con energia positiva.          | 21/10/2025 05:11 | Già prenotato | 1/21      |
| HIIT Sera             | Allenamento ad alta intensità per bruciare calorie e migliorare resistenza. | 18/09/2025 21:21 | Prenota       | 0/23      |
| Stretching & Mobilità | Sessione per migliorare flessibilità e recupero muscolare.                  | 19/09/2025 09:00 | Prenota       | 0/34      |

**Le tue prenotazioni future**

| Corso          | Data             | Elimina |
|----------------|------------------|---------|
| Yoga Mattutino | 21/10/2025 05:11 |         |

**Lezioni già svolte: lascia una valutazione**

| Corso                                 | La tua valutazione |
|---------------------------------------|--------------------|
| Yoga Mattutino (05/09/2025 09:00)     | ★ ★ ★ ★ ★          |
| Pilates Intermedio (16/08/2025 18:30) | ★ ★ ★ ★ ★          |
| HIIT Sera (20/08/2025 19:30)          | ★ ★ ★ ★ ★          |

At the bottom, there is a copyright notice: © 2025 PALESTRW. Tutti i diritti riservati.

## 12. CORSI PER ADMIN

Descrizione: questa è la pagina dei corsi visualizzata da un admin.

Screenshot:

**PALESTRW** Home Promo Chi Siamo FAQ Profilo Logout

### Le nostre lezioni

+ Aggiungi Nuovo Corso Base + Aggiungi Nuova Lezione

| Corso                 | Descrizione                                                                 | Data & Ora       | Prenota                 | Prenotati | Modifica | Elimina |
|-----------------------|-----------------------------------------------------------------------------|------------------|-------------------------|-----------|----------|---------|
| Yoga Mattutino        | Un corso rilassante per iniziare la giornata con energia positiva.          | 21/10/2025 05:11 | Admin non può prenotare | 1/21      |          |         |
| HIIT Sera             | Allenamento ad alta intensità per bruciare calorie e migliorare resistenza. | 18/09/2025 21:21 | Admin non può prenotare | 0/23      |          |         |
| Stretching & Mobilità | Sessione per migliorare flessibilità e recupero muscolare.                  | 19/09/2025 09:00 | Admin non può prenotare | 0/34      |          |         |

### Gestione Corsi Base

| Nome                  | Descrizione                                                                 | Difficoltà | Lezioni | Feedback | Media | Modifica | Elimina |
|-----------------------|-----------------------------------------------------------------------------|------------|---------|----------|-------|----------|---------|
| Yoga Mattutino        | Un corso rilassante per iniziare la giornata con energia positiva.          | Facile     | 4       | 4        | 3.75  |          |         |
| Pilates Intermedio    | Tonifica il tuo corpo con movimenti controllati e respirazione consapevole. | Medio      | 2       | 1        | 3.00  |          |         |
| HIIT Sera             | Allenamento ad alta intensità per bruciare calorie e migliorare resistenza. | Difficile  | 2       | 1        | 2.00  |          |         |
| Stretching & Mobilità | Sessione per migliorare flessibilità e recupero muscolare.                  | Facile     | 2       | 1        | 4.00  |          |         |
| Cardio leggero        | Attività aerobica a basso impatto, adatta a tutti i livelli.                | Facile     | 0       | 0        | -     |          |         |

© 2025 PALESTRW. Tutti i diritti riservati.

## 13. AGGIUNGI CORSO

Descrizione: l'admin, nella pagina dei corsi, può aggiungere un nuovo corso.

Screenshot:

**PALESTRW** Home Promo Chi Siamo FAQ Profilo Logout

### Le nostre lezioni

+ Aggiungi Nuovo Corso Base + Aggiungi Nuova Lezione

#### Aggiungi Nuovo Corso Base

Nome Corso:

Descrizione:

Difficoltà:

Salva Corso Annulla

| Corso                 | Descrizione                                                                 | Data & Ora       | Prenota                 | Prenotati | Modifica | Elimina |
|-----------------------|-----------------------------------------------------------------------------|------------------|-------------------------|-----------|----------|---------|
| Yoga Mattutino        | Un corso rilassante per iniziare la giornata con energia positiva.          | 21/10/2025 05:11 | Admin non può prenotare | 1/21      |          |         |
| HIIT Sera             | Allenamento ad alta intensità per bruciare calorie e migliorare resistenza. | 18/09/2025 21:21 | Admin non può prenotare | 0/23      |          |         |
| Stretching & Mobilità | Sessione per migliorare flessibilità e recupero muscolare.                  | 19/09/2025 09:00 | Admin non può prenotare | 0/34      |          |         |

### Gestione Corsi Base

## 14. AGGIUNGI LEZIONE

Descrizione: l'admin, nella pagina dei corsi, può aggiungere una nuova lezione.

Screenshot:

The screenshot shows the 'Le nostre lezioni' (Our lessons) page. At the top, there are two buttons: '+ Aggiungi Nuovo Corso Base' and '+ Aggiungi Nuova Lezione'. Below them is a form titled 'Aggiungi Nuova Lezione'. The form has two main sections: 'Corso Base:' with a dropdown menu showing 'Yoga Mattutino', and 'Data e Ora Lezione:' with a date and time picker. Below these is a 'Posti Totali:' field with a slider. At the bottom of the form are two buttons: 'Salva Lezione' (green) and 'Annulla' (red). Below the form is a table of existing lessons.

| Corso                 | Descrizione                                                                 | Data e Ora       | Prenota                 | Prenotati | Modifica | Elimina |
|-----------------------|-----------------------------------------------------------------------------|------------------|-------------------------|-----------|----------|---------|
| Yoga Mattutino        | Un corso rilassante per iniziare la giornata con energia positiva.          | 21/10/2025 05:11 | Admin non può prenotare | 1/21      |          |         |
| HIIT Sera             | Allenamento ad alta intensità per bruciare calorie e migliorare resistenza. | 18/09/2025 21:21 | Admin non può prenotare | 0/23      |          |         |
| Stretching & Mobilità | Sessione per migliorare flessibilità e recupero muscolare.                  | 19/09/2025 09:00 | Admin non può prenotare | 0/34      |          |         |

Below the table is a section titled 'Gestione Corsi Base' with a table of course details.

| Nome           | Descrizione                                                        | Difficoltà | Lezioni | Feedback | Media | Modifica | Elimina |
|----------------|--------------------------------------------------------------------|------------|---------|----------|-------|----------|---------|
| Yoga Mattutino | Un corso rilassante per iniziare la giornata con energia positiva. | Facile     | 4       | 4        | 2.75  |          |         |

## 15. MODIFICA LEZIONE

Descrizione: l'admin può modificare una lezione esistente tramite l'apposito pulsante.

Screenshot:

The screenshot shows the 'Le nostre lezioni' (Our lessons) page. At the top, there are two buttons: '+ Aggiungi Nuovo Corso Base' and '+ Aggiungi Nuova Lezione'. Below them is a form titled 'Modifica Lezione'. The form has two main sections: 'Data e Ora:' with a date and time picker, and 'Posti:' with a slider. At the bottom of the form are two buttons: 'Salva' (yellow) and 'Annulla' (red). Below the form is a table of existing lessons.

| Corso                 | Descrizione                                                                 | Data e Ora       | Prenota                 | Prenotati | Modifica | Elimina |
|-----------------------|-----------------------------------------------------------------------------|------------------|-------------------------|-----------|----------|---------|
| Yoga Mattutino        | Un corso rilassante per iniziare la giornata con energia positiva.          | 21/10/2025 05:11 | Admin non può prenotare | 1/21      |          |         |
| HIIT Sera             | Allenamento ad alta intensità per bruciare calorie e migliorare resistenza. | 18/09/2025 21:21 | Admin non può prenotare | 0/23      |          |         |
| Stretching & Mobilità | Sessione per migliorare flessibilità e recupero muscolare.                  | 19/09/2025 09:00 | Admin non può prenotare | 0/34      |          |         |

Below the table is a section titled 'Gestione Corsi Base' with a table of course details.

| Nome                  | Descrizione                                                                 | Difficoltà | Lezioni | Feedback | Media | Modifica | Elimina |
|-----------------------|-----------------------------------------------------------------------------|------------|---------|----------|-------|----------|---------|
| Yoga Mattutino        | Un corso rilassante per iniziare la giornata con energia positiva.          | Facile     | 4       | 4        | 2.75  |          |         |
| Pilates Intermedio    | Tonifica il tuo corpo con movimenti controllati e respirazione consapevole. | Medio      | 2       | 1        | 3.00  |          |         |
| HIIT Sera             | Allenamento ad alta intensità per bruciare calorie e migliorare resistenza. | Difficile  | 2       | 1        | 2.00  |          |         |
| Stretching & Mobilità | Sessione per migliorare flessibilità e recupero muscolare.                  | Facile     | 2       | 1        | 4.00  |          |         |
| Cardio leggero        | Attività aerobica a basso impatto, adatta a tutti i livelli.                | Facile     | 0       | 0        | -     |          |         |

## 16. LISTA DEI PRENOTATI

*Descrizione: l'admin può visualizzare l'elenco dei prenotati cliccando sull'apposito pulsante nella pagina dei corsi.*

*Screenshot:*

**PALESTRW** Home Promo Chi Siamo FAQ Profilo Logout

### Le nostre lezioni

+ Aggiungi Nuovo Corso Base + Aggiungi Nuova Lezione

| Corso          | Descrizione                                                        | Data & Ora       | Prenota                 | Prenotati | Modifica | Elimina |
|----------------|--------------------------------------------------------------------|------------------|-------------------------|-----------|----------|---------|
| Yoga Mattutino | Un corso rilassante per iniziare la giornata con energia positiva. | 21/10/2025 05:11 | Admin non può prenotare | 1/21      |          |         |

#### Prenotanti per Yoga Mattutino

luisa (ID 405)

|                       |                                                                             |                  |                         |      |  |  |
|-----------------------|-----------------------------------------------------------------------------|------------------|-------------------------|------|--|--|
| HIIT Sera             | Allenamento ad alta intensità per bruciare calorie e migliorare resistenza. | 18/09/2025 21:21 | Admin non può prenotare | 0/23 |  |  |
| Stretching & Mobilità | Sessione per migliorare flessibilità e recupero muscolare.                  | 19/09/2025 09:00 | Admin non può prenotare | 0/34 |  |  |

### Gestione Corsi Base

| Nome                  | Descrizione                                                                 | Difficoltà | Lezioni | Feedback | Media | Modifica | Elimina |
|-----------------------|-----------------------------------------------------------------------------|------------|---------|----------|-------|----------|---------|
| Yoga Mattutino        | Un corso rilassante per iniziare la giornata con energia positiva.          | Facile     | 4       | 4        | 2.75  |          |         |
| Pilates Intermedio    | Tonifica il tuo corpo con movimenti controllati e respirazione consapevole. | Medio      | 2       | 1        | 3.00  |          |         |
| HIIT Sera             | Allenamento ad alta intensità per bruciare calorie e migliorare resistenza. | Difficile  | 2       | 1        | 2.00  |          |         |
| Stretching & Mobilità | Sessione per migliorare flessibilità e recupero muscolare.                  | Facile     | 2       | 1        | 1.00  |          |         |

## 17. CHI SIAMO UTENTI

*Descrizione: questa pagina mostra le informazioni generali della palestra.*

*Screenshot:*

**PALESTRW** Home Promo Corsi Profilo FAQ Logout

## CHI SIAMO

1. INFO  
2. Dove: Via Alessio Rossi 14, Latina  
3. Contatto: +39 365 564 8300

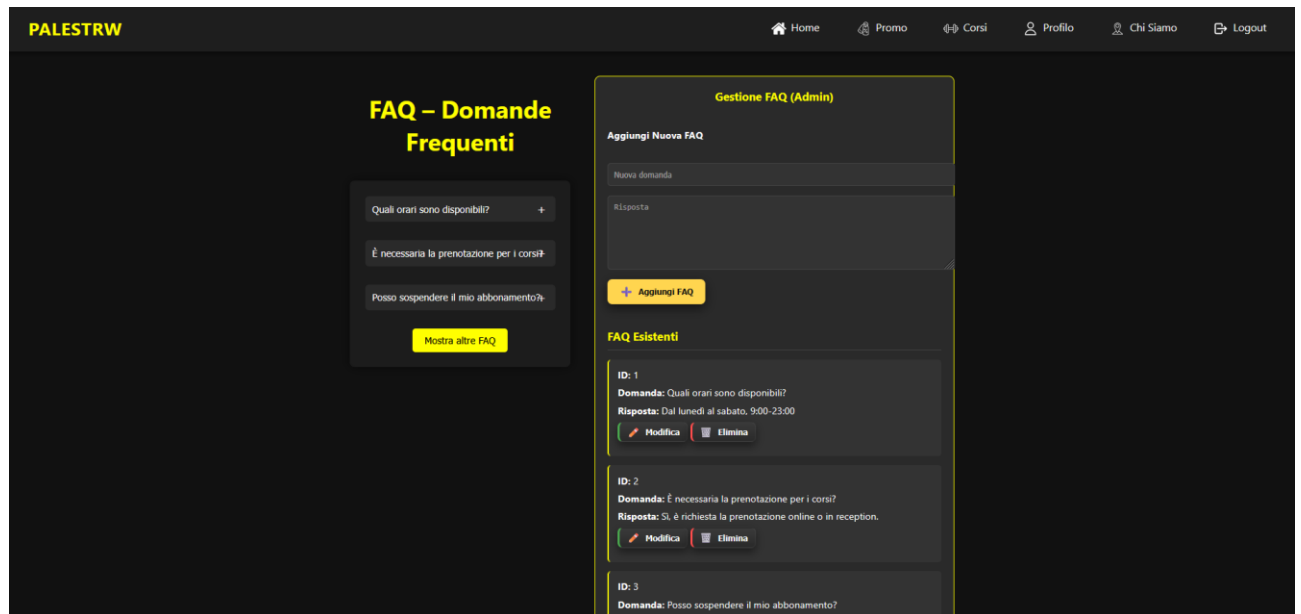
Clicca sulla mappa per aprirla

© 2025 PALESTRW. Tutti i diritti riservati.

## 18. FAQ ADMIN

*Descrizione: questa pagina mostra le FAQ, con le varie operazioni(aggiungi,modifica,elimina) su FAQ che può fare l'admin.*

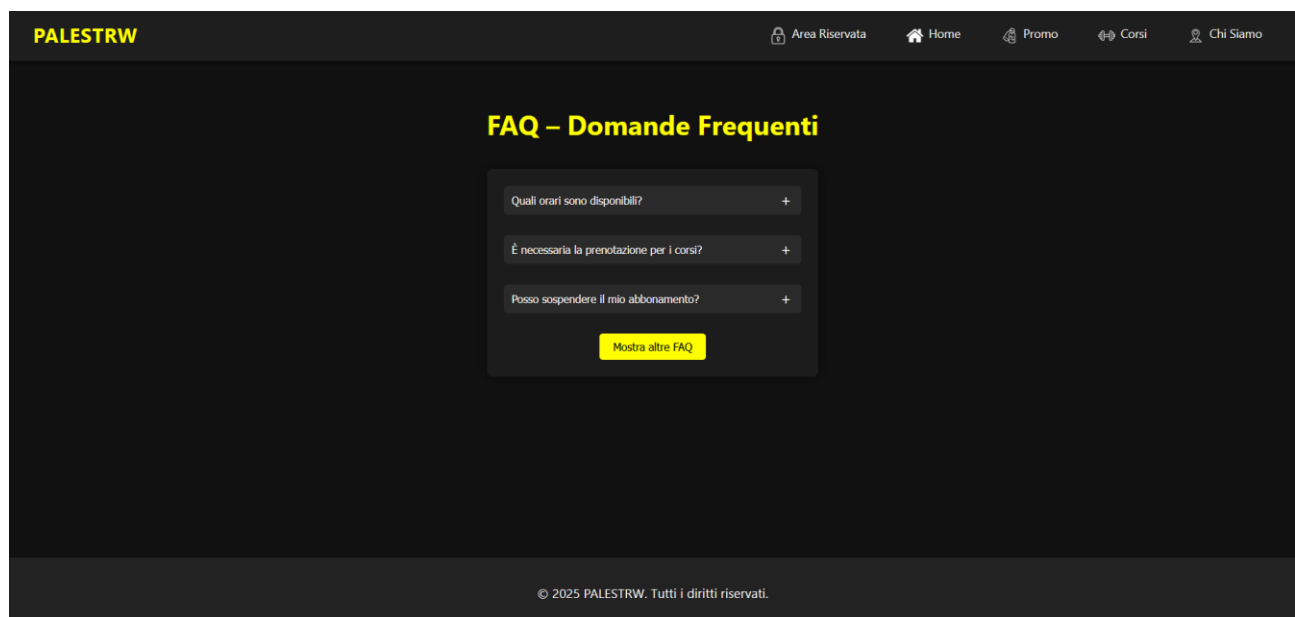
*Screenshot:*



## 19. FAQ UTENTE

*Descrizione: questa pagina mostra le FAQ, con le varie operazioni(aggiungi,modifica,elimina) su FAQ che può fare l'admin.*

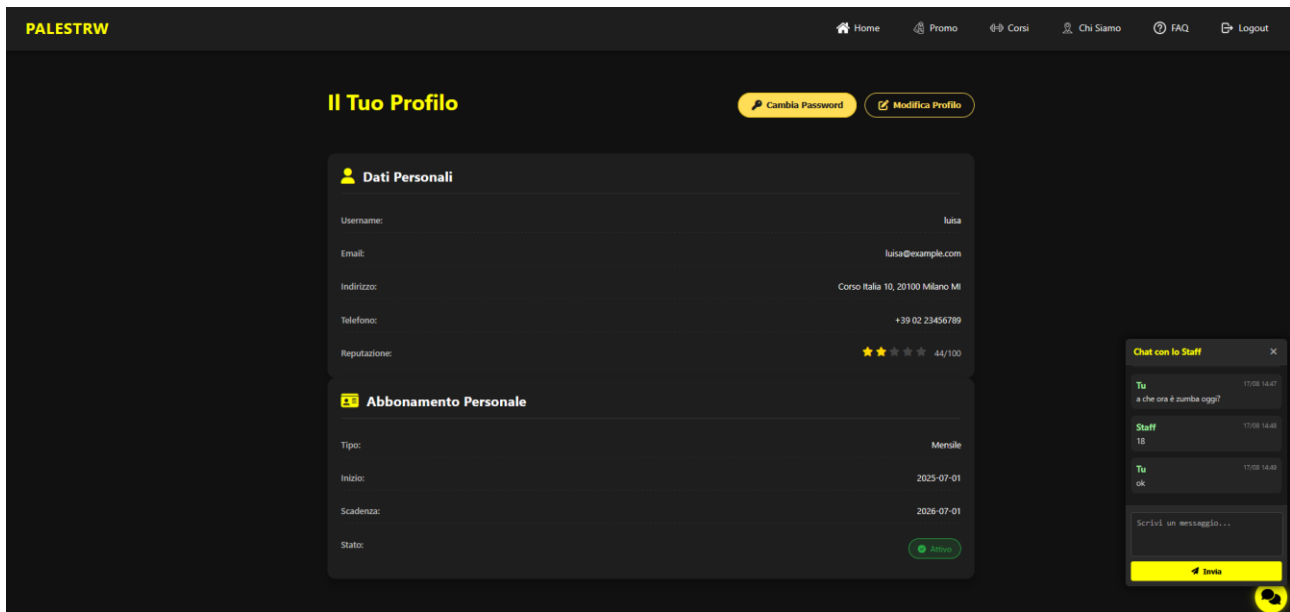
*Screenshot:*



## 20. PROFILO UTENTE ABBONATO

*Descrizione: questa pagina mostra il profilo dell'utente abbonato, con informazioni generali (con stato di reputazione), lo stato dell'abbonamento e l'accesso alla chat privata con lo staff*

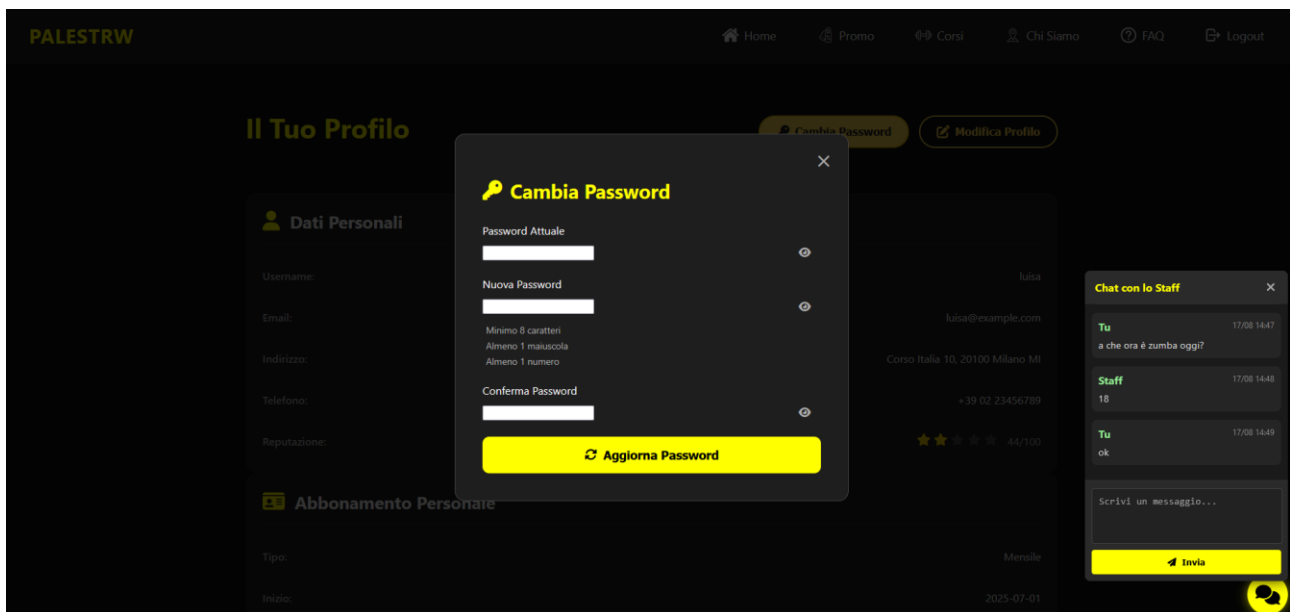
*Screenshot:*



## 21. CAMBIA PASSWORD UTENTE

*Descrizione: l'utente può cambiare la password nella pagina profilo.*

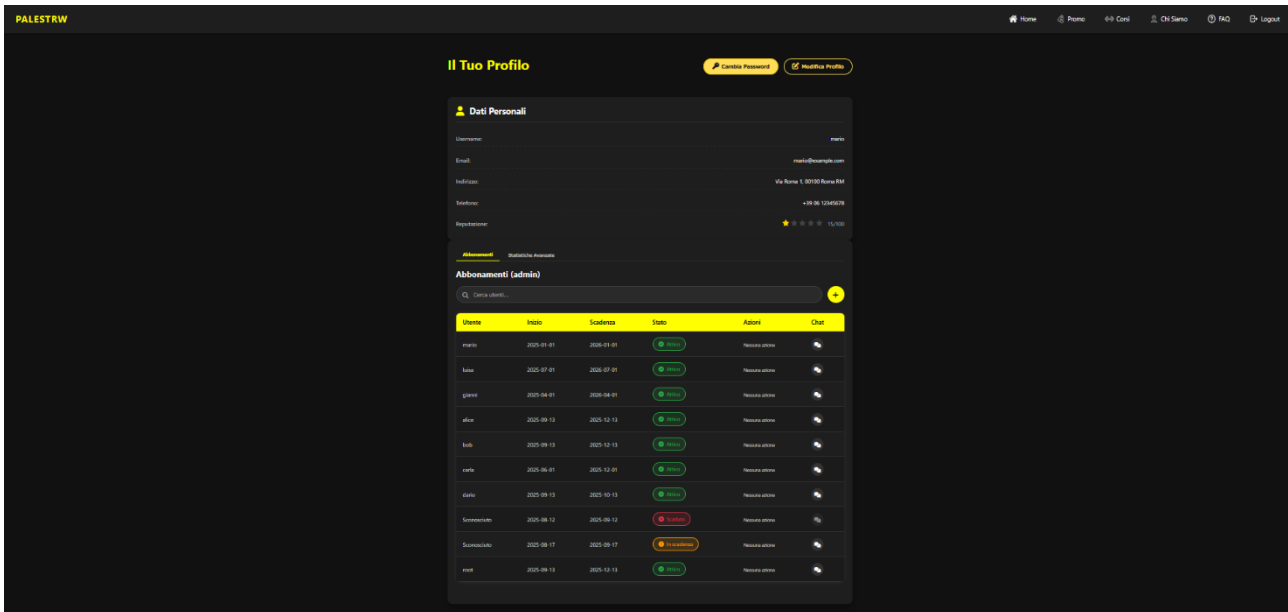
*Screenshot:*



## 22. PROFILO ADMIN

*Descrizione: questa pagina mostra il profilo dell'admin, con informazioni generali, elenco degli utenti con lo stato dell'abbonamento, accesso alla chat privata per ciascun utente, visualizzazione delle statistiche avanzate, barra di ricerca e pulsante per aggiungere un nuovo abbonato.*

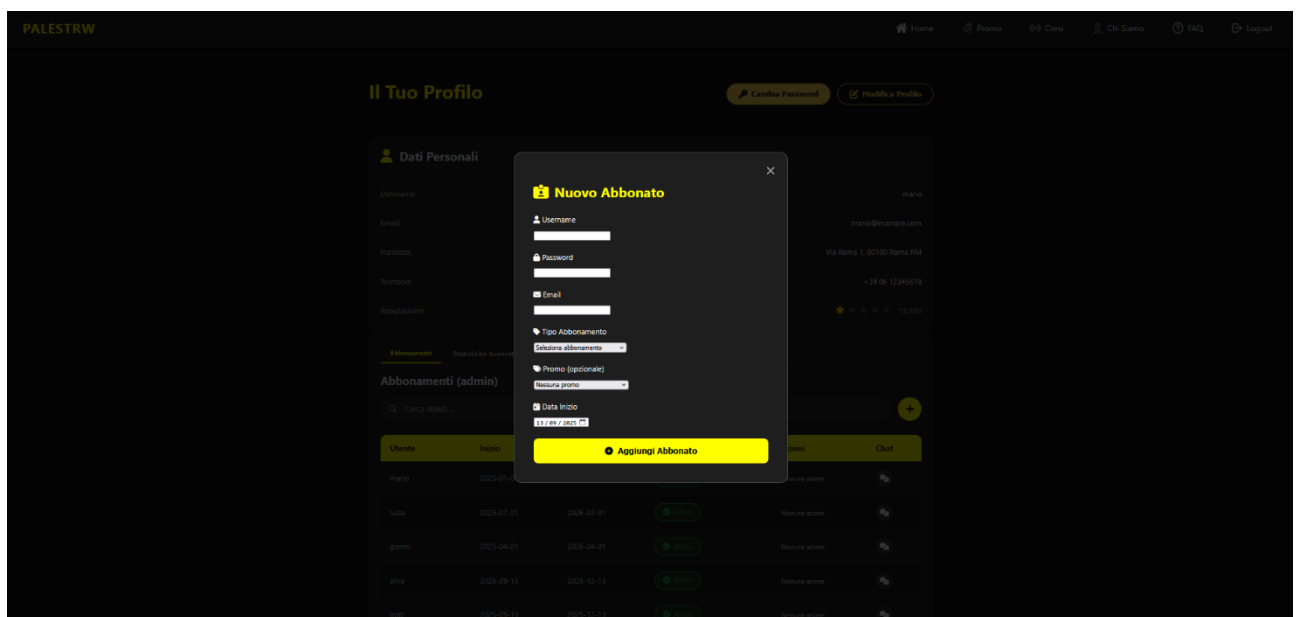
*Screenshot:*



## 23. AGGIUNGI ABBONATO

*Descrizione: l'admin può aggiungere un nuovo abbonato cliccando sull'apposito pulsante.*

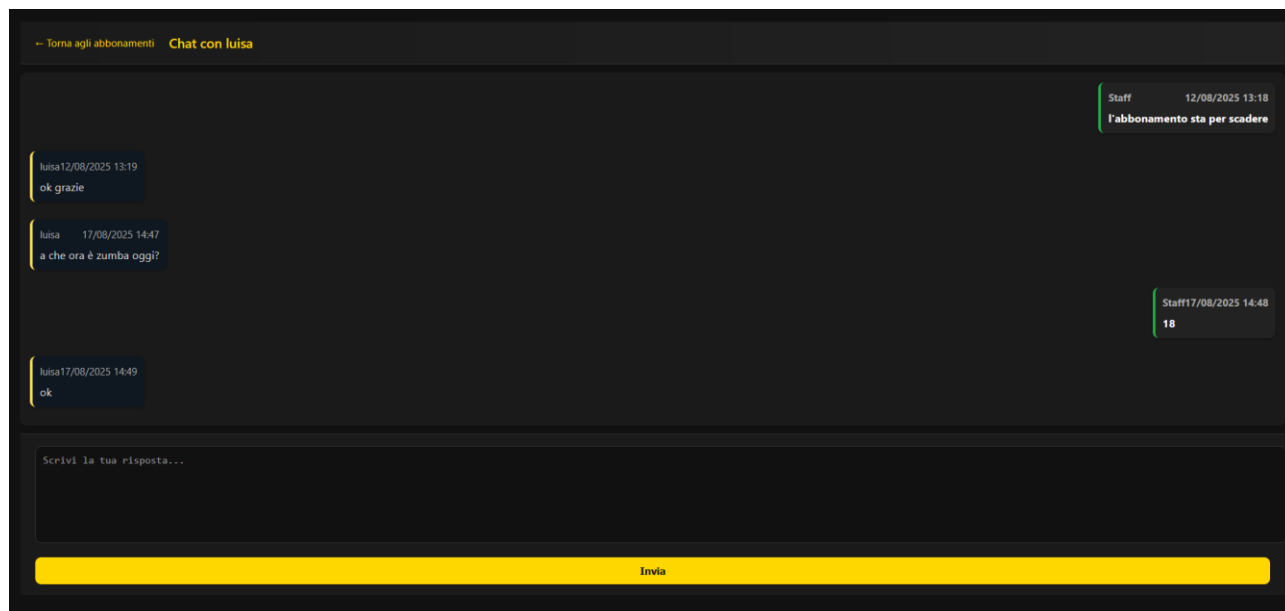
*Screenshot:*



## 24. PRIVATE CHAT

*Descrizione: questa è la chat privata visibile all'admin, che consente di comunicare con i diversi utenti con abbonamento attivo.*

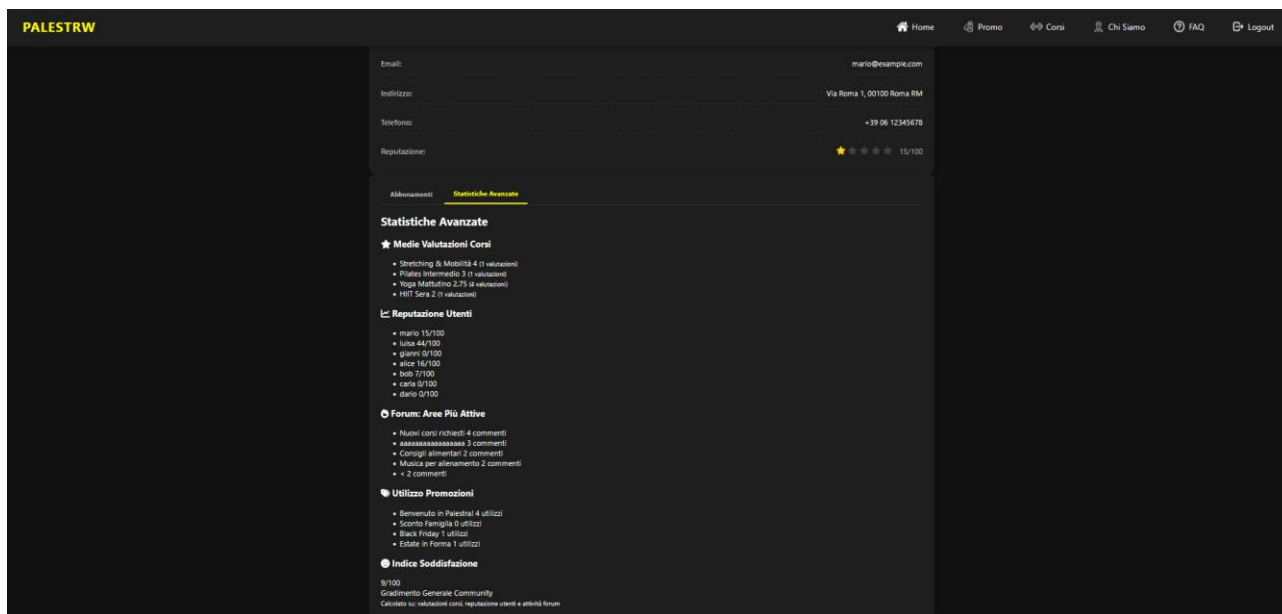
*Screenshot:*



## 25. STATISTICHE AVANZATE PER ADMIN

*Descrizione: questa sezione mostra le statistiche avanzate essenziali per l'admin. Vengono visualizzate le medie delle valutazioni dei corsi, la reputazione degli utenti, le aree più attive nel forum, l'utilizzo delle varie promozioni e l'indice di gradimento generale della community.*

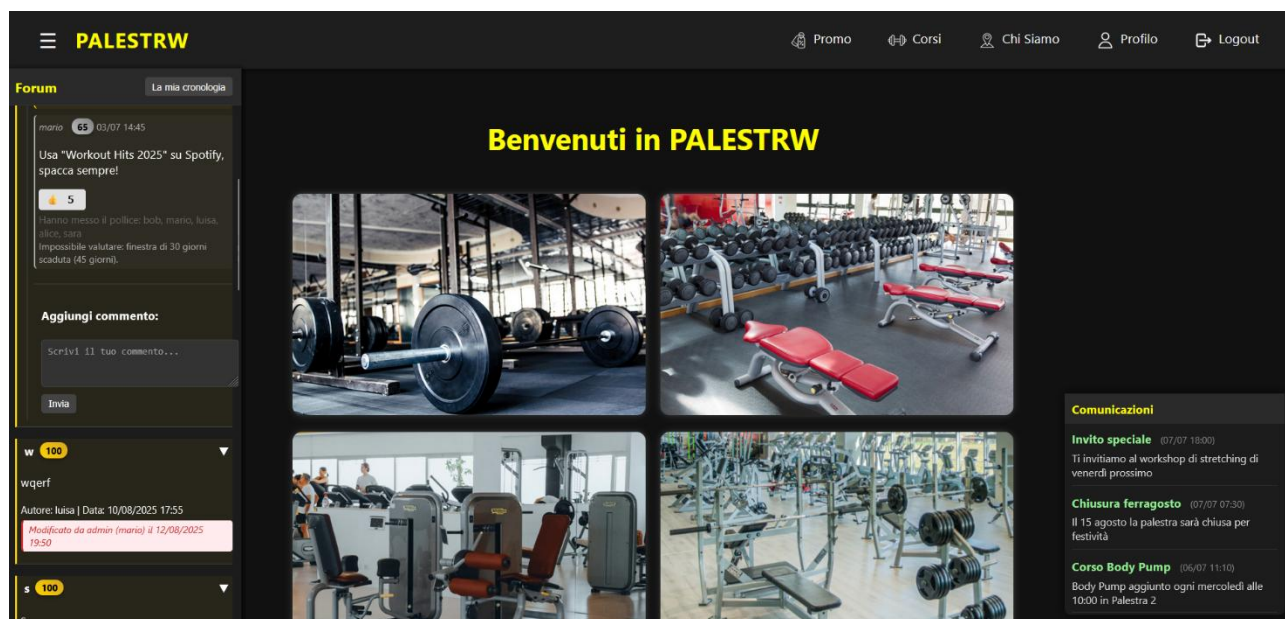
*Screenshot:*



## 26. FORUM UTENTE

*Descrizione: il forum è accessibile dalla home page. Qui utenti e admin possono discutere, condividere consigli e valutazioni, creando un ambiente collaborativo volto al miglioramento. La reputazione viene calcolata in base alla partecipazione al forum.*

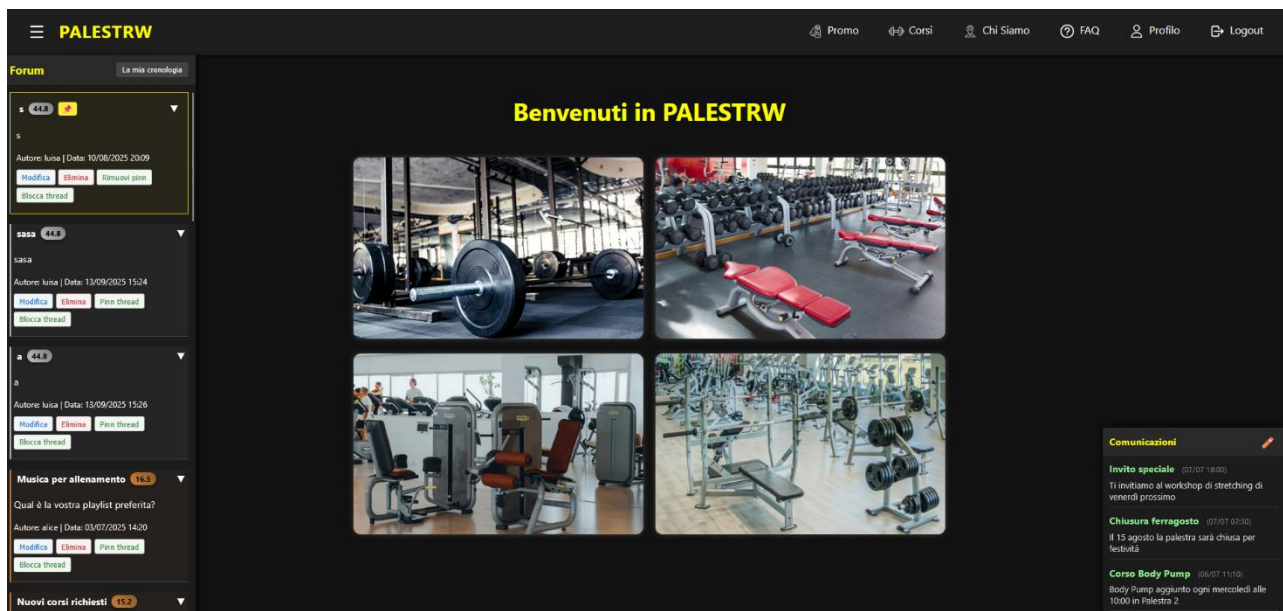
*Screenshot:*



## 27. FORUM ADMIN

*Descrizione: L'admin ha funzionalità avanzate come modifica e elimina di contenuti, bloccare thread attivi, fissare i messaggi più rilevanti.*

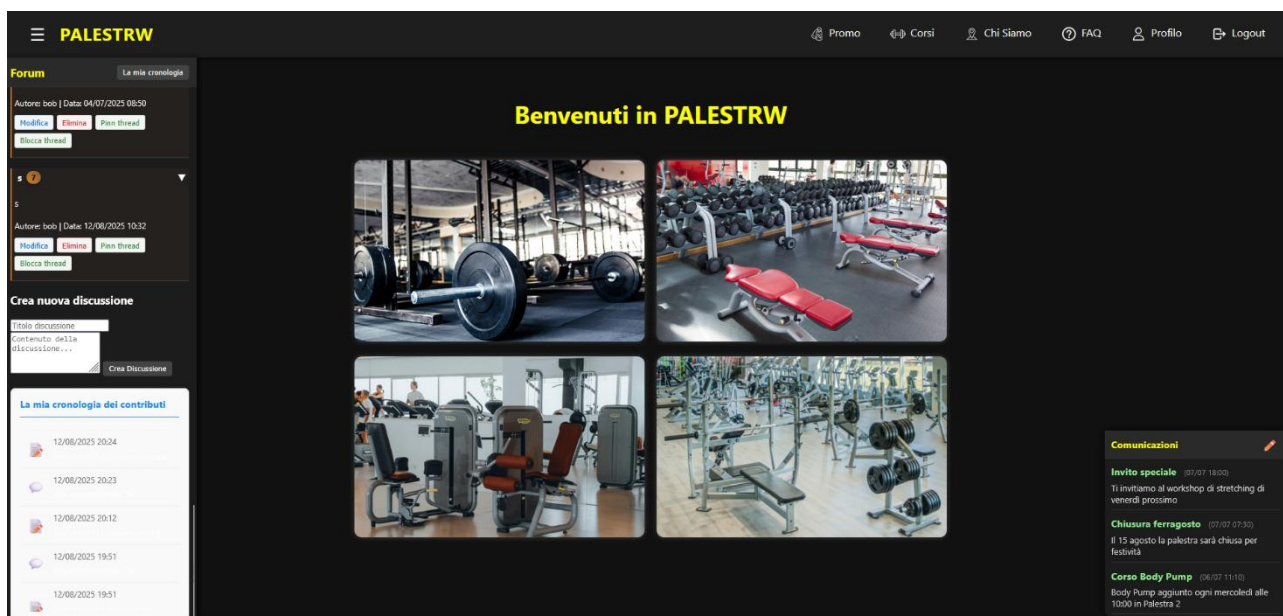
*Screenshot:*



## 28. CRONOLOGIA CONTRIBUTI FORUM

*Descrizione: Utenti e admin hanno accesso alla propria cronologia dei contributi al forum.*

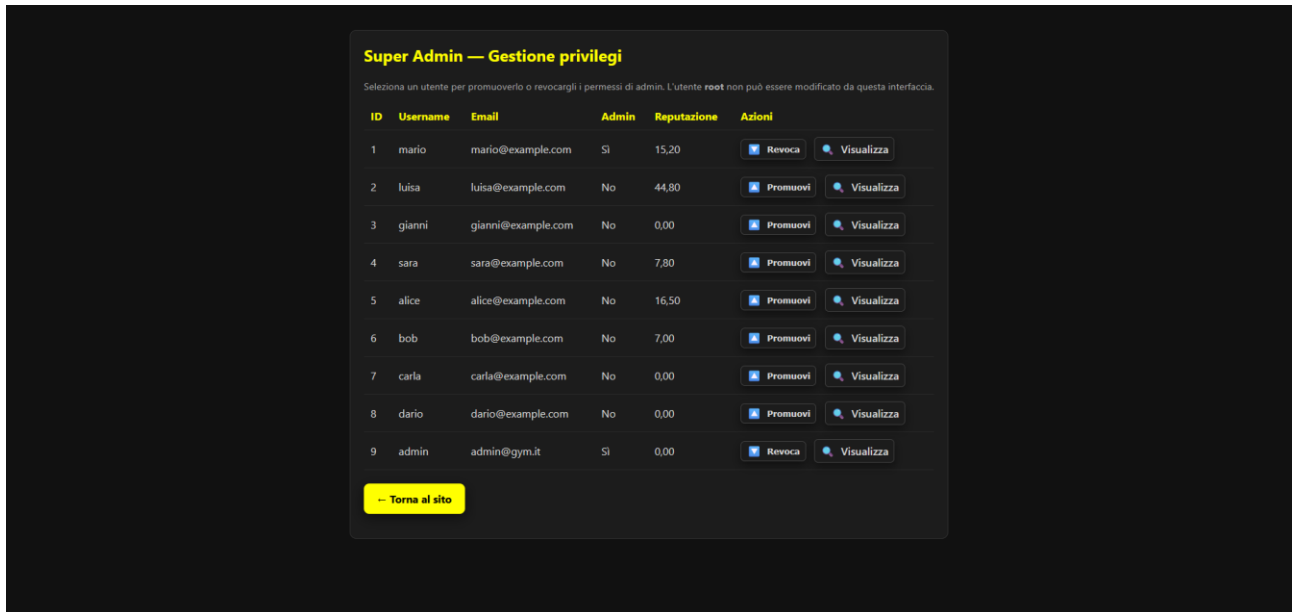
*Screenshot:*



## 29. GESTIONE PER “SUPERADMIN”

*Descrizione: in questa pagina il superadmin può promuovere o revocare la funzione di admin agli utenti. Inoltre può accedere al profilo di ogni utente.*

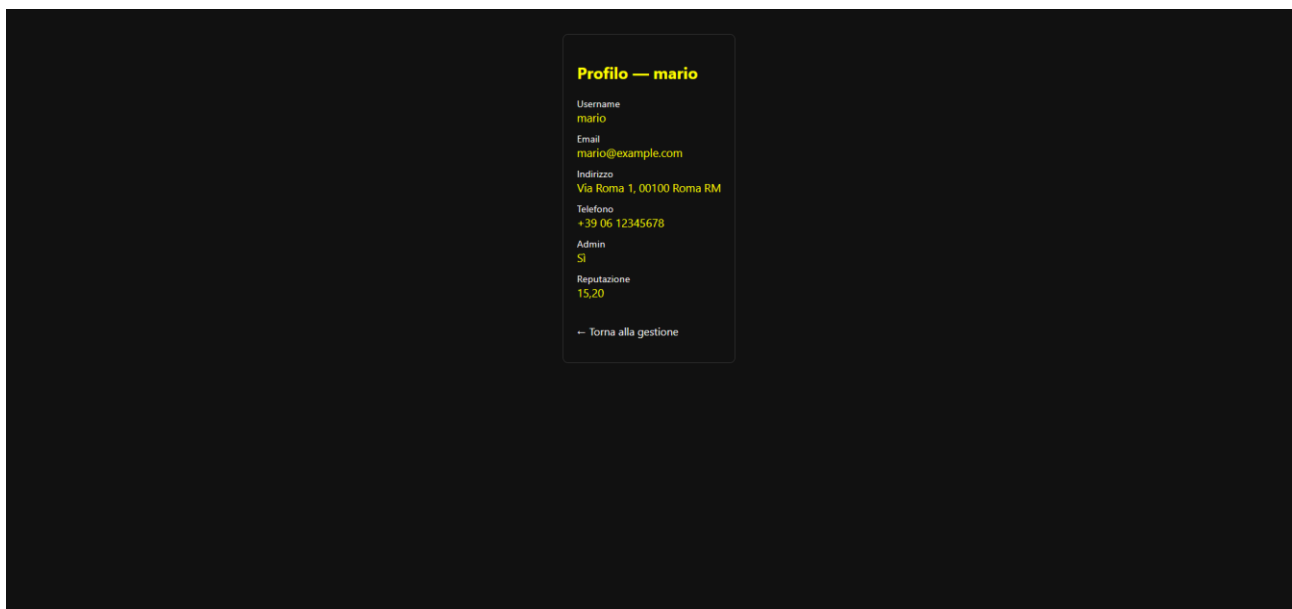
*Screenshot:*



## 30. PROFILO PUBBLICO PER SUPERADMIN

*Descrizione: in questa pagina il superadmin può visualizzare il profilo dell'utente interessato.*

*Screenshot:*



# Capitolo 3 – Programmazione

## 1. SCRIPT:login.php

### SCOPO

Gestire l'autenticazione degli utenti (login): verifica credenziali, creazione della sessione utente, redirectione verso la pagina appropriata (home) e supporto a meccanismi di sicurezza (CSRF, password hashnate, cambio password temporanea).

### DIPENDENZE

- db.php : fornisce la connessione \$conn al database (MySQLi).
- Funzione h() definita nello script per escaping HTML.
- CSS: style/style\_login.css
- Pagine collegate: profilo.php, home\_page.php, password\_dimenticata.php, register.php.

### CAMPI DEL DB UTILIZZATI

Tabella users (campi rilevanti per lo script):

- id\_user (INT, PK)
- username (VARCHAR)
- email (VARCHAR)
- password (VARCHAR) : hash (password\_hash)
- is\_admin (TINYINT / BOOLEAN)
- password\_temp (TINYINT / BOOLEAN) : indica se la password è temporanea
- force\_pw\_change (TINYINT / BOOLEAN) : richiede il cambio password

### WORKFLOW

1. session\_start() : avvia sessione PHP.
2. Inclusione db.php per connessione DB.
3. Generazione (se mancante) del token CSRF: \$\_SESSION['csrf\_token'] = bin2hex(random\_bytes(32)).
4. Se la richiesta è POST:
  - Confronto del token CSRF (hash\_equals) per prevenire CSRF.
  - Lettura e trim di identifier (email o username) e password.
  - Query preparata per cercare l'utente con email = ? OR username = ? (LIMIT 1).
  - Se l'utente esiste: password\_verify(\$password, \$user['password']).
    - Se verifica OK:
      - session\_regenerate\_id(true) per mitigare session fixation.
      - Impostazione variabili di sessione:
        - id\_user, username, role (admin o user), is\_admin, force\_pw\_change.

- Se password\_temp o force\_pw\_change true → redirect a profilo.php (forzare cambio).
  - Altrimenti → redirect a home\_page.php.
- Altrimenti: errore generico credenziali non valide.
- Se GET: mostra il form HTML; eventuale messaggio di errore preso da sessione ed escapato.

## IN DETTAGLIO

- CSRF check:
 

```
if (!hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'] ?? '')) {
    $error = 'Richiesta non valida.';
}
```
- Verifica password e creazione sessione:
 

```
if (password_verify($password, $user['password'])) {
    session_regenerate_id(true);

    $_SESSION['id_user'] = $user['id_user'];

    $_SESSION['username'] = $user['username'];

    $_SESSION['role'] = $user['is_admin'] ? 'admin' : 'user';
}
```
- Escaping output per prevenire XSS:
 

```
function h(string $s): string {
    return htmlspecialchars($s, ENT_QUOTES, 'UTF-8');
}
```

## SCELTE TECNICHE

- Prepared statements (bind\_param): prevengono SQL injection.
- password\_hash / password\_verify: gestione sicura degli hash di password.
- CSRF token + hash\_equals: protezione da CSRF e confronto in tempo costante.
- session\_regenerate\_id(true): mitigazione session fixation dopo il login.
- POST-Redirect-GET: evita il resubmit del form e mantiene messaggi di errore nella sessione.
- Messaggio di errore generico: evita di rivelare se l'email/username esiste (evita enumeration).

## DATI SALVATI IN SESSIONE

- id\_user (INT)

- username (string)
- role (string: 'admin' | 'user')
- is\_admin (int 0/1)
- force\_pw\_change (bool)
- csrf\_token (string) : token CSRF per i form
- login\_error (string) : messaggio d'errore temporaneo

## GESTIONE DEGLI ERRORI

Lo script gestisce gli errori a due livelli:

- Controlli e validazioni applicative (CSRF, campi vuoti, credenziali non valide).
- Comunicazione all'utente tramite una variabile di sessione (\$\_SESSION['login\_error']) che viene mostrata nella pagina di login e poi rimossa.

Questa strategia separa la logica di controllo (server-side) dalla presentazione (HTML) e utilizza il pattern POST-Redirect-GET per evitare il resubmission del form.

## 2. SCRIPT:register.php

### SCOPO

Gestire la registrazione di un nuovo utente: validazione input, verifica unicità (email / username / telefono), hashing della password, inserimento nel DB, creazione della sessione utente e redirect verso la home.

### DIPENDENZE

- db.php : fornisce la connessione \$conn al database (MySQLi).
- Funzione h() definita nello script per escaping HTML.
- CSS: style/style\_login.css (riutilizzato).
- Pagine collegate: login.php, home\_page.php

### CAMPI DEL DB UTILIZZATI

Tabella users :

- id\_user (INT, PK, AUTO\_INCREMENT)
- username (VARCHAR)
- email (VARCHAR)
- password (VARCHAR) : hash con password\_hash()
- indirizzo (VARCHAR)
- telefono (VARCHAR)
- reputazione (INT)

## WORKFLOW

1. `session_start()` : avvia sessione PHP.
2. Inclusione `db.php` per connessione DB.
3. Definizione funzione `h()` per escaping output HTML.
4. Se la richiesta è POST:
  - Lettura e trim dei campi: `username`, `email`, `password`, `indirizzo`, `telefono`.
  - Validazioni server-side:
    - campi obbligatori non vuoti;
    - email valida (`filter_var`);
    - regole password (min 8 caratteri, almeno una maiuscola e un numero).
  - Verifica di unicità (query prepare):
    - a. `SELECT id_user FROM users WHERE email = ?`
    - b. `SELECT id_user FROM users WHERE username = ?`
    - c. `SELECT id_user FROM users WHERE telefono = ?`
  - Se tutte le verifiche passano:
    - `password_hash()` per generare hash della password;
    - `INSERT INTO users (...) VALUES (...)` con prepared statement;
    - se `execute()` OK: `session_regenerate_id(true)` e settare variabili di sessione (`id_user`, `username`); redirect a `home_page.php`.
    - altrimenti mostra errore generico.
5. Se la richiesta è GET: mostra il form HTML con valori (escaped) e messaggi di errore.

## IN DETTAGLIO

- Hash della password:  
`$password_hash = password_hash($password, PASSWORD_DEFAULT);`
- Prepared statement per l'inserimento:  
`$stmt = $conn->prepare("INSERT INTO users (username, email, password, indirizzo, telefono, reputazione) VALUES (?, ?, ?, ?, ?, ?)");`  
`$stmt->bind_param("sssssi", $username, $email, $password_hash, $indirizzo, $telefono, $reputazione);`
- Rigenerazione sessione e set session vars:  
`session_regenerate_id(true);`  
`$_SESSION["id_user"] = $stmt->insert_id;`  
`$_SESSION["username"] = $username;`
- Escaping output:  
`function h(string $s): string {`

```

        return htmlspecialchars($s, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
    }

```

## SCELTE TECNICHE

- Prepared statements per tutte le query di selezione e inserimento: prevengono SQL injection.
- password\_hash() per memorizzare l'hash sicuro della password.
- Validazione lato server per email, forza della password e campi obbligatori.
- Session\_regenerate\_id(true) dopo la creazione dell'account per mitigare session fixation.
- PRG implicito: dopo INSERT redirect verso home\_page.php per evitare double-submit.
- Escaping output (h()) per prevenire XSS in valori mostrati nel form.

## DATI SALVATI IN SESSIONE

- id\_user (INT) : id dell'utente appena creato (da insert\_id).
- username (string): username per personalizzazione e autenticazione lato UI.

## GESTIONE DEGLI ERRORI

- Errori di validazione (campi mancanti, email non valida, password debole, duplicati) sono comunicati attraverso la variabile \$error e mostrati nell'HTML .
- Errore generico in caso di fallimento INSERT per evitare di esporre dettagli tecnici all'utente.

## 3. SCRIPT:Password\_dimenticata.php

### SCOPO

Consentire all'utente di richiedere il recupero della password: verifica dell'email, generazione di una password temporanea (hashata in DB), forzatura del cambio password al prossimo login e aggiornamento degli indicatori password\_temp / force\_pw\_change. Presenta un form per inserire l'email e fornisce feedback all'utente.

### DIPENDENZE

- db.php : fornisce la connessione \$conn al database (MySQLi).
- CSS: style/style\_login.css.
- Pagine collegate: login.php.
- Funzione h() per escaping (nel template viene usato htmlspecialchars inline).

### CAMPI DEL DB UTILIZZATI

Tabella users:

- id\_user (INT, PK)
- email (VARCHAR)
- password (VARCHAR) : hash con password\_hash()

- password\_temp (TINYINT/BOOLEAN): indica che la password corrente è temporanea
- force\_pw\_change (TINYINT/BOOLEAN) : richiede il cambio password al login

## WORKFLOW

1. session\_start() : avvia sessione PHP.
2. Inclusione db.php per connessione DB.
3. Se la richiesta è POST:
  - Legge e valida email (server-side con filter\_var).
  - Query preparata SELECT id\_user FROM users WHERE email = ? LIMIT 1 per verificare esistenza.
  - Se l'email esiste:
    - genera una password temporanea (bin2hex(random\_bytes(...)) nel codice fornito), crea l'hash con password\_hash();
    - aggiorna il record utente con UPDATE users SET password = ?, password\_temp = 1, force\_pw\_change = 1 WHERE id\_user = ? (prepared statement);
    - se l'update va a buon fine, il codice corrente mostra la password temporanea nell'HTML .
  - Se l'email non esiste: imposta messaggio di errore.
4. Se la richiesta è GET: mostra il form per inserire l'email.

## IN DETTAGLIO

- Generazione password temporanea e hash:  
`$temp_password = substr(bin2hex(random_bytes(6)), 0, 10);`  
`$new_hash = password_hash($temp_password, PASSWORD_DEFAULT);`
- UPDATE con prepared statement:  
`$upd = $conn->prepare('UPDATE users SET password = ?, password_temp = 1, force_pw_change = 1 WHERE id_user = ?');`  
`$upd->bind_param('si', $new_hash, $id_user);`  
`$upd->execute();`
- Escaping output (esempio nel markup):  
`htmlspecialchars($temp_password, ENT_QUOTES, 'UTF-8')`

## SCELTE TECNICHE

- Prepared statements per SELECT e UPDATE : prevengono SQL injection.
- password\_hash() per memorizzare la nuova password in modo sicuro.
- Forzatura cambio password tramite i flag password\_temp e force\_pw\_change per obbligare l'utente a impostare una password definitiva al login successivo.
- Validazione server-side dell'email con filter\_var().
- Chiusura esplicita di statement/connessione (\$stmt->close(), \$conn->close()).

## DATI SALVATI IN SESSIONE

Lo script non salva dati sensibili in sessione.

## GESTIONE DEGLI ERRORI

- Errori di validazione (email non valida) mostrati all'utente tramite \$error (escaped).
- Errori DB: nel codice viene impostato un generico 'Errore durante l'aggiornamento.'; raccomandato loggare il dettaglio con `error_log($conn->error)` per diagnostica senza rivelare dettagli all'utente.
- Per evitare enumerazione, usare messaggi identici sia nel caso di successo che di email inesistente.

## 4. SCRIPT:Db.php;Dati\_generali.php

### SCOPO

- `dati_generali.php`: definire in un unico punto i parametri di connessione al database (host, utente, password, nome DB).
- `db.php`: creare una connessione MySQLi riutilizzabile in tutti gli script del progetto, impostare charset, e fornire una funzione di utilità `h()` per l'escaping HTML.

### DIPENDENZE

- `dati_generali.php` (richiesto da `db.php` tramite `require_once`).
- Utilizzato da tutti gli script che accedono al database (`login.php`, `register.php`, `password_dimenticata.php`, `profilo.php`, `home_page.php`, ecc.).

### CAMPI DEL DB UTILIZZATI

Questi file non interagiscono direttamente con tabelle ma forniscono la connessione \$conn.

### WORKFLOW

`dati_generali.php`

- Definizione delle variabili di configurazione:

*\$host : host del DB (es. localhost).*

*\$user: utente del DB.*

*\$password : password DB.*

*\$dbname : nome del database.*

`db.php`

- `require_once __DIR__ . '/dati_generali.php'` → importa configurazione.

- Crea connessione MySQLi:  
`$conn = new mysqli($host, $user, $password, $dbname);`
- Controlla eventuali errori:  
`if ($conn->connect_error) {  
 die("Connessione fallita: " . $conn->connect_error);  
}`
- Imposta charset UTF-8 con utf8mb4 (supporto pieno Unicode, anche emoji).
- Definisce funzione h(\$s) se non già presente, per escaping sicuro HTML.

## IN DETTAGLIO

- Connessione e controllo errori:  
`$conn = new mysqli($host, $user, $password, $dbname);  
if ($conn->connect_error) {  
 die("Connessione fallita: " . $conn->connect_error);  
}`
- Escaping output:  
`function h(string $s): string {  
 return htmlspecialchars($s, ENT_QUOTES|ENT_SUBSTITUTE, 'UTF-8');  
}`

## SCELTE TECNICHE

- Separazione configurazione / logica: i dati sensibili (credenziali DB) sono isolati in `dati_generali.php`, mentre `db.php` gestisce la connessione e funzioni comuni.
- `require_once`: impedisce inclusioni multiple.
- `utf8mb4`: garantisce supporto completo Unicode, evitando problemi di charset.
- Funzione `h()`: riduce duplicazione, consente escaping uniforme contro XSS.
- Condizione `!function_exists('h')`: evita ridefinizione se `db.php` è incluso più volte.

## DATI SALVATI IN SESSIONE

nessuno.

## GESTIONE DEGLI ERRORI

- Errore di connessione: lo script esegue `die("Connessione fallita: ...")`.
- Miglioria raccomandata: loggare errori con `error_log($conn->connect_error)` e mostrare un messaggio generico all'utente per non esporre dettagli sensibili.

## 5. SCRIPT:home\_page.php

### SCOPO

home\_page.php fornisce:

- Controllo della sessione utente e verifica permessi amministratore.
- Verifica dello stato dell'abbonamento leggendo xml/user\_abbonamenti.xml e segnala se l'abbonamento è attivo o sta per scadere.
- Caricamento, filtraggio e visualizzazione delle comunicazioni da xml/comunicazioni.xml.
- Implementazione di un mini-forum basato su XML (xml/forum.xml) con thread, commenti, "mi piace", valutazioni (utilità/accordo), ordinamento per reputazione e moderazione/admin (pin, lock, edit, delete).
- Calcolo e aggiornamento dei punteggi di reputazione in xml/reputazione.xml aggregando:
  - punteggio base salvato nel file XML,
  - contributi da like e valutazioni,
  - decadimento esponenziale per inattività dopo 30 giorni.
- Generazione dell'interfaccia HTML (menu differenziato per utente loggato/non, contenuto centrale, widget comunicazioni, pannello forum).
- Offerta di funzionalità admin: aggiunta comunicazioni, modifica/eliminazione/pinnaggio/blocco di thread e commenti, gestione file XML .

## DIPENDENZE

File e risorse usate dallo script:

- db.php
- XML (cartella xml/):
  - xml/user\_abbonamenti.xml : informazioni sugli abbonamenti utenti.
  - xml/comunicazioni.xml : comunicazioni pubbliche/privato (messaggi).
  - xml/forum.xml : forum (thread, commenti, consensi, valutazioni).
  - xml/reputazione.xml : utenti con punteggi e metadata.
- Backup generati come \*.bak\_YYYYmmdd\_HHis e temp .tmp\_\*
- aggiungi\_comunicazione.php : riceve/salva nuove comunicazioni (solo admin).
- aggiungi\_messaggio\_forum.php : crea thread / commenti.
- mi\_piace.php : registra "mi piace"
- valuta\_commento.php : registra valutazioni commenti
- modifica\_contenuto.php : modifica thread/comment (admin).
- elimina\_contenuto.php : elimina thread/comment (admin).
- toggle\_admin.php : pinnaggio / lock toggle (admin).
- style/style\_home\_page.css : foglio di stile.
- Icone icon/\*.png, immagini img/\*.png.

## CAMPI DEL DB UTILIZZATI

lo script interroga la tabella users per:

- id\_user : identificativo univoco utente (usato in sessione).
- is\_admin : flag amministratore (0/1).
- username : nome utente (string).

L'interrogazione è eseguita con prepared statement:

*SELECT is\_admin, username FROM users WHERE id\_user = ?*

I valori letti servono per stabilire i permessi (admin vs user) e il nome visualizzato/associato ai contributi nel forum.

## DATI SALVATI IN SESSIONE

- \$\_SESSION['id\_user'] : identifica l'utente loggato.  
Usato per:
  - verificare login;
  - controllare visualizzazione messaggi privati;
  - abilitare funzioni forum (creazione commenti/thread, valutazioni, like);
  - raccogliere cronologia contributi personali.
- Parametri di sessione impostati all'avvio:
  - session.gc\_maxlifetime = 3600 (60 minuti).
  - session.cookie\_lifetime = 3600.
  - La pagina imposta header anti-cache (Cache-Control, Pragma) e meta anti-cache nell'HTML per sicurezza.

## WORKFLOW

### 1. Avvio sessione e intestazioni anti-cache

- Impostazione ini\_set('session.gc\_maxlifetime', 3600) e session\_start().
- Header HTTP:
  - a. Cache-Control: no-store, no-cache, must-revalidate, max-age=0
  - b. Cache-Control: post-check=0, pre-check=0 (false)
  - c. Pragma: no-cache
- Meta HTML <meta http-equiv="Cache-Control" ...> e simili per ridurre caching lato browser.

### 2. funzioni

- normalize\_username(?string \$u): ?string
  - trim(), empty → null, mb\_strtolower(..., 'UTF-8') per mapping case-insensitive.
- safe\_float\_from\_node(\$node, \$default = 0.0)
  - estrazione float robusta da nodo XML (gestisce virgole, fallback).
- getReputationClass(\$score)
  - restituisce high-rep / medium-rep / low-rep per visual.

### 3. Verifica login e permessi admin

- `$id_user = $_SESSION['id_user'] ?? null`.
- Se `$id_user` e `db.php` esistono:
  - a. include `db.php`; se `$conn` presente, esegue prepared statement per leggere `is_admin` e `username`.
- `$is_admin = ($flag_admin == 1)`; `$username` usato per mappare contributi.
- `$usernameNormalized = normalize_username($username)` usata per matching case-insensitive.

### 4. Controllo abbonamento

- Se `$id_user` e `xml/user_abbonamenti.xml` esiste:
  - a. Carica con `DOMDocument (libxml_use_internal_errors(true))`.
  - b. Scansiona `<abbonamento_utente>`:
    - i. legge `<id_user>`, `<stato>`, `<data_inizio>`, `<data_scadenza>`.
  - c. Se `id_user` coincide, `stato === 'attivo'` e `today` compreso in `[data_inizio, data_scadenza]` allora:
    - i. `$is_subscribed = true`.
    - ii. `daysLeft = diff(scadenza, oggi)`; se  $\leq 7$  allora `$is_expiring = true`.
  - d. Uso di `DateTimeImmutable` per confronti e calcolo giorni rimanenti.

### 5. Flag UI

- `$showConfirm = isset($_GET['ok']) && $_GET['ok'] === '1'` per mostrare il modal di conferma dopo invio comunicazione.
- `$expandThreadId = $_GET['thread_id'] ?? null` per espandere automaticamente un thread se richiesto via querystring.

### 6. Raccolta statistiche dal forum (likes e valutazioni per autore)

- Se `xml/forum.xml` esiste:
  - a. Carica file e scorre `<thread> → <commento>`:
    - i. per ogni commento raccoglie autore, consensi e valutazioni\_commento (elenco di `<valutazione>` con utilita e accordo).
  - b. Costruisce `$userStats[normalized_username] = ['likes'=>int, 'ratings_count'=>int, 'sum_utilita'=>float, 'sum_accordo'=>float]`.
  - c. Questo permette di avere, per ciascun autore, dati aggregati per il calcolo della reputazione.

### 7. Caricamento reputazioni, calcolo punteggi e salvataggio

- Se `xml/reputazione.xml` esiste:
  - a. Carica DOM, scorre `<utente>`:
    - i. legge `<username>`, `<punteggio>` (punteggio base), `<ultima_attivita>`.
    - ii. Normalizza username per lookup (chiave case-insensitive).
    - iii. Recupera stats da `$userStats` (likes, ratingsCount, avgUtilita, avgAccordo).
    - iv. Calcola contributi:

1.  $\text{likesContribution} = \text{likes} * \text{WEIGHT\_LIKE}$ .
  2.  $\text{ratingScale} = \log_{10}(1 + \text{ratingsCount})$ .
  3.  $\text{utilitaContribution} = \text{avgUtilita} * \text{WEIGHT\_UTILITA} * \text{ratingScale}$ .
  4.  $\text{accordoContribution} = \text{avgAccordo} * \text{WEIGHT\_ACCORDO} * \text{ratingScale}$ .
  - v.  $\text{adjustedBase} = \text{punteggioRaw} + \text{likesContribution} + \text{utilitaContribution} + \text{accordoContribution}$ .
  - vi. Applica decadimento per inattività: se  $\text{daysInactive} > 30 \rightarrow \text{decayDays} = \text{daysInactive} - 30 \rightarrow \text{decayFactor} = 0.99^{\text{decayDays}} \rightarrow \text{currentScore} = \text{adjustedBase} * \text{decayFactor}$ .
  - vii. Clamp finale in  $[\text{MIN\_SCORE}, \text{MAX\_SCORE}]$  e round a 1 decimale.
  - viii. Aggiorna nodo `<punteggio>` nel DOM in modo robusto (crea/rimpiazza text node).
  - b. Effettua backup `reputazione.xml.bak_YYYYmmdd_HHis` (se non esiste già) e salvo atomico su temp file + `rename()` per evitare corruzioni.
  - Mappa risultante:
    - a.  $\text{\$reputationScoresByLower}[\text{normalized\_username}] = \text{score}$ .
    - b.  $\text{\$reputationScoresDisplay}[\text{originalUsername}] = \text{score}$ .
8. Caricamento/rigenerazione comunicazioni.xml
- Creazione di `DOMImplementation` + `DOMDocument` con UTF-8, `formatOutput`.
  - Se `comunicazioni.xml` esiste:
    - a. prova a caricarlo; se fallisce  $\rightarrow$  copia backup `.bak_YYYYmmdd_HHis` e rigenera file vuoto con DTD (se disponibile).
  - Se non esiste: crea struttura `<comunicazioni>` con DTD e salva (crea dir se necessario).
  - Scorre ogni `<messaggio>`:
    - a. legge titolo, contenuto.
    - b. determina tipo: preferisce attributo tipo, poi elemento `<tipo>`. Default 'pubblico' se mancante.
    - c. determina destinatario: attributo o elemento.
    - d. legge `data_invio`: attributo `data_invio` o elemento `<data_invio>`.
    - e. se incluso  $\rightarrow$  aggiunge ad array `\$messaggi`.
  - Ordina `\$messaggi` per `data_invio` decrescente.
9. Caricamento Forum (ordinamento per reputazione, gestione attributi)
- Se `\$is_admin` || `\$is_subscribed`:
    - a. Carica `xml/forum.xml`.
    - b. Per ogni `<thread>`:
      - i. legge id (attributo o md5 del titolo), titolo, autore, `data_creazione`, contenuto.
      - ii. legge attributi opzionali: `pinned`, `locked`, `last_edit_by`, `last_edit_date`.

- iii. calcola threadScore come reputazione dell'autore (lookup in \$reputationScoresByLower).
- iv. raccoglie commenti:
  - 1. ogni commento ottiene id, autore, data, contenuto, consensi, valutazioni\_commento.
  - 2. calcola commentScore come reputazione dell'autore del commento.
  - 3. costruisce struttura commento con consensi (array), valutazioni (array).
  - 4. raccoglie contributi dell'utente corrente (userContributions) se l'username coincide (case-insensitive) con autore/valutatore/consenso.
- v. ordina i commenti per reputazione decrescente.
- vi. aggiunge il thread all'array \$forumThreads.
- c. ordina i thread: pinnati prima, poi per reputazione autore decrescente.
- d. ordina \$userContributions per data decrescente.

#### 10. Output HTML (costruzione interfaccia)

- head:
  - a. <meta charset="UTF-8">, <title>PALESTRW</title>, include style/style\_home\_page.css, meta anti-cache.
- body:
  - a. se ?ok=1 → visualizza modal conf. (#confirm-modal) con gestione JS che rimuove ?ok=1 dall'URL.
  - b. header principale con logo e menu (menu differente per !\$id\_user e per \$id\_user).
  - c. bottone hamburger che apre / chiude #forum-panel.
  - d. #forum-panel:
    - i. se non abbonato e non admin → messaggio che segnala che serve abbonamento.
    - ii. se non ci sono thread → invito a iniziare discussione.
    - iii. altrimenti lista thread con:
      - 1. header: titolo, badge reputazione, icone pinned/locked, toggle.
      - 2. contenuto: testo, autore, data, nota modifica admin se presente.
      - 3. admin controls (edit/delete/pin/lock) se \$is\_admin.
      - 4. lista commenti (hidden se non espanso), each comment include like, valutazioni, badge reputazione, note admin.
      - 5. se \$is\_subscribed e thread non locked → form per nuovo commento.
    - iv. area per creare nuova discussione se \$is\_subscribed.

- v. pannello "La mia cronologia" con \$userContributions (hidden per default).
  - e. #main-content con welcome e tabella immagini quadranti.
  - f. msg-widget con lista \$messaggi (o testo "Nessuna comunicazione disponibile"); bottone  per admin apre modal #comunicazione-modal.
  - g. #comunicazione-modal (solo per admin): form con titolo (maxlength 200) e contenuto → POST su aggiungi\_comunicazione.php.
  - h. footer con anno dinamico <?= date('Y') ?>.
  - Tutte le uscite dinamiche che possono contenere input utente sono stampate con htmlspecialchars(..., ENT\_QUOTES) e nl2br() dove opportuno per prevenire XSS.
11. JavaScript client-side principale
- Chiusura modal conferma ?ok=1 e rimozione parametro dall'URL .
  - Toggle #forum-panel con bottone hamburger.
  - Toggle "La mia cronologia".
  - Toggle espansione thread (click su .thread-header mostra/nasconde comments e aggiorna icona ▲/▼).
  - Disabilitazione pulsante submit dopo invio dei form (testo "Invio in corso...") per evitare doppio invio.
  - Like handling:
    - a. click .like-btn → POST JSON via fetch('mi\_piace.php') con { thread\_id, comment\_id, username }.
    - b. aggiorna conteggio e lista utenti che hanno messo like su risposta positiva.
    - c. in caso di errore ripristina stato bottone.
  - Valutazione commenti:
    - a. click .rate-btn → apre modal dinamico con select per utilita e accordo (1–5).
    - b. client-side verifica che data-days ≤ 30 (block se scaduta).
    - c. submit della modal invia JSON a valuta\_commento.php, su success reload.
  - Admin handlers:
    - a. edit: apre modal con textarea precompilata; submit invia JSON a modifica\_contenuto.php.
    - b. delete: conferma con confirm() e invia JSON a elimina\_contenuto.php.
    - c. pin/lock: invia JSON a toggle\_admin.php e ricarica su successo.
  - Apertura/chiusura modal Nuova comunicazione .
  - Handling di ?thread\_id=... per espandere e mettere in evidenza il thread alla caricamento.

## 12. Endpoint richiesti lato server

- mi\_piace.php — riceve POST JSON, valida sessione, aggiorna xml/forum.xml (aggiunge nome in <consensi> o aggiorna formato legacy), salvaguardia race con LOCK\_EX, ritorna { success, liked\_users[], message }.
- valuta\_commento.php — valida sessione, verifica finestra 30 giorni lato server, scrive valutazione in xml/forum.xml sotto valutazioni\_commento.

- `aggiungi_messaggio_forum.php` — crea nuovo thread o commento, valida permessi (`$is_subscribed` o `admin`).
- `aggiungi_comunicazione.php` — verifica `admin`, sanitizza e aggiunge `<messaggio>` in `comunicazioni.xml`, salva atomico, reindirizza `?ok=1`.
- `modifica_contenuto.php`, `elimina_contenuto.php`, `toggle_admin.php` — tutte devono validare permessi `admin` e rispondere JSON.

## SCELTE TECNICHE

- Sessione con durata estesa: 3600 secondi per bilanciare usabilità vs sicurezza (in produzione valutare rolling session).
- Anti-cache: header HTTP (Cache-Control, Pragma) + meta HTML per minimizzare esposizione di dati sensibili su cache client/intermediari.
- Storage su XML (file-based):
- Backup e salvataggio atomico:
  - Creazione di backup `.bak_YYYYmmdd_HH` prima di ricreare file corrotti.
  - Scrittura su file `.tmp_<uniqid>` + `rename()` per atomicità.
  - Uso di `LOCK_EX` con `file_put_contents()` per evitare race condition.
- Algoritmo reputazione ibrido:
  - Parte sempre dal punteggio base salvato nel file XML per non azzerare utenti quando dati esterni sono mancanti.
  - Contributi:
    - likes con peso `WEIGHT_LIKE` (configurabile).
    - valutazioni utilità e accordo con pesi `WEIGHT_UTILITA` e `WEIGHT_ACCORDO`
    - scala logaritmica  $\log_{10}(1 + \text{ratingsCount})$  per attenuare impatto di count elevati.
  - Decadimento per inattività: 1% al giorno dopo 30 giorni (esponenziale):  $0.99^{\text{decayDays}}$ .
  - Clamp dei punteggi in `[MIN_SCORE, MAX_SCORE]`.
- Normalizzazione username:
  - `mb_strtolower(..., 'UTF-8')` per matching case-insensitive e supporto caratteri multi-byte.
- Interfaccia reattiva client-side:
  - Uso di `fetch()` JSON per operazioni asincrone (like, valutazione, operazioni admin).
  - Disabilitazione pulsanti di submit per prevenire invii multipli.

## GESTIONE ERRORI

- `libxml_internal_errors`:
  - `libxml_use_internal_errors(true)` prima di caricare XML per evitare warning visibili in HTML.

- libxml\_clear\_errors() dopo le operazioni.
- Backup e salvataggio atomico:
  - Prima di sovrascrivere reputazione.xml si copia il file corrente su .bak\_... se non già presente con quel timestamp.
  - Scrittura su file temporaneo + rename() per ridurre rischio di file corrotto.
  - try/catch su operazioni di scrittura; in caso di eccezione non interrompere la pagina .
- Validazioni :
  - safe\_float\_from\_node() per prevenire cast su valori non numerici.
  - Validazioni su data (try/catch su new DateTimeImmutable) e fallback a valori base se la data è malformata.
  - Se non ci sono messaggi → visuale “Nessuna comunicazione disponibile”.
- Protezione lato server per endpoint:
  - Gli endpoint (mi\_piace.php, valuta\_commento.php, modifica\_contenuto.php, ecc.) devono:
    - verificare sessione e permessi (admin o abbonato quando richiesto);
    - applicare controllo finestra temporale (30 giorni) lato server per le valutazioni;
    - eseguire locking su scritture XML per evitare race condition.
- Gestione errori client-side:
  - fetch().then(...).catch(...) per mostrare alert in caso di errore e ripristinare stato UI (es. abilitare di nuovo il bottone like).

## 6. SCRIPT:Promo.php

### SCOPO

- Gestisce le promozioni del sito PALESTRW (lettura, visualizzazione, aggiunta ed eliminazione).
- Controlla la sessione utente e i permessi admin (solo gli admin possono aggiungere/eliminare promozioni).
- Legge e valida file XML/DTD per le promozioni e per gli abbonamenti.
- Aggiorna automaticamente il flag <attiva> delle promozioni in base alle date.
- Fornisce l'interfaccia HTML per mostrare le promozioni e, per gli admin, un form per aggiungerne di nuove.

### DIPENDENZE

- db.php
- File XML / DTD:
  - xml/abbonamenti.xml + dtd/abbonamenti.dtd : lista degli abbonamenti validi.

- xml/promo.xml + dtd/promo.dtd : elenco delle promozioni (creato/letto/salvato dallo script).
- CSS: style/style\_promo.css
- Icone / immagini nella cartella icon/

## CAMPI DEL DB UTILIZZATI

Tabella users (usata solo per verificare il ruolo admin):

- id\_user → identificatore utente (intero).
- is\_admin → flag amministratore (0/1).

## DATI SALVATI IN SESSIONE:

`$_SESSION['id_user']` → usato per identificare l'utente loggato; se presente tenta di recuperare is\_admin dal DB.

## WORKFLOW

1. Avvio e contesto
  - `session_start()` :avvia la sessione.
  - `$today = DateTimeImmutable('now', Europe/Rome)` : data corrente centralizzata usata ovunque.
2. Verifica login e permessi admin
  - Legge `$_SESSION['id_user']`.
  - Se presente include db.php e, se `$conn` è disponibile:
    - Esegue query preparata per leggere is\_admin da users WHERE id\_user = ?.
    - Imposta `$is_admin = true` se `is_admin == 1`.
3. Costanti percorsi file
  - `PROMO_XML`, `PROMO_DTD`, `ABB_XML`, `ABB_DTD` definiti con `__DIR__` per riferimenti assoluti.
4. Funzioni principali
  - `validateXmlAgainstDtd(string $path, string $dtdPath): bool`
    - Legge XML e DTD (con `file_get_contents`).
    - Controlla se il DTD dichiara l'attributo id di tipo ID. Se sì, in memoria sostituisce solo gli attributi `id="123"` con `id="n123"` perché in DTD ID richiede token name (non solo cifre).
    - Carica la stringa in `DOMDocument` e valida con DTD (usando `LIBXML_DTDVALID`).
    - Restituisce true se la validazione passa, false altrimenti; logga gli errori con `error_log`.
  - `loadXmlWithDtd(string $path, string $dtdPath, string $rootName): DOMDocument`

- Crea un DOMDocument con impostazioni base.
- Se il file \$path non esiste, crea un documento vuoto con DOCTYPE che punta al DTD (file://realpath(dtd) o path relativo), scrive sul disco (\$newDoc->save(\$path)).
- Carica il documento esistente (\$doc->load(\$path)), se fallisce lancia RuntimeException.
- Se manca il doctype costruisce in memoria un documento con doctype che punta al DTD(non salva).
- Chiama validateXmlAgainstDtd(\$path, \$dtdPath). Se la validazione fallisce e il documento NON è vuoto (cioè contiene più del solo root), lancia RuntimeException; se è documento appena creato e vuoto ignora gli errori.
- Restituisce il DOMDocument pronto per essere letto/manipolato.
- saveXml(DOMDocument \$doc, string \$path): void
  - Salva \$doc->save(\$path) ,persiste sul disco.
- getAbbonamenti(): array
  - Chiama loadXmlWithDtd(ABB\_XML, ABB\_DTD, 'abbonamenti').
  - Costruisce e ritorna un array associativo \$list[id] = tipo iterando sui nodi <abbonamento> (usa l'attributo id e il nodo <tipo>).
- 5. Gestione aggiunta promozione (POST action=add) :solo admin
  - Carica abbonamenti validi con getAbbonamenti() per validare la selezione.
  - Verifica che \$\_POST['abbonamenti'] sia array e contenga almeno 1 id valido; filtra solo gli id presenti in \$abbonamenti\_validi.
  - Apre promo.xml con loadXmlWithDtd.
  - Calcola nuovo id incrementale (cerca il max id esistente tra i nodi <promo>).
  - Crea nodo <promo id="newId"> e aggiunge campi obbligatori:
    - titolo, descrizione, data\_inizio, data\_fine, codice\_sconto
  - Aggiunge <attiva>0</attiva> di default.
  - Campi opzionali:
    - percentuale → formattata con number\_format(..., 2, ',', '.').
    - applica\_a → valore testuale.
  - Aggiunge nodo <abbonamenti> con sotto-nodi <abbonamento id="X">.
  - Appende il <promo> al root e salva con saveXml.
  - Redirect a promo.php (header Location) e exit.
- 6. Gestione cancellazione promo (GET?delete=ID) : solo admin
  - Se \$\_GET['delete'] è presente e utente admin:
    - Carica promo.xml con loadXmlWithDtd.
    - Usa DOMXPath per cercare //promo[@id='ID'] ed elimina i nodi trovati (parentNode->removeChild).
    - Salva e redirect a promo.php.
- 7. Aggiornamento flag attiva in base alle date

- Carica promo.xml.
  - Per ogni nodo <promo>:
    - Legge data\_inizio, data\_fine, e il nodo attiva (crea attiva se mancante).
    - Converte data\_inizio a inizio giornata 00:00:00 e data\_fine a 23:59:59 (timezone Europe/Rome).
    - Se \$today è tra start e end, setta attiva a '1' altrimenti '0'.
    - Se il valore cambia, setta \$mod = true.
  - Se \$mod true → salva il documento.
8. Preparazione array promozioni per rendering
- Scorre tutti i <promo> e:
    - Ignora quelli senza data\_fine.
    - Converte data\_fine a DateTimeImmutable(...)->setTime(23,59,59) e scarta le promozioni scadute (end < \$today).
    - Estrae campi (id, titolo, descrizione, date, codice\_sconto, percentuale, applica\_a, array di abbonamenti).
    - Aggiunge ogni promo valida a \$activePromos[] includendo anche endDate (oggetto DateTimeImmutable) per ordinamento.
  - Ordina \$activePromos per endDate crescente (prima scadono prima mostrate).
9. Rendering HTML
- Header con charset, title e css.
  - Menu differenziato in base a \$id\_user (non loggato vs loggato).
  - Corpo:
    - Ciclo su \$activePromos e output dei relativi blocchi promozione (escape con htmlspecialchars e nl2br per la descrizione).
    - Se \$is\_admin mostra il pulsante/tile per aggiungere una promo e link di eliminazione per ciascuna promo (con onclick="return confirm('Eliminare?')").
  - Form aggiungi-promozione (nascosto di default) presente solo per admin; include tutti i campi sopra descritti e la lista di abbonamenti (checkbox) ottenuta da getAbbonamenti().
  - Footer.
10. JavaScript lato client
- Toggle di visualizzazione del form di aggiunta promo (apertura/chiusura, focus).
  - Validazione client: impedisce submit se non è selezionato almeno 1 abbonamento.
  - Comportamento accessibilità: pulsante add risponde anche a Enter/Space.

## SCELTE TECNICHE

- XML + DTD: scelta di validare XML localmente con DTD. loadXmlWithDtd crea file vuoti se mancanti e salva il file creato.

- ID come interi: gli id delle promo e degli abbonamenti sono trattati come interi nel file XML, ma per la validazione contro DTD (che dichiara id ID) viene applicato in memoria un prefisso n per trasformare 123 in n123, la trasformazione è solo per la validazione in memoria e non modifica i file su disco.
- Sicurezza DB: la query per is\_admin usa prepared statement .
- Sanificazione input: htmlspecialchars(..., ENT\_XML1, 'UTF-8') viene usato quando si scrive nel DOM per evitare caratteri XML non validi; percentuale viene cast e formattata.
- Timezone: tutte le date sono parse/gestite con DateTimeImmutable(..., new DateTimeZone('Europe/Rome')) .

## **GESTIONE ERRORI**

- libxml\_use\_internal\_errors(true) usato per evitare warning diretti e catturare errori tramite libxml\_get\_errors().
- Errori di caricamento XML generano RuntimeException in loadXmlWithDtd (eccetto quando il documento è vuoto appena creato).
- validateXmlAgainstDtd logga gli errori con error\_log e ritorna false.
- In aggiunta promo/elimina: se la validazione degli abbonamenti fallisce lo script termina con die("Errore: ...") .

## **7. SCRIPT:Corsi.php**

### **SCOPO**

Questo file gestisce la pagina Corsi / Lezioni.

Obiettivi principali:

- caricare dati da diversi file XML (corsi base, lezioni, abbonamenti utenti, feedback);
- determinare quali lezioni sono future e quali passate;
- verificare se l'utente ha un abbonamento attivo (per poter prenotare);
- raccogliere le prenotazioni dell'utente e mostrare le sue prenotazioni future / lezioni passate per lasciare feedback;
- mostrare funzionalità di gestione (aggiungi corso, aggiungi lezione, modifica, elimina) se l'utente è admin;
- offrire API client-side (fetch) per prenotare/eliminare prenotazioni e per salvare feedback.

### **DIPENDENZE**

- db.php
- File XML:

- xml/user\_abbonamenti.xml — abbonamenti utenti (per verificare se l'utente ha un abbonamento attivo).
- xml/corsi\_base.xml — definizione dei corsi base (nome, descrizione, difficoltà).
- xml/lezioni.xml — elenco lezioni, con info su datetime, prenotanti, posti.
- xml/feedback.xml — valutazioni delle lezioni.
- CSS: style/style\_corsi.css.
- Font Awesome (per icone client-side, incluso via CDN).
- Endpoint server-side esterni (usati dal JS):
  - prenota\_lezione.php, elimina\_prenotazione.php, save\_feedback.php, aggiungi\_corso\_base.php, aggiungi\_lezione.php, modifica\_lezione.php, cancella\_lezione.php, modifica\_corso\_base.php, cancella\_corso\_base.php.

## CAMPI DEL DB UTILIZZATI

Tabella users (solo per controllare il ruolo):

- id\_user → identificatore dell'utente.
- is\_admin → flag amministratore (0/1).

Dati utente attesi in sessione: \$\_SESSION['id\_user'].

## WORKFLOW

### 1. Avvio

- session\_start() avvia la sessione.
- date\_default\_timezone\_set('Europe/Rome') imposta la timezone: tutte le date vengono trattate coerentemente come Europe/Rome.
- Si legge \$\_SESSION['id\_user'] (cast a int) e si inizializzano le variabili: \$is\_admin, \$has\_active\_abbonamento, \$id\_user\_abbonamento\_attivo.

### 2. Verifica ruolo admin (se presente id\_user)

- Se esiste db.php e la connessione \$conn, lo script esegue una prepared statement per leggere is\_admin da users WHERE id\_user = ?.
- Se la prepare fallisce, logga l'errore (error\_log) ma non interrompe lo script (modalità read-only supportata).
- Se l'utente è admin, viene impostato \$is\_admin = true e \$has\_active\_abbonamento viene forzato a false (gli admin non vengono considerati con abbonamento attivo per funzioni utente).

## FUNZIONI PRINCIPALI

- `getXML(string $path): ?SimpleXMLElement`

Carica un file XML con `simplexml_load_file`, gestisce gli errori di parsing tramite `libxml_use_internal_errors(true)` e ritorna null in caso di problemi. Log degli errori su `error_log`.

- `validateXMLWithDTD(string $path): bool`

Prova a validare il file XML in base al DOCTYPE/D\td dichiarato nel file stesso. Usa `DOMDocument::validate()`.

- `getAttr(SimpleXMLElement $node, string $attrName, $default = "")`

Helper per leggere attributi in modo sicuro da SimpleXML, evitando avvisi se l'attributo non esiste.

- `parseXmlDate(string $dateStr)`

Funzione per convertire stringhe di data in oggetti `DateTime`. Prova alcuni formati comuni (Y-m-d H:i:s, Y-m-d\TH:i, Y-m-d, ecc.) e come fallback usa `strtotime()`. Ritorna `DateTime` o `false`.

- `h($s)`

Escape HTML comodo (`htmlspecialchars` con `ENT_QUOTES | ENT_SUBSTITUTE`, UTF-8) usato in output per prevenire XSS visuale.

### 3. Verifica abbonamento attivo (solo utenti non-admin)

- Se `id_user` presente e utente non admin:
  - carica `xml/user_abbonamenti.xml` tramite `getXML`.
  - opzionalmente chiama `validateXMLWithDTD` (non bloccante).
  - itera `abbonamento_utente` e per ogni entry:
    - legge id (attributo), `id_user`, `stato`, `data_inizio`, `data_scadenza`.
    - converte le date con `parseXmlDate`.
    - se le date sono valide e l'`id_user` corrisponde alla sessione e `stato === 'attivo'` e la data odierna è compresa tra inizio e scadenza → imposta `$has_active_abbonamento = true` e memorizza `$id_user_abbonamento_attivo`.
  - Se date non valide logga e salta la voce.

### 4. Caricamento `corsi_base.xml`

- Carica e valida `corsi_base.xml`.
- Costruisce la mappa `$corsiBaseMap[id] = ['id','nome','descrizione','difficolta']` dopo controlli minimi su id (logga e salta se mancante o non valido).

## 5. Caricamento e processamento lezioni.xml

- Carica e valida lezioni.xml.
- Per ogni <lezione>:
  - verifica attributo id valido.
  - legge id\_corso\_base e recupera il corso base corrispondente da \$corsiBaseMap. Se manca il corso base, la lezione viene ignorata (log).
  - legge e parse la data datetime\_lezione con parseXmlDate. Se non valida, skip e log.
  - costruisce due strutture principali:
    - \$allLezioni[\$id] → storico completo (tutte le lezioni).
    - \$corsiMap[\$nome]['lezioni'] → raggruppamento per nome corso.
  - se la lezione è futura (timestamp >= now), la aggiunge anche a \$lezioni con conteggio prenotati e lista prenotanti (filtra i prenotanti con stato != 'cancellato').
  - per ogni prenotante viene preso id, user, id\_abb (abbonamento usato).

## 6. Prenotazioni utente (se ha abbonamento attivo)

- Se \$id\_user && \$has\_active\_abbonamento && \$lezioniXML:
  - scorre tutte le lezioni e i relativi prenotanti.
  - se trova un prenotante il cui id\_abb corrisponde a \$id\_user\_abbonamento\_attivo, aggiunge la prenotazione alle strutture:
    - \$prenotazioni\_per\_lezione[\$lid] = true (per indicare che l'utente è prenotato per quella lezione);
    - \$prenotazioni\_utente[] con dettagli (id prenotazione, id lezione, nome corso, datetime, id abbonamento).
  - Questo serve per marcare "Già prenotato" e per costruire la lista delle prenotazioni future/passerate di quell'utente.

## 7. Feedback e statistiche

- Carica feedback.xml.
- Per ogni valutazione\_corso:
  - prende id\_lezione e voto.
  - se la lezione è conosciuta in \$allLezioni aggiorna le statistiche per quel corso (cnt, sum, array voti).
  - se la valutazione appartiene all'abbonamento attivo dell'utente, popola \$feedbacks[\$lid] = \$v (in modo che l'utente veda la sua valutazione).

- Calcola media (avg) per corso: round(sum/cnt, 2).

#### 8. Dati di amministrazione per corsi base

- Se \$is\_admin costruisce \$adminCorsiBase contenente per ogni corso base:

id, nome, descrizione, difficoltà, numero lezioni, numero feedback, media feedback.

- Questa tabella viene mostrata nella sezione di gestione admin.

#### 9. Output HTML (UI)

- Header e menu (differenziato per utente/guest).
- Se admin: pulsanti per aggiungere corso base / lezione e i relativi form (toggle via JS).
- Se non ci sono lezioni future: messaggio “Nessuna lezione in programma”.
- Tabella lezioni future:
  - mostra info, posti disponibili, pulsante prenota (solo per utenti con abbonamento attivo che non siano admin).
  - admin vede pulsanti per modificare/eliminare; può espandere lista prenotanti.
- Se utente non admin e ha abbonamento attivo:
  - se ha prenotazioni future mostrate in tabella con possibilità di cancellare (se manca < 2 ore dalla lezione, il pulsante è disabilitato).
  - se ha lezioni passate cui ha partecipato può lasciare valutazione (stelle interattive).
- Se admin: sezione “Gestione Corsi Base” con stats e possibilità di edit/delete inline.

#### 10. JavaScript client

- prenotaLezione(id) → POST a prenota\_lezione.php, aspetta JSON, notifica e ricarica pagina.
- eliminaPrenotazione(id, id\_lezione) → POST a elimina\_prenotazione.php.
- Rating stars: listeners su .star-rating che POSTano id\_lezione e voto a save\_feedback.php e aggiornano l’interfaccia (riempiono stelle).
- Toggle di elementi (form inline, liste prenotanti) con funzioni helper.
- I form admin vengono mostrati/nascosti con funzioni toggle.

### SCELTE TECNICHE

- SimpleXML vs DOM: lo script usa SimpleXML per semplicità di lettura; la validazione DTD viene fatta opzionalmente con DOMDocument::validate() ma non è requisito bloccante.
- Modalità read-only: se db.php o connessione non sono presenti lo script continua a funzionare (lettura XML).
- parseXmlDate tenta più formati: questo permette flessibilità nel formato delle date presenti negli XML ma può nascondere incoerenze se i dati non sono standardizzati.
- Admin vs utente normale: gli admin non possono prenotare (codice impone \$is\_admin → admin non prenota).
- Uso di timestamp: molte operazioni si basano su timestamp e confronto con time()/DateTime per decidere future/past e abilitare/disabilitare bottoni.
- Sicurezza su output: viene usata la funzione h() per escapare testo verso HTML, buona pratica per prevenire XSS.
- Log: lo script logga molti casi anomali con error\_log() (date invalidi, id mancanti, DB prepare fallita) (utile per debug).

## GESTIONE DEGLI ERRORI

- Lo script preferisce loggare con error\_log() e continuare. (però può nascondere errori critici all'utente)
- La validazione DTD è non bloccante: se non raggiungibile il DTD si procede comunque.

## 8. SCRIPT:Chi\_siamo.php

### SCOPO

Pagina informativa statica che mostra le informazioni aziendali (titolo "CHI SIAMO", indirizzo, contatto, mappa cliccabile) e fornisce l'header/menu adattato allo stato della sessione (guest / utente loggato). Non gestisce FAQ né funzioni di amministrazione.

### DIPENDENZE

- db.php : incluso solo se esiste \$\_SESSION['id\_user'] (usato qui esclusivamente per decidere quale menu mostrare).
- CSS: style/style\_Chi\_siamo.css.
- Icone / immagini locali (mappa, icone menu).

### WORKFLOW

1. session\_start() e recupero \$\_SESSION['id\_user'] in \$id\_user.
2. Se \$id\_user truthy: include db.php; se \$conn è disponibile esegue prepared statement SELECT is\_admin FROM users WHERE id\_user = ? e legge is\_admin come nel file FAQ;

3. imposta `$is_admin`. In questo file `$is_admin` non viene usato per mostrare controlli amministrativi, ma solo potenzialmente per la logica del menu.
4. Genera HTML statico:
  - header/menu con voci diverse se utente loggato o meno;
  - sezione principale con titolo, elenco info (indirizzo, contatto) e immagine mappa che apre Google Maps in nuova scheda (`target="_blank" rel="noopener noreferrer"`).
5. Footer con `<?= date('Y') ?>`.

### **SCELTE TECNICHE**

- Minimalismo e separazione: la pagina è mantenuta statica per rendere il caricamento leggero e la manutenzione semplice.
- Inclusione condizionale di `db.php` solo se necessario (ma nota: il codice attuale include comunque il DB se l'utente è loggato — vedi nota sotto sulle ottimizzazioni).
- Attributi di sicurezza per link esterni: `rel="noopener noreferrer"` per l'apertura di Google Maps.
- Uso di risorse locali (immagine mappa) per velocità e controllo grafico.

### **GESTIONE DEGLI ERRORI**

- Se `db.php` manca o `$conn` non è definita, il codice continua senza mostrare errori (nessuna sezione admin presente in questa pagina).
- Non c'è parsing XML o scrittura file in questo script quindi il rischio principale è l'assenza di risorse (CSS/immagini) o permessi di file; tali errori non vengono gestiti esplicitamente.
- Possibili miglioramenti: evitare la query DB se non si usa `is_admin` qui (riduce chiamate DB), aggiungere fallback visivo se l'immagine mappa non è disponibile, o loggare errori di include DB per debug.

## **11.     SCRIPT: `faq.php`**

### **SCOPO**

Gestire la pagina FAQ: lettura/creazione del file XML `xml/faq.xml`, preparazione dell'array `$faqs` per l'output HTML, visualizzazione pubblica delle FAQ e pannello di amministrazione (aggiungi / modifica / elimina) accessibile solo ad admin.

### **DIPENDENZE**

- `db.php`: incluso solo se esiste `$_SESSION['id_user']` e se la variabile `$conn` è disponibile dopo l'`include`.
- File XML: `xml/faq.xml` (viene creato se non esiste).
- CSS: `style/style_faq.css`.

- Icone / immagini locali (menu, bottoni, mappa se presenti).
- Endpoint server-side esterni usati dai form admin: aggiungi\_faq.php, modifica\_faq.php, elimina\_faq.php.

## WORKFLOW

1. session\_start() e recupero \$\_SESSION['id\_user'] in \$id\_user (null se assente).
2. Se \$id\_user è truthy: include db.php; se dopo l'include esiste \$conn viene eseguito un prepared statement:
  - SELECT is\_admin FROM users WHERE id\_user = ?
  - bind\_param("i", \$id\_user), execute(), bind\_result(\$flag\_admin), fetch(), close()
  - imposta \$is\_admin = (\$flag\_admin == 1); chiude la connessione \$conn->close().
  - Se db.php manca o \$conn non è definita, il file continua in modalità read-only (menu + visualizzazione FAQ).
3. Imposta \$xmlFile = 'xml/faq.xml'.
  - Se il file non esiste crea la cartella xml/ se necessario (mkdir(..., 0755, true)) e salva un SimpleXMLElement('<?xml...?><faqs></faqs>') su disco con asXML(\$xmlFile).
4. Se il file esiste e filesize(\$xmlFile) > 0:
  - abilita libxml\_use\_internal\_errors(true), carica con simplexml\_load\_file(\$xmlFile), poi libxml\_use\_internal\_errors(false).
  - se \$xml valido e contiene <faq> itera ogni nodo e popola \$faqs[] con chiavi stringhe: id, question, answer.
5. Genera l'HTML:
  - header/menu differenziato per guest/utente loggato;
  - elenco FAQ: prime 3 visibili, le successive hanno classe hidden e si possono mostrare con #toggle-faqs;
  - se \$is\_admin true: mostra pannello admin con form per aggiunta, lista FAQ con bottoni Modifica (apre modal) ed Elimina (form con confirm JS).
6. JavaScript client-side: toggle risposte, toggle per mostrare/nascondere FAQ oltre le prime 3, openEditModal() / closeEditModal() per popolamento modal, chiusura modal cliccando fuori.

## SCELTE TECNICHE

- SimpleXML per lettura/scrittura XML: semplice e diretto per questo caso d'uso.
- Prepared statements per la query is\_admin (uso di bind\_param("i", \$id\_user)).
- Controllo isset(\$conn) dopo l'include di db.php: permette fallback in modalità read-only se la connessione DB non è disponibile.
- Creazione automatica della cartella xml/ (mkdir(..., true)) prima di salvare il file XML per evitare errori di scrittura.
- Escaping output: htmlspecialchars() usato per question, answer, message e htmlspecialchars(..., ENT\_QUOTES) per i valori passati nel onclick della Modifica (riduce problemi con apici).

- Separazione responsabilità: le azioni mutanti (aggiungi/modifica/elimina) sono delegate a script separati (aggiungi\_faq.php, modifica\_faq.php, elimina\_faq.php).

## GESTIONE DEGLI ERRORI

- Se xml/faq.xml non esiste viene creato automaticamente.
- Parsing XML: libxml\_use\_internal\_errors(true) sopprime warning di libxml durante il parsing; il codice non effettua logging automatico degli errori di parsing.
- DB: il codice verifica l'esistenza di \$conn prima dell'uso; non ci sono try/catch o log dedicati per errori DB.
- File system: verifica e creazione della directory xml/ riducono errori di file non trovati; permessi di scrittura non sono verificati esplicitamente (possibile failure se permessi insufficienti).
- Sicurezza: output escapato per mitigare XSS; tuttavia i form admin non includono CSRF token nativamente (potenziale rischio).
- Possibili miglioramenti non implementati: logging degli errori libxml/DB, validazione robusta degli input prima della scrittura XML, casting esplicito intval() su \$\_SESSION['id\_user'] prima di bind\_param, rate-limit o controllo accessi più strict per endpoint admin.

## 10. [SCRIPT: Profilo.php;Mark\\_messages\\_read.php](#)

### SCOPO

#### profilo.php

Questo gestisce la pagina profilo utente (UI + logica server) e diverse funzionalità collegate:

- Mostra i dati personali dell'utente (username, email, indirizzo, telefono, reputazione).
- Visualizza lo stato dell'abbonamento dell'utente (attivo / in\_scadenza / scaduto).
- Permette al singolo utente di:
  - cambia la password (con validazione lato server e client);
  - aggiorna i dati del profilo;
  - usa la chat con lo staff (se ha abbonamento attivo).
- Fornisce funzionalità amministrative (se is\_admin):
  - lista abbonamenti di tutti gli utenti;
  - modulo per creare un nuovo utente + associare un abbonamento (salvataggio su XML).

- Interagisce con file XML: xml/abbonamenti.xml, xml/user\_abbonamenti.xml, xml/reputazione.xml, xml/private\_chat.xml.
- Funzionalità client-side: modals, validazione password in tempo reale, polling per nuovi messaggi, notifiche desktop, comportamento della chat (apri/chiudi, scroll).
- Statistiche avanzate (admin):
  - Medie valutazioni corsi (da feedback XML)
  - Reputazione utenti con abbonamento attivo
  - Mappa termica forum (top 5 thread)
  - Utilizzo promozioni
  - Indice di soddisfazione (algoritmo personalizzato)

Obiettivi principali raggiunti nel codice:

- caricamento/creazione file XML con helper getXML();
- uso di prepared statements per operazioni sensibili su DB;
- calcolo stato dell'abbonamento centralizzato in getSubscriptionStatus();
- creazione atomica e robusta di nodi in user\_abbonamenti.xml (generazione id tipo aN);
- UI differenziata per admin / utente.

mark\_messages\_read.php

Questo file espone un endpoint JSON che marca come letti i messaggi nella chat privata (XML) secondo il contesto:

- Se chiamato da admin con ?user\_id=... marca i messaggi inviati dall'utente target verso l'admin corrente.
- Se chiamato da utente normale marca tutti i messaggi inviati da qualsiasi admin verso l'utente.
- Supporta mittente/destinatario sia come id numerico che come username (tentativo secondario).
- Salvataggio atomico del file XML (scrittura su .tmp e rename).

Obiettivi principali:

- Permettere al frontend (profilo.php) di marcare messaggi come letti via fetch() e aggiornare badge in UI.
- Restituire JSON con il numero di messaggi modificati ({ ok: bool, changed: int }).

## DIPENDENZE

- db.php - Connessione DB
- invio\_mess\_utente.php - Gestione chat
- mark\_messages\_read.php - Endpoint messaggi
- check\_new\_messages.php - Polling notifiche
- abbonamenti.xml - Catalogo abbonamenti
- user\_abbonamenti.xml - Storico abbonamenti
- reputazione.xml - Punteggi utenti
- private\_chat.xml - Messaggi chat
- corsi\_base.xml/feedback.xml/lezioni.xml - Dati statistiche
- forum.xml/promo.xml - Dati supplementari
- icone
- CSS locale (style\_profilo.css)

## CAMPI DEL DB UTILIZZATI

Tabella users :

- id\_user : identificatore utente (sessione).
- username
- password : hash (password\_verify / password\_hash).
- email
- is\_admin :Flag (0/1).
- indirizzo
- telefono
- password\_temp (usato per resettare / flag)
- force\_pw\_change : flag che forza apertura modal cambio password.

Sessione:

- \$\_SESSION['id\_user'], \$\_SESSION['force\_pw\_change'],
- \$\_SESSION['cp\_error'], \$\_SESSION['cp\_success'],
- \$\_SESSION['add\_error'], \$\_SESSION['add\_success'].

## WORKFLOW

profilo.php

1.session\_start(); recupera dati di sessione (id\_user, flags messaggi, ecc.).

2.Se utente non loggato (!isset(\$\_SESSION['id\_user'])) redirect a login.php.

3.Include db.php e controlla \$conn->connect\_error.

#### 4. Funzioni principali:

- h(\$s) per htmlspecialchars.
- getXML(\$path, \$root) che crea file XML vuoto se non esiste (se viene passato \$root) e torna SimpleXMLElement o null con logging errori libxml.
- getSubscriptionStatus(\$data\_inizio, \$data\_scadenza, \$today=null):
  - ritorna 'scaduto' se data mancante, non valida o minore di oggi;
  - 'in\_scadenza' se mancano <= 7 giorni;
  - 'attivo' altrimenti.

5. Se \$id\_user carica da DB username, email, is\_admin, indirizzo, telefono con prepared statement; imposta \$is\_admin.

6. Carica reputazione da xml/reputazione.xml (se presente).

7. Cambio password (POST con change\_password):

- validazioni: campi non vuoti, esistenza utente, password\_verify corrente, lunghezza minima (8), almeno 1 maiuscola e 1 numero, conferma = nuovo.
- password\_hash() e UPDATE users SET password = ?, password\_temp = 0, force\_pw\_change = 0 WHERE id\_user = ? (prepared statement).
- setta \$\_SESSION['cp\_success'] o \$\_SESSION['cp\_error'] e redirect a profilo.php.

8. Aggiorna profilo (POST update\_profile):

- validazione campi non vuoti.
- controlli di unicità (username, email, telefono) con query di selezione WHERE ... AND id\_user != ? (prepared).
- se ok UPDATE users SET username = ?, email = ?, indirizzo = ?, telefono = ? WHERE id\_user = ?.
- aggiorna variabili per output e setta messaggio di successo/error.

9. Caricamento tipi abbonamento da xml/abbonamenti.xml per popolare.

10. Aggiunta abbonamento ( per ADMIN) (POST add\_subscription):

- validazione input.
- ricerca tipo abbonamento nel abbonamenti.xml (durata, prezzo, valuta, prezzo\_base, max\_prenotazioni).

- begin\_transaction() sul DB, crea nuovo utente (INSERT INTO users (...) VALUES (?, ?, ?, 0)), prende insert\_id.
- calcola data\_scadenza in base a durata\_mesi o fallback a +7 day.
- calcola stato via getSubscriptionStatus.
- carica user\_abbonamenti.xml (se non esiste lo crea), genera nuovo id robusto estraendo numeri esistenti e incrementando (aN).
- aggiunge nodo XML con tutti i campi (id\_user, id\_abbonamento, data\_inizio, data\_scadenza, stato, prezzo, valuta, prezzo\_base, max\_prenotazioni\_settimanali) e un blocco test\_convalida con prezzo\_atteso, conforme, differenza, note.
- salva XML con asXML() e fa commit() su DB; in caso di errore fa rollback() e setta \$\_SESSION['add\_error'].

11. Caricamento abbonamenti utente da xml/user\_abbonamenti.xml per costruire \$subs e ricalcolo stato con getSubscriptionStatus.

12. Mappatura utenti: estrae gli id\_user unici dai \$subs, costruisce una query SELECT id\_user, username FROM users WHERE id\_user IN (\$in) con la stringa implode(',', \$ids) .

13. Determinazione abbonamento corrente per utente loggato (non admin): cerca nel \$subs e imposta \$currentSubscription e \$tipoAbbonamento.

14. Statistiche admin:

- Valutazioni corsi: Media voti da feedback.xml aggregati per corso
- Reputazione utenti: Punteggi filtrati per abbonati attivi
- Heatmap forum: Thread ordinati per commenti (top 5)
- Utilizzo promo: Conteggio applicazioni promozioni
- Indice soddisfazione:
- $(\text{Media valutazioni} \times 0.4) + (\text{Media reputazione} \times 0.3) + (\text{Commenti forum} \times 0.3)$

15. Preparazione chat:

- Carica xml/private\_chat.xml.
- Se admin: raccoglie tutti i messaggi (filtra mittente/destinatario), costruisce \$messages con HTML per widget, conta non letti, ordina per timestamp, compone \$user\_chat\_content.
- Se utente (con abbonamento attivo): raccoglie messaggi rilevanti per quell'utente (admin->user e user->admin), compone widget e conta non letti.

- Costruisce mappa \$unread per admin (conteggi per mittente).

16.Pre-caricamento lista utenti per admin (SELECT id\_user, username FROM users).

17.Chiude connessione \$conn->close().

18.Output HTML: template con vari modals, tab admin, tabelle, badge di unread, script client (modals, ricerca, toggle password, validazione live, polling, notifiche, aggiornamento badge admin).

mark\_messages\_read.php

1.session\_start(), include db.php, imposta header JSON.

2.Se \$\_SESSION['id\_user'] vuoto ritorna { ok: false, changed: 0 }.

3.Recupera is\_admin dell'utente con prepared statement (SELECT is\_admin FROM users WHERE id\_user = ?).

4.Costruisce lista admin\_ids facendo SELECT id\_user FROM users WHERE is\_admin = 1.

5.Se xml/private\_chat.xml mancante ritorna { ok: false, changed: 0 }.

6.Carica file con DOMDocument, crea DOMXPath.

7.Se è admin e \$\_GET['user\_id'] è passato:

- cerca nodi //messaggio[@mittente='{target}' and @destinatario='{me}' and @risposta='0'] e imposta risposta='1'.
- se nessun cambiamento, prova a risolvere username del target (SELECT username) e ripete la query XPath con username escapata.

8.Se non admin: per ogni admin\_id cerca nodi //messaggio[@mittente='{admin\_id}' and @destinatario='{me}' and @risposta='0'] e setta risposta='1'.

9.Se changed > 0 salva in modo atomico (\$tmp = \$xmlPath . '.tmp', save(\$tmp), rename(\$tmp, \$xmlPath)), altrimenti restituisce { ok: true, changed: 0 }.

10.Restituisce JSON con ok e changed.

## **SCELTE TECNICHE**

- SimpleXML e DOMDocument+XPath: uso combinato per lettura veloce (SimpleXML) e modifiche XPath più robuste (DOMDocument) dove necessario.

- Prepared statements per la maggior parte delle query SQL sensibili (SELECT/UPDATE/INSERT).
- Atomic save per XML in mark\_messages\_read.php (scrittura su .tmp e rename) per evitare file corrotti su scrittura concorrente.

## GESTIONE DEGLI ERRORI

- getXML() usa libxml\_use\_internal\_errors(true) e logga errori di parsing; ritorna null se parsing fallisce.
- DB: la maggior parte dei prepared statements viene chiusa correttamente; nelle operazioni transazionali si usa begin\_transaction / commit / rollback e eccezioni per comunicare errori.
- Salvataggio XML: controllo di asXML(\$path) === false e lancio Exception in caso di errore (per add\_subscription).
- mark\_messages\_read.php ritorna sempre JSON coerente; in caso di errore di scrittura prova anche save(\$xmlPath) come fallback.
- Messaggi di errore/risultato sono passati tramite \$\_SESSION e mostrati nella UI dopo redirect (profilo.php).

## 12. [SCRIPT:save\\_feedback.php;modifica\\_lezione.php;cancella\\_lezione.php ;aggiungi\\_lezione.php](#)

### SCOPO

#### save\_feedback.php

Salva o aggiorna la valutazione (voto 1..5) che un utente lascia per una lezione. Prima di salvare effettua tutte le verifiche principali: utente autenticato, abbonamento attivo, lezione esistente e già conclusa, utente effettivamente prenotato (stato != 'cancellato') per la lezione. Effettua la scrittura su xml/feedback.xml in modo atomico.

Obiettivi principali raggiunti:

- parsing flessibile delle date (parseXmlDate) con più formati;
- verifica dello stato abbonamento leggendo xml/user\_abbonamenti.xml;
- verifica presenza lezione in xml/lezioni.xml e che la data sia passata;
- controllo che la prenotazione del relativo id\_user\_abbonamento non sia cancellata;
- scrittura atomica e concorrente-safe del file xml/feedback.xml tramite flock, ftruncate e fwrite.

### modifica\_lezione.php

Permette ad un amministratore di modificare i campi principali di una lezione (datetime\_lezione, posti\_totali) in xml/lezioni.xml. Effettua validazioni di input, verifica ruolo admin e salva in modo atomico con file temporaneo + rename.

Obiettivi principali:

- controllo di autenticazione e autorizzazione (is\_admin);
- validazione e normalizzazione della data (strtotime -> formato Y-m-d H:i:s);
- aggiornamento del nodo lezione corrispondente nell'XML;
- salvataggio atomico via .tmp + rename.

### cancella\_lezione.php

Permette ad un amministratore di rimuovere una lezione (id passato via GET) da xml/lezioni.xml. Dopo la cancellazione rimuove anche eventuali feedback collegati in xml/feedback.xml. Salvataggi atomici su file XML.

Obiettivi principali:

- controllo di autenticazione e autorizzazione (is\_admin);
- rimozione del nodo lezione via dom\_import\_simplexml e removeChild;
- persistenza atomica della lista lezioni;
- rimozione dei feedback collegati (se presenti) e salvataggio del file feedback.xml (non bloccante: log su fallimento).

### aggiungi\_lezione.php

Permette ad un amministratore di creare una nuova lezione collegata a un id\_corso\_base esistente (verificato su xml/corsi\_base.xml). Calcola un nuovo id progressivo e aggiunge il nodo lezione con prenotanti vuoto; salva atomico.

Obiettivi principali:

- controllo admin;
- validazione input (id corso base, data, posti);
- verifica esistenza corso base;
- creazione del nuovo nodo lezione con id progressivo;
- salvataggio atomico su xml/lezioni.xml.

## **DIPENDENZE**

- db.php
- File XML nella cartella xml/:
  - user\_abbonamenti.xml : usato in save\_feedback.php per verificare abbonamenti attivi;
  - lezioni.xml : usato/aggiornato in tutti gli script lezione;
  - feedback.xml: usato/aggiornato in save\_feedback.php e cancella\_lezione.php;
  - corsi\_base.xml :usato da aggiungi\_lezione.php per validare id\_corso\_base.
- Funzioni PHP: SimpleXML, DOMDocument, DOMXPath, flock, fopen, ftruncate, rename, strtotime, DateTime.
- Sessione utente (session\_start()).

## CAMPI DEL DB UTILIZZATI

Tabella users (usate direttamente o ricavate via DB):

- id\_user : identificatore sessione.
- is\_admin : flag (0/1)

La maggior parte dei dati dinamici (lezioni, prenotazioni, feedback, abbonamenti) è gestita via XML; il DB è usato quasi esclusivamente per la verifica del ruolo amministratore e per l'identità della sessione.

## WORKFLOW

save\_feedback.php

1.session\_start() + set timezone + header JSON.

2.Funzioni:

- respond(\$success, \$message) per output JSON;
- parseXmlDate() che prova molti formati (Y-m-d H:i:s, Y-m-d H:i, Y-m-d\TH:i:s, Y-m-d\TH:i, Y-m-d) e fallback a strtotime;
- getSimpleXML() con libxml\_use\_internal\_errors(true) e logging.

3.Verifica autenticazione: se non loggato -> respond(false, 'Utente non autenticato').

4.Legge input POST: id\_lezione (int), voto (int 1..5); validazione base.

5.Carica xml/user\_abbonamenti.xml e cerca un abbonamento attivo per l'id\_user corrente dove stato === 'attivo' e la data odierna è tra inizio e scadenza; ricava id\_abbonamento (attributo o elemento).

6.Se nessun abbonamento attivo -> errore.

7. Carica xml/lezioni.xml, cerca la lezione con attributo id == id\_lezione; se non trovata -> errore.

8. Converti la data lezione con parseXmlDate() e verifica che la lezione sia già conclusa (dt < now).

9. Controlla tra i prenotanti/prenotante se è presente un elemento con id\_abb uguale a id\_abbonamento e con stato != 'cancellato'.

10. Se partecipante ok -> procede a caricare/creare xml/feedback.xml:

- apre file feedback.xml in modalità c+, ottiene flock(LOCK\_EX);
- legge contenuto, costruisce DOMDocument (se file vuoto o corrotto crea root <feedback>);
- cerca una valutazione\_corso esistente per la stessa id\_lezione e id\_user\_abbonamento:
  - se trovata -> aggiorna <voto> e data\_valutazione;
  - altrimenti -> crea nuovo nodo valutazione\_corso con nuovo id progressivo, aggiunge id\_lezione, id\_user\_abbonamento, voto, data\_valutazione.
- serializza DOMDocument in XML, ftruncate, rewind, fwrite, fflush, flock(LOCK\_UN), fclose.

11. Restituisce JSON success: true o false con messaggio.

#### modifica\_lezione.php

1. session\_start() + set timezone; se utente non loggato -> redirect login.php.

2. Include db.php e ottiene \$id\_user.

3. Verifica admin via prepared statement SELECT is\_admin FROM users WHERE id\_user = ?.

4. Se non admin -> HTTP 403 + die.

5. Legge input POST: id\_lezione, datetime\_lezione, posti\_totali. Validazione base.

6. Valida data con strtotime (fallback semplice); se non valida -> die.

7. Carica xml/lezioni.xml con simplexml\_load\_file() (libxml errors logged).

8. Scorre nodi <lezione> e se trova il id corrispondente aggiorna datetime\_lezione e posti\_totali.

9. Salvataggio atomico: asXML(\$tmpFile) + rename su file originario; su errore elimina tmp e die.

10. Imposta \$\_SESSION['success\_message'] e redirect a corsi.php.

#### cancella\_lezione.php

1. session\_start() + timezone; verifica login.

2. Include db.php e verifica admin (is\_admin).

3. Legge \_GET['id'].

4. Carica xml/lezioni.xml tramite simplexml\_load\_file() (libxml checks).

5. Trova la lezione con id richiesto, rimuove il nodo usando dom\_import\_simplexml(\$lezione)->parentNode->removeChild(...).

6. Salvataggio atomico lezioni (.tmp + rename). Su errore aborta.

7. Se xml/feedback.xml esiste lo carica e rimuove tutti i nodi valutazione\_corso dove id\_lezione === id cancellata; tenta salvataggio atomico (se fallisce logga ma non blocca la cancellazione).

8. Setta \$\_SESSION['success\_message'] e redirect a corsi.php.

#### aggiungi\_lezione.php

1. session\_start() + timezone; verifica login.

2. Include db.php e verifica admin (is\_admin).

3. Legge POST: id\_corso\_base, datetime\_lezione, posti\_totali; validazione input.

4. Valida data con strtotime.

5. Controlla esistenza di corsi\_base.xml e che id\_corso\_base esista.

6. Carica o crea xml/lezioni.xml (se mancante crea <lezioni></lezioni>).

7. Calcola newId come max(existing ids) + 1.

8. Aggiunge nuovo nodo <lezione id="\$newId"> con figli id\_corso\_base, datetime\_lezione (formattata Y-m-d H:i:s), posti\_totali, prenotanti vuoto.

9. Salvataggio atomico: asXML(\$tmp) + rename.

10. Setta \$\_SESSION['success\_message'] e redirect a corsi.php.

### **SCELTE TECNICHE**

- SimpleXML per lettura e scrittura semplice di XML quando l'operazione è diretta e il file è sano.
- DOMDocument + DOMXPath in save\_feedback.php per editing e per poter misurare e ricercare nodi in modo preciso.

- Locking con flock() in save\_feedback.php per garantire che la scrittura su feedback.xml sia concorrente-safe.
- Validazione lato server (date, id numerici, range voto) e controllo ruolo admin con prepared statements.
- Fallback su file corrotto: quando parse XML fallisce si genera un nuovo documento (con logging); evita crash ma può causare perdita dati se file realmente corrotto .

### **GESTIONE DEGLI ERRORI**

- save\_feedback.php risponde sempre con JSON (success + message) e fa logging degli errori di parsing (libxml\_get\_errors()).
- Gli script admin (modifica, cancella, aggiungi) usano die() con messaggi semplici o redirect + session message per comunicare risultato all'UI.
- Operazioni di scrittura controllano esito di asXML(\$tmp) e rename; in caso di fallimento eliminano temp file e abortiscono.
- save\_feedback.php si assicura di rilasciare il lock (flock(LOCK\_UN)) e chiudere il file anche in caso di errore.
- Alcune situazioni (es. fallimento salvataggio feedback dopo cancellazione lezione) vengono loggate ma non bloccano l'operazione principale.

## **13. SCRIPT:**

[invio\\_mess\\_utente.php](#); [invio\\_mess\\_admin.php](#); [auto\\_responder.php](#); [get\\_chat\\_messages.php](#); [check\\_new\\_messages.php](#)

### **SCOPO**

[invio\\_mess\\_utente.php](#)

Gestisce l'invio di un messaggio dall'utente verso lo staff (mittente = username o id). Crea/aggiorna xml/private\_chat.xml, appende un nodo <messaggio> con metadati (id, mittente, destinatario, data, risposta, sollecito\_inviato) e il testo in CDATA. Salvataggio atomico tramite file temporaneo + rename. Dopo l'invio marca opzionalmente come lette le risposte admin->utente.

Obiettivi principali:

- verificare sessione utente e metodo POST;
- recuperare username dal DB per coerenza;
- creare struttura XML se mancante (CON DTD);
- append del messaggio con CDATA;
- salvataggio atomico del file chat;

### invio\_mess\_admin.php

Gestisce l'invio di un messaggio da parte di un amministratore verso un utente specifico. Verifica che il mittente sia admin, recupera username destinatario, appende il messaggio in xml/private\_chat.xml con attributo admin\_id. Inoltre marca come lette i messaggi dell'utente destinati ad admin (così non compaiono più come non letti).

Obiettivi principali:

- verifica autenticazione e ruolo admin via DB;
- risoluzione username destinatario da user\_id;
- append del messaggio admin->utente con admin\_id;
- mark-as-read automatico dei messaggi utente->admin;
- salvataggio sicuro con fallback (tmp+rename, poi file\_put\_contents LOCK\_EX).

### auto\_responder.php

Script da lanciare periodicamente (cron) che scansiona xml/private\_chat.xml per trovare messaggi destinati ad admin non ancora gestiti e, se sono passate almeno 48 ore senza risposta da parte di un admin, genera automaticamente una risposta admin (auto-reply) e marca il messaggio originale con sollecito\_inviato='1' per non ri-sollecitarlo.

Obiettivi principali:

- leggere chat XML e mapping id\_user->username;
- per ogni messaggio ad admin verificare se sono passate >=48h e non esistono risposte admin successive;
- creare un messaggio admin di risposta automatica e marcare il sollecito;
- salvataggio atomico dell'XML.

### get\_chat\_messages.php

Endpoint usato dagli admin per ottenere in HTML i messaggi scambiati con un singolo utente (user\_id). Carica xml/private\_chat.xml, filtra messaggi dove (mittente=user / destinatario=admin) o (mittente=admin / destinatario=user), ordina per data e renderizza frammenti HTML (con escaping) per il widget di chat admin.

Obiettivi principali:

- verifica permessi admin dal DB;
- risoluzione username per user\_id;
- query XPath per estrarre cronologia conversazione;

- output HTML sicuro (htmlspecialchars).

check\_new\_messages.php

Endpoint JSON polled dal frontend (admin o utente) che ritorna:

per admin: mappa user\_id => count di messaggi non letti (destinatario = admin & risposta = '0');

per utente: conteggio unread\_count di messaggi inviati da admin al suo username con risposta = '0'.

Obiettivi principali:

- verificare sessione e ruolo;
- leggere xml/private\_chat.xml in SimpleXML e contare messaggi non letti;
- restituire JSON con struttura adatta al polling frontend.

## DIPENDENZE

- db.php
- xml/private\_chat.xml : archivio conversazioni; creato se mancante.
- Sessione PHP (\$\_SESSION['id\_user']).
- Funzioni PHP usate: DOMDocument, DOMXPath, SimpleXML, uniqid, flock non sempre, file\_put\_contents(..., LOCK\_EX) come fallback.

## CAMPI DEL DB UTILIZZATI

Tabella users:

- id\_user : identificatore sessione.
- username : usato nei messaggi come mittente/destinatario testuale.
- is\_admin : flag (0/1) per autorizzare invio\_mess\_admin.php e get\_chat\_messages.php.

## WORKFLOW

invio\_mess\_utente.php

1. session\_start() e controllo \$\_SESSION['id\_user'] e metodo POST; altrimenti redirect a profilo.php.
2. Prendi \$uid e il testo \$\_POST['testo']. Se testo vuoto redirect.
3. Assicurati che xml/private\_chat.xml esista; se non c'è crea struttura base con DOMImplementation() e DTD.
4. Carica il DOM (DOMDocument->load(\$path)), se fallisce ricrea struttura base.
5. Recupera username dal DB (SELECT username FROM users WHERE id\_user = ?); fallback a id numerico se non trovato.

6. Crea nuovo nodo <messaggio> con attributi:
  - id = uniqid('m\_', true),
  - mittente = username,
  - destinatario = 'admin',
  - data = ora formattata,
  - risposta = '0',
  - sollecito\_inviato = '0'.
  - Inserisci <testo> come CDATA.
7. Append al root e salva in modo atomico: save(\$tmp) + rename(\$tmp,\$path).
8. Redirect a profilo.php.

#### invio\_mess\_admin.php

1. session\_start(), include DB; se sessione mancante o metodo diverso da POST => 403.
2. Verifica is\_admin e recupera admin\_username tramite prepared statement.
3. Leggi user\_id target e testo; se invalidi redirect.
4. Risolvi target\_username con query DB; se non trovato redirect a admin\_chat.php.
5. Assicurati che xml/private\_chat.xml esista o crealo (DTD).
6. Carica DOM; se fallisce ricrea struttura.
7. Crea <messaggio> con:
  - id = uniqid,
  - mittente = 'admin',
  - destinatario = \$target\_username,
  - data, risposta='0', sollecito\_inviato='0',
  - admin\_id = id admin,
  - <testo> in CDATA.
8. Append al root.
9. Marca i messaggi dell'utente destinati ad admin come risposta='1' (due xpath checks: per username e per id numerico).
10. Salvataggio sicuro: save(\$tmp) + rename; se rename fallisce fallback file\_put\_contents(..., LOCK\_EX).
11. Redirect a admin\_chat.php?user\_id=....

#### auto\_responder.php

1. Include DB; carica xml/private\_chat.xml con DOMDocument.
2. Costruisci mappa usersById (id\_user => username) per fallback.
3. Cerca messaggi @destinatario='admin' e non @sollecito\_inviato='1'.
4. Per ogni messaggio:

- parse data (robusto: Y-m-d H:i:s fallback strtotime);
- se sono passate >=48 ore dall'invio e non esiste risposta admin successiva -> creare messaggio auto-reply admin:
  - a. mittente='admin', destinatario = username se disponibile, data = now, sollecito\_inviato='1', admin\_id='0', testo di cortesia in CDATA.
- segnare l'originale sollecito\_inviato='1'.
- 5. Salvataggio atomico: save(\$tmp) + rename; fallback write con LOCK\_EX.

#### get\_chat\_messages.php

1. session\_start()
2. Leggi user\_id da GET; risolvi username.
3. Carica xml/private\_chat.xml via DOMDocument.
4. Esegui XPath che prende messaggi where:
  - a. (@destinatario = 'admin' and @mittente = '\$username') OR (@mittente = 'admin' and @destinatario = '\$username')
5. Ordina i nodi per data e per ciascuno:
  - a. estrai mittente, data, testo (testo sanitizzato con htmlspecialchars), determina se is\_unread e render HTML per widget.
6. Stampe frammento HTML (non JSON) per l'interfaccia admin.

#### check\_new\_messages.php

1. session\_start() e include DB; header JSON.
2. Recupera is\_admin e username del chiamante via prepared statement.
3. Se file chat mancante o SimpleXML non valido => ritorna struttura vuota ({ok:true, unread... zero}).
4. Se admin:
  - a. itera xml->messaggio e conta messaggi @risposta='0' && @destinatario='admin', raggruppandoli per mittente;
  - b. per ogni mittente prova a risolvere id\_user via username (o se mitt numeric cast);
  - c. restituisce JSON { ok: true, unread\_by\_user: { userId: count, ... } }.
5. Se utente normale:
  - a. conta messaggi @risposta='0' && @mittente='admin' && @destinatario = \$username e restituisce { ok: true, unread\_count: N }.

### **SCELTE TECNICHE**

- XML come datastore: tutti i messaggi sono conservati in xml/private\_chat.xml.

- DOMDocument + DOMXPath per manipolazione robusta e query XPath complesse (necessario per ricerca e marking).
- ID dei messaggi generati con uniqid('m\_', true) per unicità.
- Salvataggio atomico: preferito save(\$tmp) + rename(\$tmp,\$path); in fallback si usa file\_put\_contents(..., LOCK\_EX).
- Uso di username nel campo mittente/destinatario .

## GESTIONE DEGLI ERRORI

- Tutti gli script admin/utente controllano sessione e ruolo; ritornano redirect o 403 dove necessario.
- Parsing XML usa libxml\_use\_internal\_errors(true) e logga errori (error\_log); in caso di file corrotto si ricrea struttura base .
- Salvataggi controllano esito di save() e rename() e loggano fallback errori.
- invio\_mess\_utente.php e invio\_mess\_admin.php non fanno flock() prima di save(): si affidano a tmp+rename atomicità del filesystem, con fallback LOCK\_EX in invio\_mess\_admin.php.
- check\_new\_messages.php e get\_chat\_messages.php ritornano risposte pulite (JSON/HTML); in assenza di file restituiranno contatori vuoti o stringhe vuote senza crash.

## 14. SCRIPT:Admin\_chat.php

### SCOPO

Gestire l'interfaccia di amministrazione per la chat privata tra lo staff (admin) e un singolo utente.

Funzioni principali:

- Verificare che l'operatore sia autenticato e sia admin.
- Caricare i messaggi di chat (da un file XML) tra l'admin e l'utente selezionato.
- Segnare come "letti" i messaggi dell'utente destinati all'admin.
- Renderizzare l'intera conversazione in HTML (con escaping per prevenire XSS).

### DIPENDENZE

- db.php
- File XML di persistenza: xml/private\_chat.xml.
- CSS: style/style\_chat\_admin.css.
- Script esterni collegati (AJAX endpoints / back-end):
  - invio\_mess\_admin.php : riceve POST per inviare un messaggio dall'admin.

- get\_chat\_messages.php : restituisce fragment HTML dei messaggi (usato per aggiornare la chat).
- check\_new\_messages.php : restituisce JSON con conteggio messaggi non letti.
- Estensioni PHP usate: DOMDocument, DOMXPath, libxml e funzioni sessioni (session\_start()).
- Funzione h() definita nello script per escaping HTML.

## CAMPI DEL DB UTILIZZATI

Tabella users (usata in due query):

- id\_user (INT) : identificatore utente.
- username (VARCHAR) : nome mostrato nella chat.
- is\_admin (TINYINT / BOOLEAN) : flag se è admin.

## WORKFLOW

1. session\_start()
2. Verifica che esista \$\_SESSION['id\_user'], altrimenti 403 e exit.
3. Verifica in DB che l'utente di sessione abbia is\_admin = 1 (prepared statement). Se non è admin → 403.
4. Legge user\_id da \$\_GET e lo cast a intero; se non valido → exit.
5. Se xml/private\_chat.xml non esiste la crea con struttura minima (<chat>).
6. Carica il documento XML in DOMDocument, crea DOMXPath.
7. Recupera username dal DB per user\_id (prepared statement).
8. Con una query XPath segna come letti tutti i messaggi con:
  - @destinatario = 'admin' AND @mittente = '\$username' AND @risposta = '0'
  - Se cambiano nodi salva su file temporaneo e rename() sul file reale.
9. Recupera con XPath tutti i messaggi tra admin e \$username
10. Trasforma i nodi in array PHP, ordina per attributo data (usando strtotime) e li renderizza in HTML.
11. Fornisce la UI HTML con:
  - area section.chat-container che contiene i messaggi (con classi admin/user e unread),
  - form di risposta che invia a invio\_mess\_admin.php tramite fetch (JS),
  - polling JavaScript per aggiornare automaticamente la chat e controllare nuovi messaggi.

## SCELTE TECNICHE

- Uso di prepared statements per le query al DB (protezione da SQL injection).
- DOMDocument + XML come storage semplice per conversazioni
- Salvataggio su file .tmp e rename() per ridurre il rischio di file corrotto durante la scrittura (tentativo di salvataggio atomico).
- Escaping output con htmlspecialchars (funzione h()) per prevenire XSS sul front-end.
- Polling JS: semplice implementazione per ottenere aggiornamenti real-time senza socket/WebSocket.
- Cast a (int) per user\_id per mitigare injection via GET param.
- Separazione back-end / front-end: il rendering principale è fatto su server, ma gli aggiornamenti sono gestiti da endpoint separati.

### **DATI SALVATI IN SESSIONE**

1. \$\_SESSION['id\_user'] : usato per identificare l'account corrente e verificare se è admin.

### **GESTIONE DEGLI ERRORI**

2. Se l'utente non è loggato o non è admin restituisce 403 Forbidden e exit('Accesso negato').
3. Per l'XML usa libxml\_use\_internal\_errors(true) ma non stampa gli errori; eventuali errori di parsing non vengono notificati all'utente.
4. Quando user\_id non è valido (<=0) viene fatto exit('Utente non valido').
5. Salvataggio su file usa una tecnica .tmp + rename() ma non ha gestione esplicita di errori se save o rename falliscono.

## **15. [SCRIPT:toggle\\_admin.php,valuta\\_commento.php,mi\\_piace.php](#)**

### **SCOPO**

Gestire in modo sicuro e tracciabile le operazioni sul forum eseguite da utenti e amministratori : come bloccare o appuntare thread, valutare commenti e registrare consensi ,con l'obiettivo di promuovere la collaborazione, mantenere la qualità delle discussioni e proteggere l'integrità dei dati. Le azioni degli utenti sono collegate a un sistema di reputazione che riflette il loro contributo e può influire su privilegi e visibilità.

### **DIPENDENZE**

- db.php
- xml/forum.xml : archivio principale dei thread/commenti (creato/gestito a parte).

- xml/user\_abbonamenti.xml : (usato da valuta\_commento.php per verificare abbonamenti).
- Sessione PHP (\$\_SESSION['id\_user'], \$\_SESSION['username']).
- Funzioni PHP usate: DOMDocument, DOMXPath (ove opportuno), DateTime, uniqid, libxml\_use\_internal\_errors, file\_put\_contents(..., LOCK\_EX), rename, copy (backup).

## CAMPI DEL DB UTILIZZATI

Tabella users:

- id\_user : identificatore sessione.
- username : nome testuale usato nei controlli/attributi.
- is\_admin : flag (0/1) per autorizzare operazioni admin (usato in toggle\_admin.php).

## WORKFLOW

### toggle\_admin.php

1. session\_start() e header Content-Type: application/json.
2. Includere db.php.
3. Controllare \$\_SESSION['id\_user']; se mancante → errore JSON.
4. Parse payload JSON (file\_get\_contents('php://input')) e ottenere type, id, status.
5. Validare parametri; se mancanti → errore JSON.
6. Verificare ruolo admin tramite prepared statement (SELECT is\_admin, username FROM users WHERE id\_user = ?).
7. Controllare esistenza xml/forum.xml.
8. Caricare DOMDocument; se malformato → errore JSON.
9. Cercare <thread id="..."> iterando i nodi thread.
10. Applicare attributo richiesto (pinned o locked) con valore 'true'/'false' in base a status.
11. Salvataggio atomico: scrivi su \$xmlFile . 'tmp' con LOCK\_EX, poi rename; se fallisce fallback file\_put\_contents(\$xmlFile, \$xmlString, LOCK\_EX).
12. Restituire { success: true }.

### valuta\_commento.php

1. session\_start() e header JSON; disabilitare display\_errors e abilitare logging.
2. Includere db.php.
3. Leggere payload JSON; validare struttura.
4. Verificare autenticazione (\$\_SESSION['username'], \$\_SESSION['id\_user']).
5. Validare thread\_id, comment\_id, utilita, accordo (numerici e range 1-5).
6. Chiamare user\_has\_active\_subscription(\$id\_user) che:
  - Carica xml/user\_abbonamenti.xml.
  - Scansiona nodi <abbonamento\_utente>, confronta id, stato, e date (data\_inizio/data\_scadenza) con DateTimeImmutable.
7. Se non abbonato → errore JSON.

8. Verificare esistenza e permessi su xml/forum.xml (readable/writable).
9. Caricare DOMDocument e gestire errori libxml; se malformato → errore con messaggi di libxml (log).
10. Trovare thread e commento iterando.
11. Se thread locked="true" → errore.
12. Estrarre autore del commento e impedire auto-valutazione.
13. Estrarre data del commento, parse robusto (DateTime o fallback strtotime) e calcolare differenza in giorni; se > 30 → errore.
14. Trovare o creare nodo <valutazioni\_commento>.
15. Controllare duplicato valutazione per lo stesso valutatore (case-insensitive).
16. Creare nodo <valutazione valutatore="..."> + figli <utilita>, <accordo>, <data\_valutazione>.
17. Fare backup \$xmlFile . '.bak\_YYYYmmdd\_HHis' con copy.
18. Salvataggio atomico (.tmp + rename) con fallback scrittura LOCK\_EX.
19. Restituire JSON { success: true, message: 'Valutazione registrata' }.

#### mi\_piace.php

1. session\_start() e header JSON.
2. Verificare autenticazione (\$\_SESSION['username']).
3. Parse payload JSON (thread\_id, comment\_id) e validare.
4. Controllare accessibilità xml/forum.xml (read/write).
5. Caricare DOMDocument; se malformato → errore JSON.
6. Trovare thread e commento iterando i nodi.
7. Recuperare o creare nodo <consensi> figlio.
8. Controllare duplicato (case-insensitive): se presente → errore JSON "Hai già messo il pollice!".
9. Creare <consenso> con createTextNode(\$username) e appenderlo.
10. Salvataggio atomico (.tmp + rename) con fallback file\_put\_contents(..., LOCK\_EX).
11. Costruire array \$likedUsers iterando i figli <consenso> e restituirlo in JSON { success: true, liked\_users: [...] }.

#### **SCELTE TECNICHE**

- XML come datastore: semplicità per file-based forum; DOMDocument/DOMXPath per robustezza e manipolazioni strutturate.
- Salvataggio atomico: preferenza per tmp + rename ,con fallback file\_put\_contents(..., LOCK\_EX) per scenari in cui rename fallisce.
- Iterazione invece di XPath diretto quando il valore proviene dall'esterno, per ridurre rischi di injection XPath e problemi di encoding.
- Backup prima di operazioni distruttive (valuta\_commento.php) per agevolare recovery in caso di corruption.

- Parsing date robusto usando DateTime con fallback strtotime per coprire formati legacy/eterogenei.
- Verifiche server-side (abbonamento, ruolo, finestra temporale) — non fidarsi solo del frontend.
- Confronti case-insensitive per username per uniformità.

## GESTIONE DEGLI ERRORI

- Tutti gli script:
  - Validano sessione e parametri; ritornano JSON coerente in caso di errore.
  - Usano libxml\_use\_internal\_errors(true) e, in caso di malformato XML, raccolgono e loggano gli errori prima di restituire un messaggio utile .
  - Effettuano controlli di accessibilità file (is\_readable / is\_writable) e segnalano permessi errati.
  - Per salvataggio: verificano esito di file\_put\_contents e rename, e applicano fallback; in caso di tutti i fallimenti ritornano errore JSON.
- valuta\_commento.php:
  - Effettua backup (.bak\_YYYYmmdd\_HHis) prima di modificare il file.
  - Fornisce messaggi di errore dettagliati ,e logga dettagli per debugging.
- mi\_piace.php e toggle\_admin.php:
  - Risposte standardizzate in JSON { success: false, message: '...' }.
  - Protezione contro duplicati/azioni invalide e controllo di ruolo dove necessario (toggle\_admin.php).

## 16. SCRIPT: superadmin.php

### SCOPO

Fornire un'interfaccia di amministrazione superiore (super admin) per gestire privilegi utente e visualizzare informazioni utente (inclusa la reputazione eventualmente letta da XML).

Restringe l'accesso al solo utente root con is\_admin = 1, permette la promozione/revoca di admin tramite form protetti da CSRF e mostra una tabella di utenti (escludendo root dalla modifica).

### DIPENDENZE

- db.php (connessione al DB).
- File XML opzionale: xml/reputazione.xml (lettura per sovrascrivere/reintegrare valori di reputazione).
- Sessione PHP (\$\_SESSION['id\_user'], \$\_SESSION['username'], \$\_SESSION['is\_admin'], \$\_SESSION['csrf\_token'], \$\_SESSION['super\_status']).
- Funzioni PHP usate: htmlspecialchars, simplexml\_load\_file, libxml\_use\_internal\_errors, libxml\_clear\_errors.
- CSS/risorse locali per layout (es. style/style\_faq.css).

## WORKFLOW

1. `session_start()`; include `db.php` con `require_once`.
2. Definizione helper `h()` che applica `htmlspecialchars(..., ENT_QUOTES, 'UTF-8')`.
3. Controllo di autenticazione e permessi: verifica che `$_SESSION['id_user']`, `$_SESSION['username']`, `$_SESSION['is_admin']` esistano e che `$_SESSION['username'] === 'root'` e `is_admin === 1`. Se fallisce, invia 403 Forbidden ed esce.
4. CSRF: se non esiste `$_SESSION['csrf_token']` ne crea uno sicuro con `bin2hex(random_bytes(32))`.
5. Recupera eventuale messaggio di stato in `$_SESSION['super_status']` e lo rimuove dalla sessione per visualizzare feedback.
6. Carica `xml/reputazione.xml` se esiste:
  - abilita internal errors (`libxml_use_internal_errors(true)`), `simplexml_load_file(..., LIBXML_NOCDATA | LIBXML_NONET)`.
  - itera i nodi `<utente>`, legge attributo `id` e figlio `<punteggio>`; normalizza e popola array `$reputations[(int)$id]`.
  - pulisce errori libxml (`libxml_clear_errors()`).
7. Query DB: prepara `SELECT id_user, username, email, is_admin, reputazione FROM users ORDER BY id_user ASC` ed esegue.
  - per ogni riga, sovrascrive il campo `reputazione` con il valore presente nell'XML (se esiste) mantenendo formati numerici.
  - accumula righe in `$users`.
8. Se la `prepare()` fallisce, imposta `$_SESSION['super_status']` con errore e redirect alla stessa pagina.
9. Render HTML: tabella utenti (esclude root dal loop), mostra pulsanti/form per promuovere/revocare (invia a `promuovi_utente.php`), include token CSRF nascosto e conferma JS `confirm(...)`.
10. Link di ritorno al sito.

## SCELTE TECNICHE

- Protezione accesso: verifica stringa `username === 'root' + is_admin` nel valore di sessione per un controllo forte sull'accesso alla pagina.
- CSRF token di sessione creato con `random_bytes` per proteggere azioni POST.
- Lettura reputazioni da XML separato (`xml/reputazione.xml`) per mantenere dati di reputazione esterni al DB e consentire override temporanei senza toccare il DB.
- Uso di `LIBXML_NOCDATA | LIBXML_NONET` in `simplexml_load_file` per sicurezza (evita fetch esterni) e conservazione CDATA.
- Helper `h()` centralizza escaping HTML (coerenza e riduzione di errori XSS).
- Sovrascrittura della reputazione a runtime: mantiene il DB come fonte di verità ma permette override dall'XML per amministrazione/sperimentazione.

- UI: esclusione del superadmin root dalla lista di azioni per prevenire logout accidentali.

## GESTIONE DEGLI ERRORI

- Autenticazione/Autorizzazione: in caso di accesso non autorizzato ritorna HTTP/1.1 403 Forbidden e interrompe l'esecuzione.
- XML: usa libxml\_use\_internal\_errors(true) e libxml\_clear\_errors(); non tenta di correggere XML malformato, semplicemente ignora l'XML non valido (può lasciare la reputazione a valori DB).
- DB: verifica il risultato di prepare(); se fallisce imposta messaggio di errore in \$\_SESSION['super\_status'] e fa redirect (fallback visibile all'admin).
- Input/output: escape dell'output HTML con h() per prevenire XSS.
- Azioni critiche (promozione/revoca) richiedono conferma JS nel client e CSRF token nel POST per ridurre rischi di CSRF/CSRF-forged requests.
- Possibili miglioramenti non implementati nel codice: logging centralizzato degli errori DB/XML, controllo più esplicito dei permessi file, gestione dei casi di lock DB e rollback di operazioni multi-step.

## 11. SCRIPT: profilo\_pubblico.php

### SCOPO

Mostrare il profilo pubblico di un utente identificato da id GET: visualizza username, email, flag admin, indirizzo/telefono se presenti e la reputazione (presa dal DB ma eventualmente sovrascritta da xml/reputazione.xml se presente).

### DIPENDENZE

- db.php (connessione al DB).
- File XML opzionale: xml/reputazione.xml (fornisce punteggio reputazione per alcuni utenti).
- Sessione PHP opzionale (non obbligatoria per la visualizzazione pubblica).
- Funzioni PHP usate: htmlspecialchars, simplexml\_load\_file, libxml\_use\_internal\_errors, libxml\_clear\_errors.
- Risorse CSS locali (style/style\_faq.css).

### WORKFLOW

1. session\_start(); include db.php.
2. Helper h() definito per htmlspecialchars(..., ENT\_QUOTES, 'UTF-8').
3. Legge id da \$\_GET['id'] e lo normalizza con (int). Se id <= 0 ritorna 400 Bad Request e termina.
4. Prepara statement SQL SELECT id\_user, username, email, is\_admin, indirizzo, telefono, reputazione FROM users WHERE id\_user = ?.

- in caso di prepare() fallito risponde 500 Internal Server Error.
5. Esegue la query con bind\_param('i', \$id), execute(), get\_result() e fetch\_assoc(). Se l'utente non esiste risponde 404 Not Found.
  6. Percorso XML xml/reputazione.xml: se il file esiste e leggibile:
    - abilita errori libxml interni, carica con simplexml\_load\_file(..., LIBXML\_NOCDATA | LIBXML\_NONET).
    - scorre <utente> e confronta attributo id con l'id dell'utente; se trova un <punteggio> numerico lo usa per sovrascrivere \$reputazione.
    - libxml\_clear\_errors().
  7. Render HTML della pagina profilo: tutti i campi testuali stampati tramite h(); reputazione formattata se numerica con number\_format.
  8. Fornisce link di ritorno a superadmin.php.

## SCELTE TECNICHE

- Validazione server-side dell'id in GET con cast (int) per evitare injection dirette o richieste non valide.
- Prepared statement per recupero utente dal DB (protezione SQL injection).
- Lettura opzionale di xml/reputazione.xml per permettere override della reputazione senza modificare il DB (stessa logica di superadmin.php).
- Uso di LIBXML\_NOCDATA | LIBXML\_NONET per sicurezza durante il parsing XML.
- Escaping centralizzato tramite h() per prevenire XSS nell'output.
- Restituzione di HTTP status appropriati (400, 404, 500) per facilitare diagnostica e client handling.

## GESTIONE DEGLI ERRORI

- Parametri: se id non valido → HTTP/1.1 400 Bad Request e messaggio semplice; evita esposizione di dettagli interni.
- DB: se prepare() fallisce invia 500 Internal Server Error con messaggio generico; se l'utente non esiste invia 404 Not Found.
- XML: libxml\_use\_internal\_errors(true) e libxml\_clear\_errors() usati per sopprimere warning, il codice non interrompe l'esecuzione se l'XML è mancante o malformato (fallback al valore DB).
- Output: tutto il testo stampato è escapato con h() per ridurre rischio XSS.
- Possibili miglioramenti consigliati (non presenti nel file):
- logging server-side degli errori DB/XML per debugging;
- rate-limit o caching della lettura XML per performance;
- validazione aggiuntiva dei campi (email, telefono) prima del display se necessario.

## Capitolo 4 – Elenco utenti

| Username | Password | Ruolo                    |
|----------|----------|--------------------------|
| -        | -        | visitatore               |
| mario    | mario    | admin                    |
| root     | root     | superadmin               |
| luisa    | luisa    | Utente abbonato          |
| alice    | alice    | Utente senza abbonamento |